

TÀI LIỆU KIẾN TRÚC PHẦN MỀM

Tài liệu thiết kế và kiến trúc phần mềm

CA/BQP Verification Platform

ca-bqp-verification-platform

*Nền tảng
xác minh và hỗ trợ tra cứu nhân sự dành cho bộ quốc phòng/ bộ công an.*

Phiên bản tài liệu	1.0
Ngày phát hành	19/09/2026
Trạng thái	Proof of Concept (POC)
Phạm vi	Toàn hệ thống: backend, frontend, worker, hạ tầng
Đối tượng đọc	Kiến trúc sư, lập trình viên, QA, vận hành, an toàn thông tin

Tài liệu này mô tả kiến trúc đang được hiện thực trong mã nguồn. Mọi phát biểu đều đối chiếu trực tiếp với tệp nguồn được dẫn chiếu; những phần chưa kiểm chứng được nêu rõ trong Chương 15.

Mục lục

1	Giới thiệu	1
1.1	Mục đích tài liệu	1
1.2	Phạm vi	1
1.3	Định nghĩa và thuật ngữ	1
1.4	Tài liệu tham chiếu	2
1.5	Cách đọc tài liệu	3
2	Biểu diễn kiến trúc	4
2.1	Khung biểu diễn	4
2.2	Nguyên tắc kiến trúc chủ đạo	4
2.3	Ngữ cảnh hệ thống	5
2.4	Luồng nghiệp vụ lõi	5
2.5	Công nghệ nền	5
3	Mục tiêu và ràng buộc kiến trúc	7
3.1	Bất biến nghiệp vụ	7
3.2	Ràng buộc kỹ thuật	8
3.3	Thuộc tính chất lượng ưu tiên	9
4	Khung nhìn Use-case	10
4.1	Tác nhân và mô hình quyền	10
4.2	Các ca sử dụng chính	10
4.2.1	UC-01: Tra cứu một hồ sơ	11
4.2.2	UC-02: Nạp danh sách theo lô	11
4.2.3	UC-03: Thẩm định thủ công	11
4.2.4	UC-04: Quản trị danh mục đơn vị / nhân sự	11
4.2.5	UC-05: Quản trị tài khoản	11
4.2.6	UC-06: Tra nhật ký kiểm toán	12
4.3	Bản đồ màn hình	12
4.4	Ngữ nghĩa trạng thái hiển thị	12
5	Khung nhìn logic	14
5.1	Phân tầng	14
5.2	Các mô-đun nghiệp vụ	15
5.3	Hạt nhân điều phối: <code>process_case</code>	15

5.3.1	Tra cứu không có quan hệ đơn vị	16
5.4	Hai nguồn thẩm quyền tách bạch	17
5.5	Frontend	18
6	Khung nhìn tiến trình	19
6.1	Toàn cảnh một Case	19
6.2	Idempotency	20
6.3	Bàn giao bền vững qua Outbox	21
6.4	Cấu hình worker	22
6.4.1	Profile docker: Celery	22
6.4.2	Profile local: runner in-process	22
6.5	Đồng thời và tranh chấp	23
6.6	Xử lý lỗi	23
6.7	Vòng đời Case	24
7	Khung nhìn triển khai	25
7.1	Hai profile, một mã nguồn	25
7.2	Profile local	26
7.3	Profile docker	27
7.4	Phát triển trên host không rebuild image	28
7.5	Cổng sức khoẻ	29
7.6	Phục vụ SPA và biên mạng	29
8	Khung nhìn dữ liệu	30
8.1	Nhóm thực thể	30
8.2	Version hoá và ảnh chụp bất biến	32
8.2.1	Chuẩn hoá khoá	33
8.3	Ràng buộc quan trọng	34
8.4	Khối bằng chứng	35
8.5	Dữ liệu danh mục và seed	35
8.6	Migration	36
9	Hiểu tài liệu (Document Intelligence)	37
9.1	Định tuyến đầu vào	37
9.2	OCR	39
9.2.1	Các engine đã bị gỡ bỏ	40
9.2.2	VietOCR được vendor	40
9.2.3	Tiền xử lý ảnh	41
9.3	Trích xuất	41

9.4	Tách nhiều chủ thể	43
9.5	Gate chất lượng và ngưỡng	44
9.6	Baseline đo lường của pipeline tiền xử lý	44
9.7	Hạn mức tài nguyên và an toàn tập	46
10	Phân giải và quản trị danh mục	47
10.1	Cascade phân giải đơn vị	48
10.1.1	Xử lý mã đơn vị	49
10.1.2	Xếp hạng lai	49
10.1.3	Calibration	50
10.2	Phân giải nhân sự	50
10.3	Quản trị danh mục	51
10.3.1	Luồng dữ liệu vào danh mục	51
10.3.2	Cường chế vòng đời	51
10.4	Tra cứu đơn vị trực tiếp	51
11	Quyết định, chính sách và thẩm định	52
11.1	Phân loại nhóm đối tượng	52
11.2	Bộ máy chính sách	52
11.3	Thang điều kiện thẩm định	54
11.4	Quy trình thẩm định thủ công	55
11.4.1	Ủy quyền	56
11.4.2	Bốn quyết định	56
11.4.3	Đồng thời	56
11.5	Nạp danh sách theo lô	57
11.5.1	Suy luận ánh xạ cột	58
11.5.2	Hạn mức và tính nhất quán	58
12	Kiến trúc bảo mật	59
12.1	Định danh và phiên	59
12.1.1	Một phiên hoạt động	60
12.1.2	Tính duy nhất của định danh tài khoản	60
12.2	Mã đăng nhập một lần	61
12.3	Vận chuyển và chống lạm dụng	61
12.3.1	Header bảo mật	61
12.3.2	Giới hạn tần suất	62
12.3.3	Xác định địa chỉ client	62
12.4	Thư đi	63
12.5	Mã hoá đầu ra và hình dạng phản hồi	63

12.6	Kiểm toán và nhật ký	64
12.7	Rào chắn production	64
13	Chiến lược chất lượng và kiểm thử	65
13.1	Nguyên tắc kiểm thử	65
13.2	Golden suites	65
13.3	Cấu hình kiểm thử backend	66
13.4	Kiểm thử frontend và E2E	66
13.5	Pipeline CI	67
13.6	Cổng phát hành	67
14	Vận hành và giám sát	69
14.1	Lệnh vận hành	69
14.1.1	Profile local	69
14.1.2	Target Makefile	69
14.2	Số liệu Prometheus	69
14.3	Khả năng mở rộng, độ trễ và độ tin cậy	70
14.4	Nhật ký	71
14.5	Runbook	71
14.6	Chẩn đoán các trục trặc đã gặp	72
15	Rủi ro, nợ kỹ thuật và ranh giới	73
15.1	Ranh giới production	73
15.2	Những gì chưa được kiểm chứng	73
15.3	Nợ kỹ thuật và giới hạn đã biết	73
15.4	Rủi ro kiến trúc	74
15.5	Hướng phát triển	75
A	Phụ lục A — Danh mục API	76
A.1	Xác thực	76
A.2	Hồ sơ và tra cứu	76
A.3	Nạp theo lô và thẩm định	77
A.4	Quản trị danh mục đơn vị	77
A.5	Quản trị danh mục nhân sự	78
A.6	Quản trị tài khoản và kiểm toán	78
A.7	Sức khoẻ và số liệu	78
A.8	Header được chấp nhận	79
B	Phụ lục B — Cấu hình	80
B.1	Profile và hạ tầng	80

B.2	Xác thực, phiên và OTP	80
B.3	Vận chuyển và chống lạm dụng	81
B.4	Ngưỡng phân giải	81
B.5	Ngưỡng tài liệu và OCR	82
B.6	Nạp theo lô	83
B.7	Phiên bản đóng dấu lên kết quả	83
B.8	Mail và quét mã độc	84
C	Phụ lục C — Mã nguồn sơ đồ Mermaid	85
C.0.1	01-context.mmd — Sơ đồ ngữ cảnh	85
C.0.2	02-containers.mmd — Sơ đồ container, profile docker	85
C.0.3	03-local-profile.mmd — Profile local	86
C.0.4	04-backend-components.mmd — Thành phần backend	86
C.0.5	05-case-sequence.mmd — Trình tự xử lý một Case	87
C.0.6	06-input-routing.mmd — Định tuyến đầu vào	88
C.0.7	07-ocr-pipeline.mmd — Pipeline OCR	89
C.0.8	08-subject-split.mmd — Tách nhiều chủ thể	89
C.0.9	09-resolution-cascade.mmd — Cascade phân giải đơn vị	89
C.0.10	10-person-resolution.mmd — Cascade phân giải nhân sự	90
C.0.11	11-decision-ladder.mmd — Thang điều kiện thẩm định	90
C.0.12	12-case-state.mmd — Máy trạng thái Case	91
C.0.13	13-registry-lifecycle.mmd — Vòng đời version danh mục	91
C.0.14	14-erd-core.mmd — Mô hình dữ liệu luồng Case	92
C.0.15	15-erd-registry.mmd — Mô hình dữ liệu danh mục và hạ tầng	93
C.0.16	16-review-sequence.mmd — Trình tự thẩm định	95
C.0.17	17-bulk-flow.mmd — Luồng nạp theo lô	95
C.0.18	18-auth-sequence.mmd — Trình tự xác thực	96
C.0.19	19-outbox.mmd — Bàn giao bền vững qua outbox	96
C.0.20	20-deployment.mmd — Sơ đồ triển khai	97
C.0.21	21-benchmark-base-quality.mmd — Benchmark chất lượng tiền xử lý cấu hình Base	97

Danh sách hình vẽ

2.1	Sơ đồ ngữ cảnh: tác nhân người dùng, hệ thống ngoài và ranh giới nền tảng . . .	5
5.1	Thành phần backend: router mởng, dịch vụ nghiệp vụ, hạ tầng dùng chung, worker	14
5.2	PersonResolver: cascade định danh riêng, luôn trả quyền quyết định tổ chức về Resolver đơn vị	17
6.1	Trình tự xử lý một Case từ tệp tải lên đến kết quả	19
6.2	Ngữ nghĩa bàn giao bền vững khác nhau giữa hai profile	21
6.3	Máy trạng thái của Case	24
7.1	Profile local: một tiến trình, chỉ phụ thuộc PostgreSQL	26
7.2	Sơ đồ container ở profile docker	27
7.3	Sơ đồ triển khai: dịch vụ, volume và biên TLS/WAF	28
8.1	Mô hình dữ liệu luồng Case	31
8.2	Mô hình dữ liệu hai danh mục và hạ tầng dùng chung	32
8.3	Vòng đời version danh mục, áp dụng độc lập cho hai danh mục	33
9.1	route_input: quyết định chiến lược đọc <i>trước</i> khi parse	37
9.2	Pipeline OCR: một engine chính, đúng một fallback được cấu hình	39
9.3	Tách nhiều chủ thể và fan-out thành các Case anh em	43
9.4	Độ chính xác fixture tiền xử lý tiếng Việt của cấu hình Base	45
10.1	Cascade phân giải tên/mã đơn vị	48
11.1	Thang điều kiện đưa Case sang NEED_REVIEW và lý do tương ứng	54
11.2	Trình tự ra quyết định thẩm định	55
11.3	Luồng nạp danh sách theo lô	57
12.1	Đăng nhập bằng mật khẩu và bằng mã một lần	59

Danh sách bảng

1.1	Phạm vi hệ thống	1
4.2	Các preset quyền	10
9.1	Độ chính xác tiền xử lý của cấu hình Base	44
9.2	Kết quả tổng hợp của cấu hình Base	45
13.2	Kết quả bộ test theo backend CSDL	66

CHƯƠNG 1

Giới thiệu

1.1 Mục đích tài liệu

Tài liệu này mô tả kiến trúc phần mềm của CA/BQP Verification Platform — hệ thống nội bộ tiếp nhận văn bản hoặc tệp hồ sơ, trích xuất thông tin đối tượng và đơn vị công tác hiện tại, phân giải đơn vị đó về danh mục chuẩn (Master Unit Registry), kết luận đơn vị thuộc Bộ Công an (B CA), Bộ Quốc phòng (BQP), tổ chức khác (OTHER) hay chưa xác định (UNKNOWN), xác định nhóm đối tượng, chạy bộ quy tắc chính sách có version, và chuyển sang thẩm định thủ công khi cần cứ không đủ.

Tài liệu phục vụ bốn nhu cầu:

1. **Định hướng hiện thực:** mô tả ranh giới thành phần, hợp đồng giữa các mô-đun, và những bất biến mà bất kỳ thay đổi nào cũng phải giữ nguyên.
2. **Cơ sở thẩm định:** giải thích *vì sao* hệ thống chọn từ chối (abstain) thay vì đoán, và những chỗ nào trong mã nguồn cưỡng chế điều đó.
3. **Bàn giao vận hành:** mô tả hai profile triển khai, các cổng sẵn sàng, số liệu giám sát và quy trình runbook.
4. **Quản trị rủi ro:** ghi nhận rõ ranh giới giữa những gì đã kiểm chứng và những gì còn là giả định (Chương 15).

1.2 Phạm vi

Bảng 1.1. Phạm vi hệ thống

Thuộc phạm vi	Tiếp nhận text/tệp; định tuyến tài liệu; OCR và trích xuất; tách nhiều chủ thể; phân giải đơn vị và phân giải nhân sự; phân loại nhóm đối tượng; bộ máy chính sách; hàng đợi thẩm định thủ công; nạp danh sách theo lô; quản trị hai danh mục có version; tài khoản, phiên và phân quyền nội bộ; nhật ký kiểm toán chỉ-ghi-thêm; kết xuất bằng chứng.
Ngoài phạm vi	Nhà cung cấp định danh ngoài (OIDC/SSO — đã bị gỡ bỏ có chủ đích); thiết bị WAF lớp 7; quản lý bí mật tập trung; kết thúc TLS; hệ thống nghiệp vụ nguồn cấp dữ liệu gốc; tính toán số tiền chế độ chính sách (hệ thống chỉ kết luận <i>phạm vi áp dụng</i>).

1.3 Định nghĩa và thuật ngữ

Thuật ngữ	Ý nghĩa trong hệ thống
Case	Một đơn vị công việc xác minh cho <i>một</i> chủ thể. Mọi đầu vào cuối cùng đều quy về Case: một đoạn text, một tệp hồ sơ, một block chủ thể được tách ra, hoặc một dòng trong danh sách nạp theo lô.
CURRENT_WORK_UNIT	Đơn vị công tác <i>hiện tại</i> của chủ thể. Đây là thực thể được phân giải, không phải tổ chức đầu tiên xuất hiện trong văn bản.
MasterUnitRegistry	Danh mục đơn vị chuẩn, có version, có QA, có ảnh chụp bất biến. Nguồn thẩm quyền duy nhất cho organization_type.
PersonRegistry	Danh mục nhân sự song song, version độc lập. Chỉ trả về canonical_unit_id; không bao giờ tự quyết định tổ chức.
Resolution	Kết quả phân giải tên/mã đơn vị: MATCHED, AMBIGUOUS, NOT_FOUND, CONFLICT.
organization_type	BCA / BQP / OTHER / UNKNOWN.
subject_group	Nhóm đối tượng: CAND, QUAN_NHAN, CO_YEU_HUONG_LUONG_NHU_QUAN_NHAN.
Abstention	Hành vi từ chối kết luận khi căn cứ không đủ; biểu hiện bằng NEED_REVIEW, AMBIGUOUS, NOT_FOUND hoặc INSUFFICIENT_DATA.
coverage_group	Phạm vi phụ trách của cán bộ thẩm định. Chỉ <i>thu hẹp</i> , không bao giờ cấp thêm quyền.
source_kind	OFFICIAL / PROVIDED / SYNTHETIC_DEMO. Dữ liệu tổng hợp luôn phải mang nhãn SYNTHETIC_DEMO.
Evidence	Khối JSON lưu cùng kết quả, chứa mọi đầu vào quyết định và số version của registry / taxonomy / parser / model / threshold / policy.
Outbox	Bảng outbox_events bảo đảm bàn giao bền vững job xử lý tài liệu, độc lập với broker.

1.4 Tài liệu tham chiếu

Tệp	Nội dung
PRODUCTION_QUALITY_2026.md	Định hướng sản phẩm và tiêu chí chất lượng.
CLAUDE.md	Bất biến kiến trúc, quy ước mã nguồn, bày đã gặp.
README.md	Tổng quan vận hành và cách chạy nhanh.
BACKEND_E2E_IMPLEMENTATION.md	Ghi chú hiện thực backend.
FRONTEND_IMPLEMENTATION.md	Ghi chú hiện thực frontend.

Tệp	Nội dung
INPUT_INTAKE_BUSINESS_ANALYSIS_Phasis	Phân tích nghiệp vụ tiếp nhận đầu vào.
RELEASE_GATES.md	Cổng phát hành.
POC_VALIDATION_2026-09-17.md	Kết quả kiểm chứng POC thực tế.
SECURITY_HARDENING_TEST_2026-09-17.md	Kiểm thử gia cố an toàn.
ops/runbooks/*.md	Runbook backup/restore, deploy/rollback, sự cố, cập nhật danh mục.

1.5 Cách đọc tài liệu

Chương 2 nêu khung biểu diễn. Chương 3 là phần *bắt buộc đọc* đối với mọi người sửa mã: nó liệt kê các bất biến nghiệp vụ. Các Chương 4–8 trình bày năm khung nhìn kiến trúc. Các Chương 9–11 đi sâu vào ba khối nghiệp vụ lõi. Các Chương 12–14 là thuộc tính chất lượng và vận hành. Chương 15 ghi rủi ro và nợ kỹ thuật.

Toàn bộ sơ đồ trong tài liệu được viết bằng **Mermaid**; mã nguồn sơ đồ nằm trong `diagrams/*.mmd` và được in đầy đủ tại Phụ lục C, nên tài liệu tự chứa ngay cả khi chưa dựng hình.

CHƯƠNG 2

Biểu diễn kiến trúc

2.1 Khung biểu diễn

Tài liệu dùng mô hình **4+1 view** của Kruchten, mở rộng thêm khung nhìn dữ liệu vì hệ thống là một nền tảng dữ liệu có version:

Khung nhìn	Trả lời câu hỏi	Chương
Use-case	Ai dùng hệ thống, để làm gì, với quyền nào?	Ch. 4
Logic	Hệ thống chia thành mô-đun nào, phụ thuộc ra sao?	Ch. 5
Tiến trình	Việc chạy ở tiến trình nào, đồng bộ hay bất đồng bộ, bền vững thế nào?	Ch. 6
Triển khai	Chạy ở đâu, cần hạ tầng gì, hai profile khác nhau ra sao?	Ch. 7
Dữ liệu	Lưu gì, ràng buộc gì, version hoá thế nào?	Ch. 8

2.2 Nguyên tắc kiến trúc chủ đạo

Evidence-driven Mỗi kết luận đi kèm khối *evidence* đầy đủ và số version của mọi thành phần đã tham gia quyết định, đủ để phát lại (replay) kết luận đó về sau.

Registry-first Không có luật suy đoán nào thay thế được danh mục. Kết luận `organization_type` chỉ đến từ một đơn vị đã phân giải trong ảnh chụp danh mục đang `PUBLISHED`.

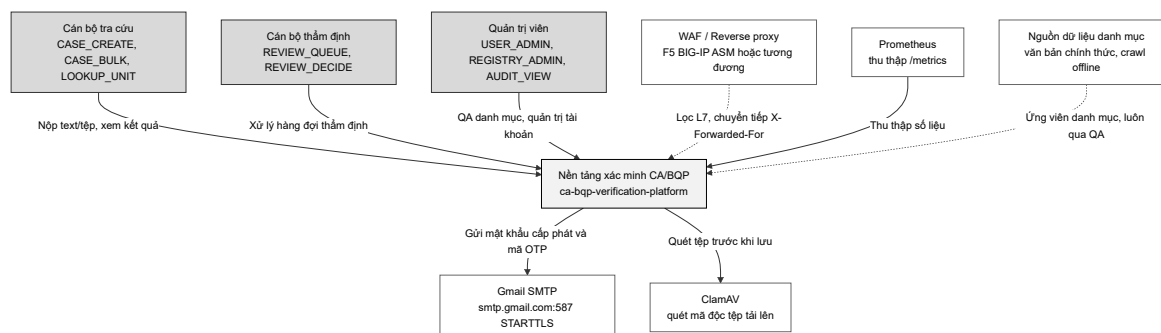
Abstention-capable Trừu xuất là kết quả hợp lệ và được ưu tiên hơn một kết luận độ tin cậy thấp. Hệ thống được thiết kế để *nói không biết*.

Deterministic and versioned Quy tắc chính sách là xác định, có version và có hiệu lực theo ngày; `as_of_date` được khoá theo Case để phát lại cho ra cùng kết quả.

Server-authoritative Mọi quyết định phân quyền đọc lại từ cơ sở dữ liệu ở mỗi request. Giao diện chỉ để để dùng, không bao giờ là nơi quyết định.

Fail-closed Khi nghi ngờ, hệ thống đóng lại: gate hỏng $\Rightarrow$ độ tin cậy 0; reviewer `DISMISS` $\Rightarrow$ Case `FAILED`; rule thiếu phạm vi $\Rightarrow$ `NOT_APPLICABLE` chứ không phải wildcard.

2.3 Ngữ cảnh hệ thống



Hình 2.1. Sơ đồ ngữ cảnh: tác nhân người dùng, hệ thống ngoài và ranh giới nền tảng

Ba nhóm tác nhân người dùng tương ứng ba tập quyền, không phải ba vai trò cứng: hệ thống chỉ có *permission* và một tài khoản quản trị duy nhất (xem Chương 12). Bốn hệ thống ngoài là Gmail SMTP (kênh giao mật khẩu cấp phát và mã OTP), ClamAV (quét tệp), Prometheus (thu thập số liệu) và thiết bị WAF/reverse proxy phía trước. Nguồn dữ liệu danh mục được vẽ bằng nét đứt vì nó *không bao giờ* ghi thẳng vào danh mục đang phục vụ: nó chỉ tạo ứng viên chờ QA.

Bất biến kiến trúc

Crawler hoặc phản hồi của người thẩm định **không bao giờ** ghi trực tiếp vào registry đang PUBLISHED. Mọi dữ liệu mới đi qua luồng ứng viên → QA → version → publish.

2.4 Luồng nghiệp vụ lõi

Toàn bộ hệ thống tồn tại để phục vụ một chuỗi biến đổi duy nhất:

```
Text/File → Case → Parser/OCR → Entity+CURRENT_WORK_UNIT → MasterUnitRegistry →
BCA/BQP/OTHER/UNKNOWN → SubjectGroup → Policy/Eligibility → Human
Review → Evidence/Audit/Export
```

Mỗi mũi tên trong chuỗi này là một *cổng có thể trừu xuất*. Không có bước nào được phép “đoán tiếp” khi bước trước không đủ căn cứ; đó là khác biệt thiết kế cơ bản giữa nền tảng này và một pipeline trích xuất thông thường.

2.5 Công nghệ nền

Lớp	Công nghệ	Ghi chú
API	FastAPI, Uvicorn, Python 3.11+	Router mở dưới /api/v1; chỉ trả JSON.

Lớp	Công nghệ	Ghi chú
ORM / migration	SQLAlchemy 2.0, Alembic	8 migration tường minh, có drill rollback.
CSDL	PostgreSQL 18	Trigger chỉ-ghi-thêm cho audit, advisory lock cho publish.
Hàng đợi	Celery 5 + Redis <i>hoặc</i> runner in-process	Chọn theo RUNTIME_PROFILE.
Lưu trữ tệp	MinIO <i>hoặc</i> hệ tệp cục bộ	Cùng một interface ObjectStorage().
Đọc tài liệu	PyMuPDF, pdfplumber, camelot, python-docx, openpyxl	Ưu tiên parser cấu trúc trước OCR.
OCR	EasyOCR (chính) + VietOCR (dự phòng)	PP-StructureV3 chỉ dùng cho cấu trúc bảng.
So khớp	rapidfuzz, BM25 tự hiện thực, sentence-transformers	Xếp hạng lai có calibration.
Frontend	React 19, TypeScript 5.9, Vite 7, Tailwind CSS 4, axios	SPA gọi thẳng /api/v1.
Giám sát	prometheus-client, log JSON có redaction	/metrics bật theo cấu hình.
Kiểm thử	pytest, golden suites, Playwright	Xem Chương 13.

CHƯƠNG 3

Mục tiêu và ràng buộc kiến trúc

3.1 Bất biến nghiệp vụ

Các phát biểu dưới đây không phải là khuyến nghị. Chúng là ràng buộc; mọi thay đổi vi phạm một trong số chúng là lỗi nghiệp vụ, kể cả khi test đơn vị vẫn xanh.

Bất biến kiến trúc

B1 — NOT_FOUND $\neq$ OTHER. Không tìm thấy đơn vị trong danh mục là một trạng thái *thiếu thông tin*. "Ngoài phạm vi CA/BQP" là một *kết luận* phải có đơn vị đã phân giải đứng sau. Bất biến này được giữ ở cả tầng phân giải, tầng API và tầng giao diện: màn hình nào biến NOT_FOUND thành "ngoài phạm vi" (hoặc ngược lại) là sai.

Bất biến kiến trúc

B2 — Thuộc đơn vị không chứng minh thuộc lực lượng. Việc một đơn vị nằm trong BCA hay BQP không tự nó chứng minh chủ thể là CAND hay quân nhân. `subject_group` cần bằng chứng riêng: nhân tường minh trong tài liệu, hoặc từ khoá trong văn bản không bị phủ định.

Bất biến kiến trúc

B3 — Nhiều tổ chức thì phải phân giải CURRENT_WORK_UNIT. Không được lấy tổ chức xuất hiện đầu tiên. Nhân lịch sử có định ngữ ("*Đơn vị công tác cũ*", "*Nguyên đơn vị công tác*") là một trường riêng (`former_unit_name`), không phải biến thể của `unit_name`.

Bất biến kiến trúc

B4 — Trùng khớp nhiều đơn vị là AMBIGUOUS, không phải một lựa chọn. Một tên phân giải ra nhiều đơn vị ở nhiều tổ chức là AMBIGUOUS. Va chạm *trong cùng một tổ chức* cũng trừu xuất, vì địa danh khác nhau có thể trùng sau khi bỏ dấu (*Phú Hữu / Phú Hựu*).

Bất biến kiến trúc

B5 — Quy tắc chính sách là xác định, có version, có hiệu lực theo ngày và phải trả INSUFFICIENT_DATA khi thiếu dữ kiện bắt buộc. `subject_groups=[]` *không* phải wildcard ngầm; rule toàn cục phải khai báo `[ '*' ]`.

Bất biến kiến trúc

B6 — Phản hồi không tự học thành dữ liệu sản xuất. Chính sửa của người thẩm định có thể sinh `RegistryCandidate`, nhưng không bao giờ tự tạo alias đã APPROVED.

Bất biến kiến trúc

B7 — Kết quả phải giữ bằng chứng và các version độc lập: registry, taxonomy, parser, model, policy, threshold. Đổi một ngưỡng là đổi các quyết định đã ghi nhận, nên phải tăng `threshold_version` trong cùng một lần sửa.

Bất biến kiến trúc

B8 — Dữ liệu tổng hợp phải mang nhãn `SYNTHETIC_DEMO` và không bao giờ được âm thầm nâng cấp thành dữ liệu có thẩm quyền.

Bất biến kiến trúc

B9 — Ưu tiên trừu xuất hơn kết luận độ tin cậy thấp. Khi lưỡng lự, hệ thống chọn `NEED_REVIEW`.

Bất biến kiến trúc

B10 — Gate chất lượng hỏng không đi kèm điểm tin cậy cao. Định tuyến hạ nguồn so điểm với ngưỡng, nên hai con số này bắt buộc phải nhất quán. Khi hai engine OCR đọc lệch trường định danh, `parse_quality.critical_disagreement=true` và độ tin cậy về 0.

3.2 Ràng buộc kỹ thuật

Ràng buộc	Hệ quả kiến trúc
Hệ thống nội bộ, ưu tiên chạy offline	Mọi tài sản mô hình (EasyOCR, VietOCR, PP-Structure, embedding) được bake sẵn vào image; không tải mô hình lúc chạy.
Không có nhà cung cấp định danh ngoài	Tài khoản, mật khẩu, phiên nằm trong chính CSDL của hệ thống. Keycloak, JWT/JWKS và dev-login đã bị gỡ bỏ có chủ đích.
Tiếng Việt là ngôn ngữ dữ liệu	Chuẩn hoá NFC, khoá bỏ dấu (<code>ascii_key</code>) lưu <i>cạnh</i> khoá có dấu; so khớp nhãn dùng fuzzy ngưỡng 75 chứ không so chính xác.
Kết quả phải phát lại được	<code>as_of_date</code> khoá theo Case; ảnh chụp danh mục bất biến; mọi version được đóng dấu lên kết quả.
Chỉ trả JSON	Không có render HTML phía máy chủ, nên không escape HTML khi ghi dữ liệu; việc escape thuộc về React tại điểm render.
Một tài khoản quản trị duy nhất	<code>admin_users.py</code> hard-code <code>is_admin=False</code> trên đường tạo; <code>_guard_admin_target</code> từ chối sửa tài khoản quản trị.

Ràng buộc	Hệ quả kiến trúc
Chạy được chỉ với PostgreSQL	Profile <code>local</code> thay MinIO bằng hệ tệp và thay Celery bằng runner in-process, dùng chung mã nguồn.

3.3 Thuộc tính chất lượng ưu tiên

Thuộc tính	Mục tiêu	Cơ chế kiến trúc
Đúng đắn	Không bao giờ kết luận sai ở mức tin cậy cao	Cascade phân giải có tầng; ngưỡng + margin + calibration; hai nguồn thẩm quyền tách bạch (đơn vị / nhân sự).
Truy vết	Mọi kết luận giải thích được sau nhiều tháng	evidence JSON, audit chỉ ghi thêm, ảnh chụp danh mục bất biến, 6 trục version.
Sẵn sàng	Mất broker không mất việc	outbox_events + Beat/sweeper; task acks late; DLQ.
Bảo mật	Token không mang claim	Phiên opaque, chỉ lưu SHA-256; quyền đọc lại mỗi request; một phiên hoạt động cho mỗi tài khoản.
Khả vận hành	Chạy được trên một máy	ops/local.py dựng trọn bộ profile local.
Hiệu năng	Chi phí mô hình chỉ trả khi cần	Bỏ qua tầng semantic khi bằng chứng từ vựng đã quyết định; OCR fallback chỉ chạy khi có lý do.

Giới hạn / cảnh báo

Nền tảng cung cấp các kiểm soát hướng production nhưng **không** tự biến dữ liệu demo thành dữ liệu nghiệp vụ thật. Trình quản lý bí mật, WAF và TLS termination phải do hạ tầng triển khai cung cấp. Artifact calibration hiện tại được fit từ nhiều tổng hợp xác định của danh mục trong repo và **phải** được fit lại trên dữ liệu kiểm chứng có nhãn thật trước khi triển khai thật.

CHƯƠNG 4

Khung nhìn Use-case

4.1 Tác nhân và mô hình quyền

Hệ thống **không** có vai trò cứng. `modules/auth/permissions.py` là danh mục quyền duy nhất; tài khoản giữ trực tiếp danh sách mã quyền. "Cán bộ thẩm định" là một *preset* gồm các ô tích mà quản trị viên bật, không phải một role.

Mã quyền	Mở khoá điều gì
CASE_CREATE	Nhập text hoặc tải tài liệu để tạo Case xác minh.
CASE_BULK	Tải danh sách nhiều người, mỗi dòng thành một Case.
LOOKUP_UNIT	Gọi resolver đơn vị trực tiếp (POST/lookup/unit).
CASE_EXPORT	Kết xuất kết quả và lịch sử ra CSV/XLSX.
CASE_VIEW_ALL	Xem mọi Case, không giới hạn ở hồ sơ tự tạo.
REVIEW_QUEUE	Mở hàng đợi thẩm định.
REVIEW_DECIDE	Nhận việc và ra quyết định thẩm định.
REGISTRY_ADMIN	QA, tạo version, publish Master Unit Registry.
PERSON_REGISTRY_ADMIN	QA, tạo version, publish Person Registry.
AUDIT_VIEW	Tra nhật ký kiểm toán.
USER_ADMIN	Tạo tài khoản và cấp/thu hồi quyền.

Bảng 4.2. Các preset quyền

USER_PRESET	CASE_CREATE, CASE_BULK, LOOKUP_UNIT, CASE_EXPORT
REVIEWER_PRESET	USER_PRESET + CASE_VIEW_ALL + REVIEW_QUEUE + REVIEW_DECIDE
Quản trị viên	Tài khoản admin duy nhất do seeder tạo; không API nào tạo thêm.

`coverage_groups` là trường *thu hẹp*: tài khoản có `CASE_VIEW_ALL` mà không khai `coverage_groups` sẽ thấy toàn bộ hàng đợi — đó chính là hình dạng của preset được bàn giao. `normalize()` loại bỏ mã không có trong danh mục, nên một quyền đã gỡ khỏi danh mục không thể tiếp tục cấp quyền thông qua bản ghi cũ còn sót trong CSDL.

4.2 Các ca sử dụng chính

4.2.1 UC-01: Tra cứu một hồ sơ

Tác nhân: cán bộ tra cứu (CASE_CREATE).

Kích hoạt: nhập text hoặc chọn tệp trên màn hình tra cứu.

Luồng chính: hệ thống tạo Case, xử lý bất đồng bộ, trả kết quả kèm bằng chứng (Hình 6.1).

Luồng thay thế:

- Tệp là danh sách nhiều dòng → 422 TABULAR_LIST_REQUIRES_BULK, giao diện chuyển hướng sang màn hình nạp theo lô. Cả hai lối vào tải tệp trên web (chọn tệp và bấm nút tra cứu) đều phải định tuyến mã lỗi này qua `openBulkFromFile`.
- Tài liệu text chứa nhiều khối "Họ và tên" → fan-out thành nhiều Case anh em.
- Không lấy được `case_id` → giao diện **phải** hiện trạng thái lỗi.

Giới hạn / cảnh báo

Giao diện tuyệt đối không được có nhánh "mô phỏng offline". Một phiên bản trước từng có ~160 dòng fallback phía client đoán BCA/BQP từ từ khoá phòng ban/chức vụ và trả về điểm số cứng kiểu '98.8%' khi lời gọi thật không trả `case_id`. Nhánh đó vì phạm đồng thời bất biến B9 và B8 và đã bị gỡ bỏ.

4.2.2 UC-02: Nạp danh sách theo lô

Tác nhân: cán bộ tra cứu (CASE_BULK). Mỗi dòng thành một Case độc lập; một lần nạp tạo tối đa BULK_MAX_ROWS (mặc định 50) Case. Chi tiết tại Mục 11.5.

4.2.3 UC-03: Thẩm định thủ công

Tác nhân: cán bộ thẩm định (REVIEW_QUEUE, REVIEW_DECIDE). Bốn quyết định: CONFIRM, UNKNOWN, INSUFFICIENT, DISMISS. Dùng khoá lạc quan (`expected_version`) cộng với khoá hàng (Hình 11.2).

4.2.4 UC-04: Quản trị danh mục đơn vị / nhân sự

Tác nhân: REGISTRY_ADMIN / PERSON_REGISTRY_ADMIN. Vòng đời DRAFT → VALIDATED → APPROVED → PUBLISHED, publish được tuần tự hoá bằng advisory lock. Hai danh mục có *lock riêng* và bất biến "đúng một version PUBLISHED" *riêng*: publish danh mục này không ảnh hưởng danh mục kia.

4.2.5 UC-05: Quản trị tài khoản

Tác nhân: USER_ADMIN. Quản trị viên nhập thông tin hành chính; hệ thống sinh tên đăng nhập từ mã số cán bộ (hoặc từ họ tên nếu không có mã) và một mật khẩu ngẫu nhiên gửi qua email. `_credential_response` chỉ trả mật khẩu về màn hình khi việc gửi mail *thất bại*. `email` và `personal_code` là duy nhất toàn hệ thống, cường chế ở hai chỗ: thông báo nghiệp vụ (409) và unique index của migration 0008.

4.2.6 UC-06: Tra nhật ký kiểm toán

Tác nhân: AUDIT_VIEW. Bảng `audit_logs` chỉ-ghi-thêm, cưỡng chế bằng trigger PostgreSQL (migration 0004).

4.3 Bản đồ màn hình

Thành phần React	Vai trò
<code>App.tsx</code>	Cổng xác thực mông: gọi <code>fetchCurrentUser()</code> , hiện <code>LoginPage</code> khi chưa đăng nhập, nhắc đổi mật khẩu khi tài khoản còn giữ mật khẩu do quản trị cấp, còn lại render <code>VerificationModule</code> .
<code>components/CABQPVerificationPage.tsx</code>	Giao diện nghiệp vụ thật (~2.9k dòng), gọi backend bằng <code>axios</code> tới đường dẫn tương đối <code>/api/v1/...</code>
<code>components/HistoryView.js</code>	Lịch sử tra cứu; render <code>totals</code> do <code>GET/cases</code> trả về.
<code>components/OcrResultModal.js</code>	Kết quả OCR.
<code>components/OriginalDossierModal.js</code>	Hồ sơ gốc; nhận nguyên văn phản hồi <code>GET/cases/{id}</code> .
<code>DetailedComparisonModal.js</code>	So sánh chi tiết ứng viên.
<code>ReviewsPage.jsx</code>	Hàng đợi thẩm định, phân trang thật.
<code>RegistryAdminPage.jsx</code>	Quản trị danh mục đơn vị.
<code>PersonRegistryAdminPage.jsx</code>	Quản trị danh mục nhân sự.
<code>UsersAdminPage.jsx</code>	Quản trị tài khoản.
<code>AuditPage.jsx</code>	Nhật ký kiểm toán.
<code>components/layout/*</code>	Header, thanh tab, footer, khối tiêu đề trang.

Lý do thiết kế

Menu quản trị được ẩn/hiện bằng `can(user, P.X)` trong `permissions.ts`. Đây thuần túy là tiện ích hiển thị: endpoint tương ứng kiểm tra lại đúng quyền đó ở máy chủ. Vì vậy `auth.ts` chỉ giữ một token phiên opaque và không bao giờ được coi là một kiểm tra phân quyền.

4.4 Ngữ nghĩa trạng thái hiển thị

Đây là nơi bất biến B1 thường bị vi phạm nhất, nên hợp đồng hiển thị được ghi rõ:

Trạng thái kết quả	Hiển thị đúng
MATCHED + OTHER	"Ngoài phạm vi CA/BQP" (appState: 'out-of-scope'). Đây là một kết luận có đơn vị đứng sau, không phải "Không tìm thấy".
NOT_FOUND	"Không tìm thấy" / "Chưa xác định được đơn vị".
organization_type vắng mặt	UNKNOWN, không bao giờ mặc định thành OTHER.
resolution_status / workflow_status vắng mặt	Giữ nguyên trạng thái vắng mặt, không tự thành MATCHED / PROCESSED.
Ứng viên không có tổ chức	"Chưa xác định".
verification_status	Đi theo <i>quyết định</i> , không theo việc quy trình đã đóng: một Case reviewer kết luận UNKNOWN đóng ở COMPLETE nhưng kết quả vẫn là NOT_FOUND/UNKNOWN. Chuỗi này đến từ API đã được diễn đạt sẵn cho người dùng; hãy render nguyên văn thay vì tra trong bảng nhãn theo mã.
Document.parse_statuses	Chỉ có ba giá trị hợp lệ PENDING/PASED/FAILED (ràng buộc CHECK của CSDL).

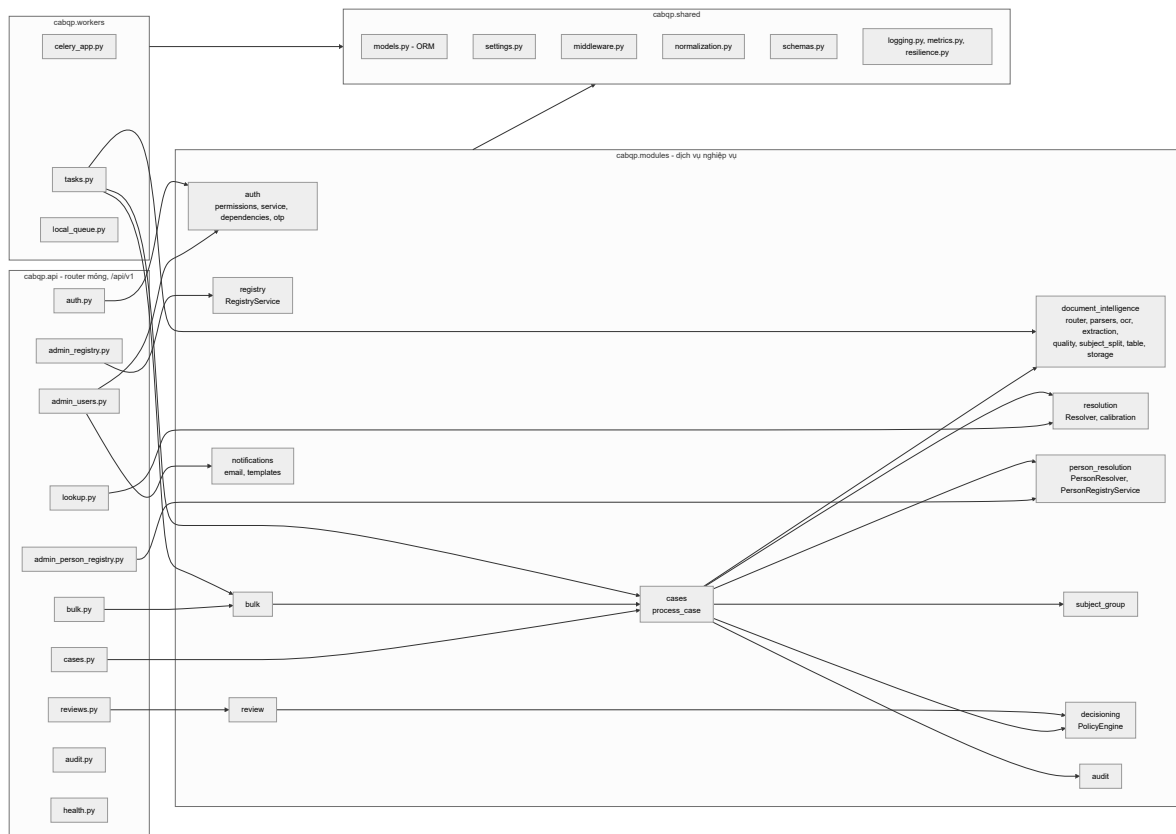
Không có màn hình nào được suy ra tên bộ từ một phép thử nhị phân `isBqp?` 'BộQuốcphong' : 'BộCôngan': *không phải BQP* không đương nhiên là BCA.

GET/cases trả về totals (all, verified, need_review, out_of_scope, no_conclusion) đếm trên *toàn bộ* hàng đợi trong phạm vi của người gọi, không phải trên trang hiện tại. Bốn thành phần được xây sao cho `verified+need_review+out_of_scope+no_conclusion==all`, với `no_conclusion` là phần dư; điều này chỉ không âm vì ba vị từ còn lại loại trừ lẫn nhau. `workflow_status=='NEED_REVIEW'` được kiểm tra *trước* và thắng nhãn của resolver: một Case còn thuộc về con người thì không bao giờ đồng thời được đếm là đã quyết định.

CHƯƠNG 5

Khung nhìn logic

5.1 Phân tầng



Hình 5.1. Thành phần backend: router middleware, dịch vụ nghiệp vụ, hạ tầng dùng chung, worker

Backend chia thành bốn gói với hướng phụ thuộc một chiều:

1. `cabqp.api` — router FastAPI *mở*. Mỗi route khai báo `require_perms(...)` hoặc `current_principal` (cho các truy vấn tự giới hạn phạm vi). Không chứa chuyển trạng thái nghiệp vụ.
2. `cabqp.modules` — dịch vụ nghiệp vụ. Mọi chuyển trạng thái sống ở `modules/*/service.py`.
3. `cabqp.shared` — ORM, cấu hình, middleware, chuẩn hoá, schema, log, metrics.
4. `cabqp.workers` — task Celery và runner in-process.

Bất biến kiến trúc

Router giữ nguyên độ mỏng; chuyển trạng thái nghiệp vụ nằm ở `modules/*/service.py`. Mọi route trả về một *projection dựng tường minh*, không bao giờ serialize thẳng một hàng ORM — đó là cơ chế giữ `password_hash` không bao giờ ra khỏi máy chủ (`admin_users._serialize` là khuôn mẫu).

5.2 Các mô-đun nghiệp vụ

Mô-đun	Trách nhiệm
auth	Danh mục quyền, băm mật khẩu PBKDF2-HMAC-SHA256, vòng đời phiên, phụ thuộc <code>current_principal / require_perms</code> , mã đăng nhập một lần.
document	Định tuyến đầu vào, đọc tài liệu, OCR, trích xuất, gate chất lượng, tách nhiều chủ thể, bảng, kiểm tra tệp, quét mã độc, lưu trữ đối tượng, hạn mức tài nguyên.
registry	Vòng đời version của Master Unit Registry, QA, ảnh chụp, diff, rollback.
resolution	Resolver: cascade phân giải tên/mã đơn vị và calibration.
person_resolution	PersonResolver (chỉ định danh) và PersonRegistryService (vòng đời version song song, độc lập).
subject_group	Phân loại nhóm đối tượng theo luật, có xử lý phủ định tiếng Việt.
decisioning	PolicyEngine: đánh giá rule có hiệu lực theo ngày.
cases	<code>process_case</code> : dàn nhạc toàn bộ quyết định cho một Case.
review	Vòng đời thẩm định thủ công, khoá lạc quan, uỷ quyền theo phạm vi.
bulk	Nạp danh sách theo lô: dò header, suy luận ánh xạ cột, xử lý từng dòng.
audit	Ghi nhật ký kiểm toán chỉ-ghi-thêm.
notifications	Gửi email qua Gmail SMTP và template MIME.

5.3 Hạt nhân điều phối: `process_case`

`modules/cases/service.py` là nơi duy nhất ghép các mô-đun lại thành một quyết định. Trình tự:

1. **Chuẩn bị:** xác định `as_of_date` — lấy từ đầu vào; nếu ngày do người dùng nhập không hợp lệ thì quay về ngày tạo Case (bất biến, để phát lại xác định), chứ không âm thầm đổi ngữ nghĩa chính sách.
2. **Trích xuất:** `extract()` trả về chủ thể, mã, chức vụ, đơn vị hiện tại, các đơn vị cũ, độ tin cậy trích xuất và độ tin cậy quan hệ.

3. Chọn đường phân giải:

- `person_lookup`: có tên/mã chủ thể nhưng *không* có đơn vị và *không* có mã đơn vị → gọi `PersonResolver`, rồi phân giải lại `canonical_unit_id` qua `Resolver` đơn vị.
 - Ngược lại → gọi thẳng `Resolver` với `unit_name/unit_code`.
4. **Nhóm đối tượng**: `classify_subject_group()` với tổ chức đã phân giải, chức vụ, văn bản và các trường nghiệp vụ.
 5. **Tổng hợp độ tin cậy**: `decision_conf` là *giá trị nhỏ nhất* của các thành phần áp dụng được. Không có phép trung bình nào che được một thành phần yếu.
 6. **Ghi kết quả**: `upsert ExtractedRecord` và `VerificationResult`, đóng dấu 5 trực version và khối `evidence` đầy đủ.
 7. **Thang điều kiện thẩm định** (Hình 11.1): nếu bất kỳ cổng nào không đạt thì Case đi `NEED _REVIEW`.
 8. **Chính sách**: chỉ chạy `PolicyEngine` *sau khi* bằng chứng thượng nguồn đã đủ mạnh. Nếu một rule áp dụng được nhưng thiếu dữ kiện, Case vẫn phải trừu xuất.
 9. **Kiểm toán**: ghi `CASE_DECISION` kèm mọi version và cờ `review_required`.

Lý do thiết kế

Chính sách là *một phần của quyết định nghiệp vụ*, không phải trang trí hậu kỳ. Nếu chạy chính sách trước khi biết đơn vị và nhóm đối tượng có đáng tin hay không, hệ thống sẽ sinh ra những đánh giá "đủ điều kiện" dựa trên căn cứ mà chính nó chưa chấp nhận.

5.3.1 Tra cứu không có quan hệ đơn vị

Hai hình dạng đầu vào không mang *quan hệ tương thuật* về đơn vị công tác:

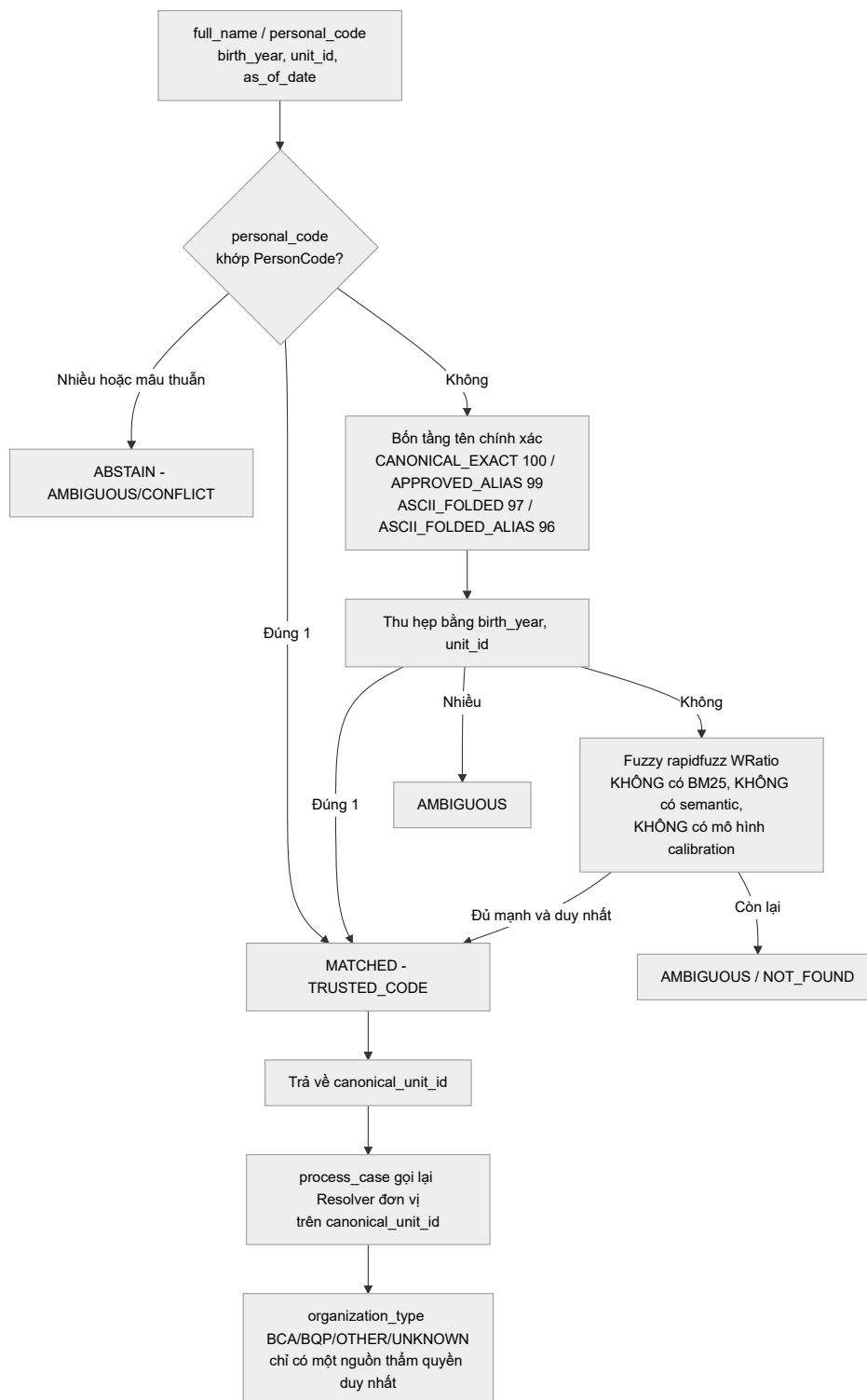
- Tra cứu bằng tên/mã chủ thể trần (`person_lookup`).
- Tra cứu có `unit_code` nhưng không có tên đơn vị (`unit_code_lookup`).

Vì từ hợp nhất `relationless_lookup=person_lookuporunit_code_lookup` miễn cho hai hình dạng này khỏi cổng `relation_confidence<0.7` và khỏi thang điểm đầy đủ bốn trường, vì chúng không tuyên bố cung cấp `current_unit`. Việc mã đơn vị có phân giải được hay không vẫn là một cổng riêng (`r.status!="MATCHED"`), nên cơ chế này không bao giờ cứu một mã không tồn tại.

Bất biến kiến trúc

Thang lý do thẩm định phải đọc *đúng cùng một vị trí* với cổng đã chặn Case. Báo `CURRENT_WORK_UNIT_UNCERTAIN` cho một Case không hề bị chặn vì lý do đó sẽ chỉ người thẩm định vào một dữ kiện sai.

5.4 Hai nguồn thẩm quyền tách bạch



Hình 5.2. PersonResolver: cascade định danh riêng, luôn trả quyền quyết định tổ chức về Resolver đơn vị

`person_resolution` là một danh mục + resolver *thứ hai*, độc lập về cấu trúc với danh mục đơn vị. Điểm mấu chốt về kiến trúc:

- `PersonResolver` **chỉ** định danh. Nó trả về `canonical_unit_id` và không bao giờ tự quyết định `organization_type`.
- `process_case` *luôn* phân giải lại `canonical_unit_id` đó qua Resolver đơn vị, nên phân loại BCA/BQP có đúng một nguồn thẩm quyền.
- `_person_candidates_with_review_scope` làm giàu ứng viên nhân sự bằng chính Resolver đơn vị, để bằng chứng ứng viên không trôi thành một nguồn thẩm quyền thứ hai.
- `subject_group_hint` từ danh mục nhân sự chỉ là gợi ý; `classify_subject_group` vẫn đối chiếu với `KNOWN_GROUPS`, nên một gợi ý lạ không thể trở thành kết luận nghiệp vụ.

5.5 Frontend

`App.tsx` là cổng xác thực mỏng và *phải* giữ nguyên độ mỏng đó.

Giới hạn / cảnh báo

Một hồi quy trước đây khiến `App.tsx` import cây router `./pages/*` (`Dashboard/Lookup/Cases/Reviews/Admin/PersonAdmin`) **không tồn tại trong repo hay trong lịch sử git**, làm `npmrunbuild` hỏng hoàn toàn. Không có lý do chính đáng nào để `App.tsx` import bất cứ thứ gì dưới `./pages/`.

Cây React thật gọi backend trực tiếp bằng `axios` tới đường dẫn tương đối `/api/v1/...` (Vite dev proxy ở profile local, Nginx ở profile docker). Repo **không** có `api.ts` hay `hooks.ts`; không dựng lại một API client có kiểu riêng trước khi kiểm tra xem cây JSX đã gọi endpoint đó chưa.

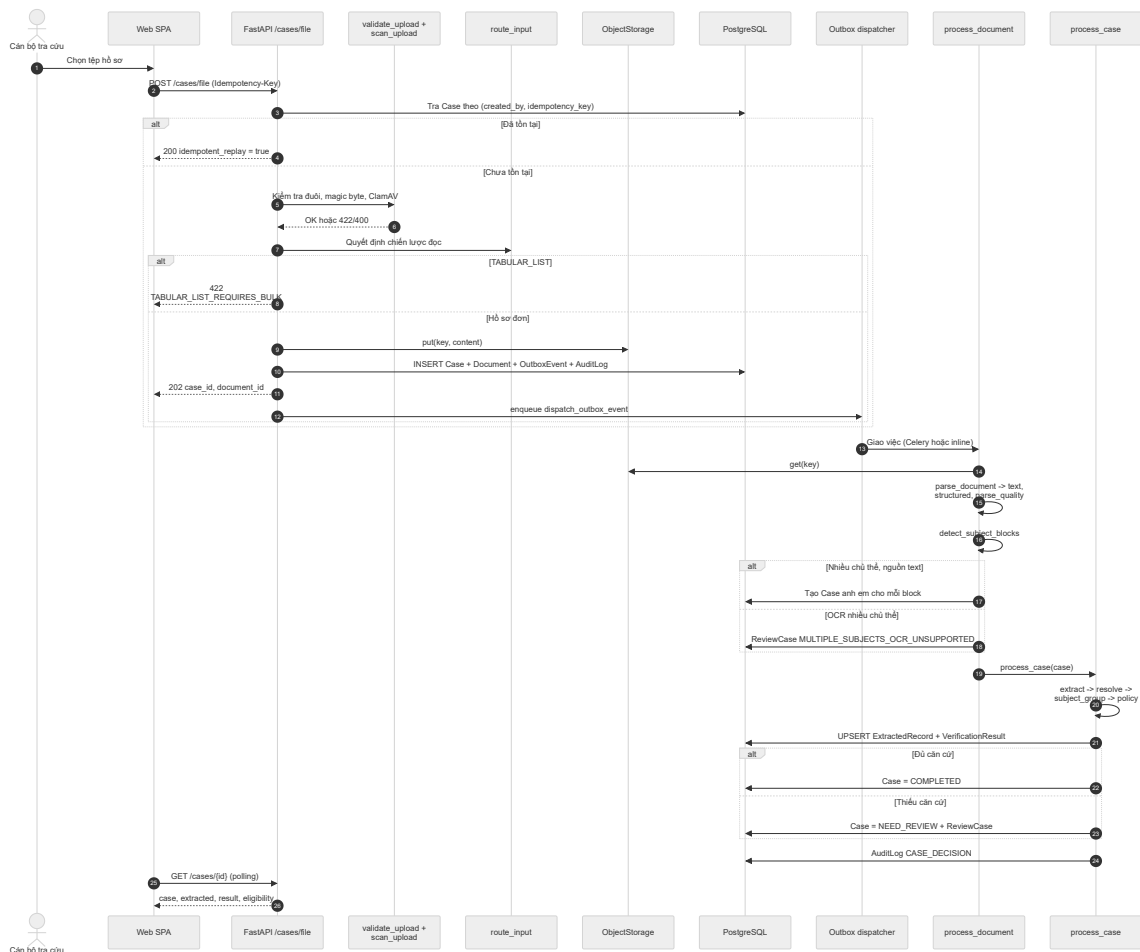
`styles.css` chỉ phủ phần phi-Tailwind của các màn hình quản trị và được bọc trong `@layerbase` để utility của Tailwind thắng; phần còn lại dùng thẳng utility class. `components/AppErrorBoundary.jsx` có tồn tại nhưng *chưa* được nối vào `main.tsx/App.tsx` — đừng giả định lỗi render đang được nó bắt.

Để ba nơi hiển thị trạng thái không bao giờ mâu thuẫn nhau, `CABQPVerification.jsx` rút trạng thái một dòng qua `historyStatusCategory`, dùng chung bởi `resolvedStateFromDetail` và `subjectRowStatus`: một dòng danh sách, một dòng chủ thể trong tài liệu nhiều người, và màn hình chi tiết mà nó mở ra đều đi qua cùng một hàm.

CHƯƠNG 6

Khung nhìn tiến trình

6.1 Toàn cảnh một Case



Hình 6.1. Trình tự xử lý một Case từ tệp tải lên đến kết quả

Đường đi có ba đoạn tách bạch:

1. **Đồng bộ, trong request:** kiểm tra idempotency, xác thực tệp, quét mã độc, định tuyến đầu vào, ghi Case + Document + OutboxEvent + AuditLog trong *một* transaction, rồi trả 202.
2. **Bàn giao:** dispatch_outbox_event. Ngữ nghĩa bên vãng khác nhau theo profile (Mục 6.3).
3. **Bất đồng bộ:** process_document đọc tệp, parse, tách chủ thể, rồi gọi process_case cho từng Case.

Bất biến kiến trúc

Xử lý Case an toàn khi chạy lại. Ghi kết quả/trích xuất dùng ngữ nghĩa upsert theo Case. Đây là bảo đảm *theo Case*, không phải theo Document: `Document.case_id` không phải khoá duy nhất trong lược đồ, nên fan-out nhiều chủ thể có thể gắn nhiều Document vào cùng một lần tải lên. Điều tuyệt đối không được xảy ra là quá một `ExtractedRecord/VerificationResult` trên một Case (`VerificationResult.case_id` là duy nhất ở CSDL).

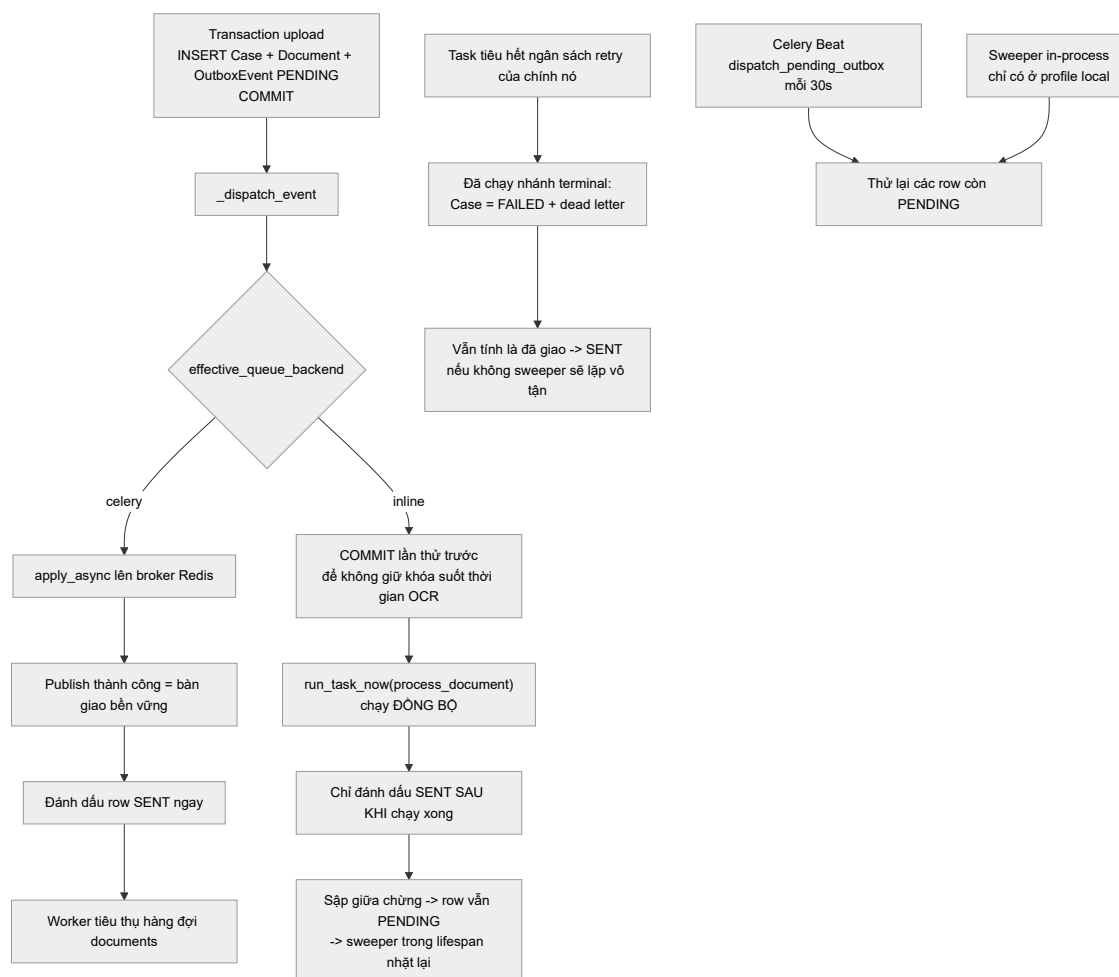
6.2 Idempotency

Case có ràng buộc duy nhất (`created_by`, `idempotency_key`). Client gửi header `Idempotency-Key`; lần gọi lặp trả về Case cũ với cờ `idempotent_replay:true`. Nếu một request song song thắng cuộc đua, `IntegrityError` được bắt và xử lý như một lần phát lại thay vì trả lỗi 500.

Lý do thiết kế

Khi tách nhiều chủ thể, block #0 — Case tạo đồng bộ lúc tải lên — **giữ nguyên** `Idempotency-Key` của client trên cả hai đường fan-out. Một phiên bản trước ghi đề nó bằng khoá nhóm, khiến lần thử lại của client mang khoá gốc không khớp gì cả và tải hồ sơ lên lần thứ hai. Vì vậy block #0 được nhận lại như thành viên nhóm thông qua `evidence.split_source.source_case_id` chứ không qua tiền tố khoá.

6.3 Bàn giao bền vững qua Outbox



Hình 6.2. Ngữ nghĩa bàn giao bền vững khác nhau giữa hai profile

Độ bền đến từ bảng `outbox_events`, *không* từ `broker.tasks._dispatch_event` phải tôn trọng ngữ nghĩa của từng profile:

Profile	Ngữ nghĩa
<code>celery</code>	<code>apply_async</code> đẩy lên broker <i>chính</i> là bàn giao bền vững, nên hàng được đánh dấu SENT ngay tại chỗ.
<code>inline</code>	<code>LocalTaskQueue.submit</code> chỉ giao task cho một thread pool trong tiến trình — hoàn toàn không bền vững. Dispatcher vì thế commit lần thử, chạy <code>process_document</code> <i>đồng bộ</i> qua <code>run_task_now</code> , rồi mới đánh dấu SENT.

Giới hạn / cảnh báo

Đánh dấu SENT ngay lúc submit ở profile inline — như một phiên bản trước từng làm — làm mất hẳn công việc mỗi khi tiến trình chết trước lúc task chạy, để lại một hàng SENT

mà không gì nhặt lại và một Case đứng mãi ở RECEIVED.

Một task đã tiêu hết ngân sách retry của chính nó thì đã chạy nhánh terminal (Case FAILED + dead letter), nên vẫn tính là đã giao và hàng đi SENT; nếu xếp lại hàng, sweeper sẽ lập vô tận trên một tài liệu đã chết hẳn.

6.4 Cấu hình worker

6.4.1 Profile docker: Celery

Thiết lập	Vì sao quan trọng
<code>cabqp.workers.tasks</code>	Thiếu nó, worker không đăng ký task nào và âm thầm vút mọi message.
<code>task_routes</code>	<code>process_document</code> → <code>documents</code> ; <code>dispatch_outbox_event/dispatch_pending_outbox</code> → <code>outbox</code> ; <code>process_bulk_job/process_bulk_row</code> → <code>batch</code> . Worker khởi động thiếu <code>-Qcelery, outbox, documents, batch</code> chỉ nhận hàng đợi mặc định và âm thầm bỏ mọi thứ được định tuyến đi nơi khác — biểu hiện là upload đứng mãi ở RECEIVED mà không có lỗi.
<code>task_acks_late=True</code>	Message chỉ được ack sau khi chạy xong.
<code>task_reject_on_worker_lost=True</code>	Worker chết thì việc quay lại hàng đợi.
<code>worker_prefetch_multiplier=1</code>	Tránh một worker ôm hàng loạt job dài.
<code>soft_time_limit</code>	Chặn một tài liệu OCR bệnh lý chiếm worker vô hạn.
Celery Beat	Bắt buộc: <code>dispatch_pending_outbox</code> và <code>reconcile_bulk_jobs</code> chạy mỗi 30 giây — đây mới là thứ thử lại bền bỉ các lần bàn giao hỏng.

6.4.2 Profile local: runner in-process

`workers/local_queue.py` chạy chính các hàm task của Celery thông qua `_InlineContext`, lớp này cung cấp `self.request.retries / self.max_retries` mà thân task đọc và biến `self.retry(...)` thành một tín hiệu để runner xử lý. Nhờ đó nhánh thất bại terminal của task (đánh dấu Case FAILED, dead-letter) chạy ở lần thử cuối y hệt như dưới worker thật.

Giới hạn / cảnh báo

Không "đơn giản hoá" phần này thành lời gọi hàm trực tiếp: gọi thẳng một bound task của Celery sẽ khiến `self.retry()` cố đẩy message lên một broker không tồn tại trong

profile này.

`main.py` khởi động một thread sweeper trong `lifespan` ở profile local để tái bàn giao những hàng mà một lần sập để lại ở trạng thái `PENDING`.

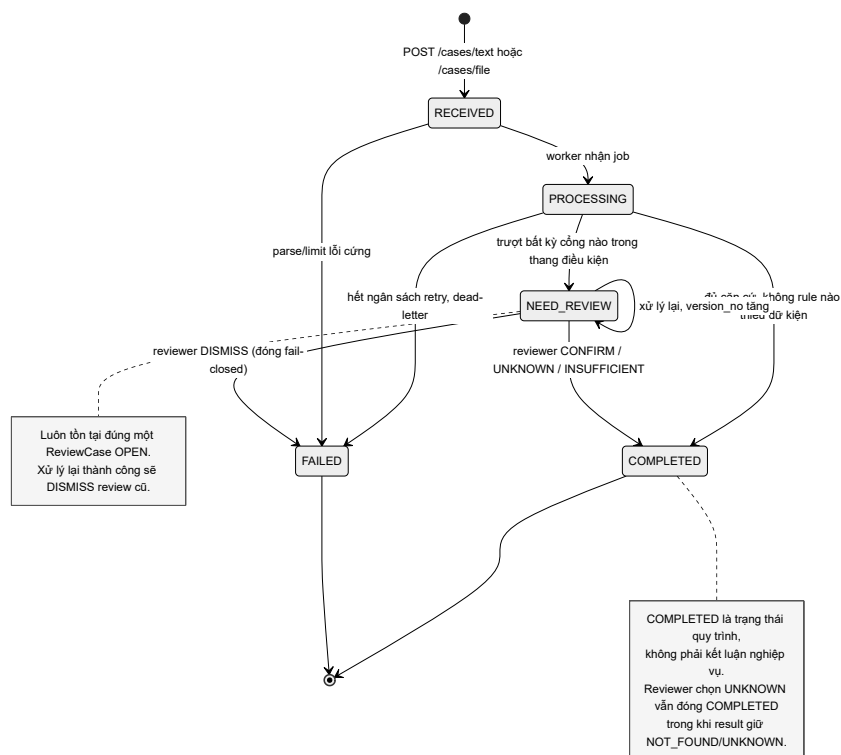
6.5 Đồng thời và tranh chấp

Điểm tranh chấp	Cơ chế
Hai reviewer cùng quyết định	Khoá lạc quan <code>expected_version + SELECT...FORUPDATE</code> . Lệnh <code>version</code> trả 409.
Đếm lại tiến độ job lô	<code>refresh_job_counts</code> khoá hàng job trước khi tính lại. Các task đồng chạy song song và mỗi task tính lại toàn bộ job; đọc-sửa-ghi không đồng bộ khiến người ghi cuối lưu một ảnh chụp cũ và treo job vĩnh viễn ở <code>PROCESSING</code> — profile local không có Beat chạy <code>reconcile_bulk_jobs</code> để kết thúc hộ.
Publish version danh mục	Advisory lock của PostgreSQL + <code>SELECT...FORUPDATE</code> trên các version đang <code>PUBLISHED</code> + partial unique index. Hai danh mục dùng hai lock khác nhau.
Tải lên trùng Idempotency-Key	Ràng buộc duy nhất ở CSDL; <code>IntegrityError</code> được xử lý như phát lại.
Nạp trùng cả tệp lô	<code>BulkIngestJob.file_sha256</code> là duy nhất.
Trùng dòng trong một job	<code>row_hash</code> + trạng thái <code>SKIPPED_DUPLICATE</code> .

6.6 Xử lý lỗi

- `DocumentLimitError` mang mã tường minh (`INVALID_PDF_DOCUMENT`, `PDF_PAGE_LIMIT_EXCEEDED`, `TABULAR_LIST_REQUIRES_BULK`, ...). Route upload trả 422; worker biến nó thành `NEED_REVIEW` mà người vận hành nhìn thấy.
- Riêng lỗi vượt `bulk_max_rows` trả kèm `{code, rows, limit, message}` vì web client hiển thị nguyên văn `message` trong hộp thoại chặn. Các mã `DocumentLimitError` khác giữ hình dạng chỉ-có-mã.
- Handler ngoại lệ toàn cục trả `{"error": {code, message, path}}` với `code=INTERNAL_ERROR` và ghi log có ngữ cảnh; thông tin nội bộ không rò ra ngoài.
- `shared/resilience.py` cung cấp retry và circuit breaker cho phụ thuộc ngoài (MinIO, ClamAV) theo `EXTERNAL_RETRY_ATTEMPTS`, `CIRCUIT_BREAKER_FAILURES`, `CIRCUIT_BREAKER_RECOVERY_SECONDS`.
- Hàng đợi `document_dlq` không có consumer mặc định: message ở lại cho người vận hành kiểm tra và phát lại.

6.7 Vòng đời Case



Hình 6.3. Máy trạng thái của Case

Bất biến kiến trúc

Một Case `NEED_REVIEW` luôn có đúng một ReviewCase đang OPEN. Xử lý lại thành công sẽ `DISMISS` review cũ; reviewer `DISMISS` thủ công đóng Case ở `FAILED` (fail-closed) thay vì để Case treo ở `NEED_REVIEW` mà không còn việc nào mở.

CHƯƠNG 7

Khung nhìn triển khai

7.1 Hai profile, một mã nguồn

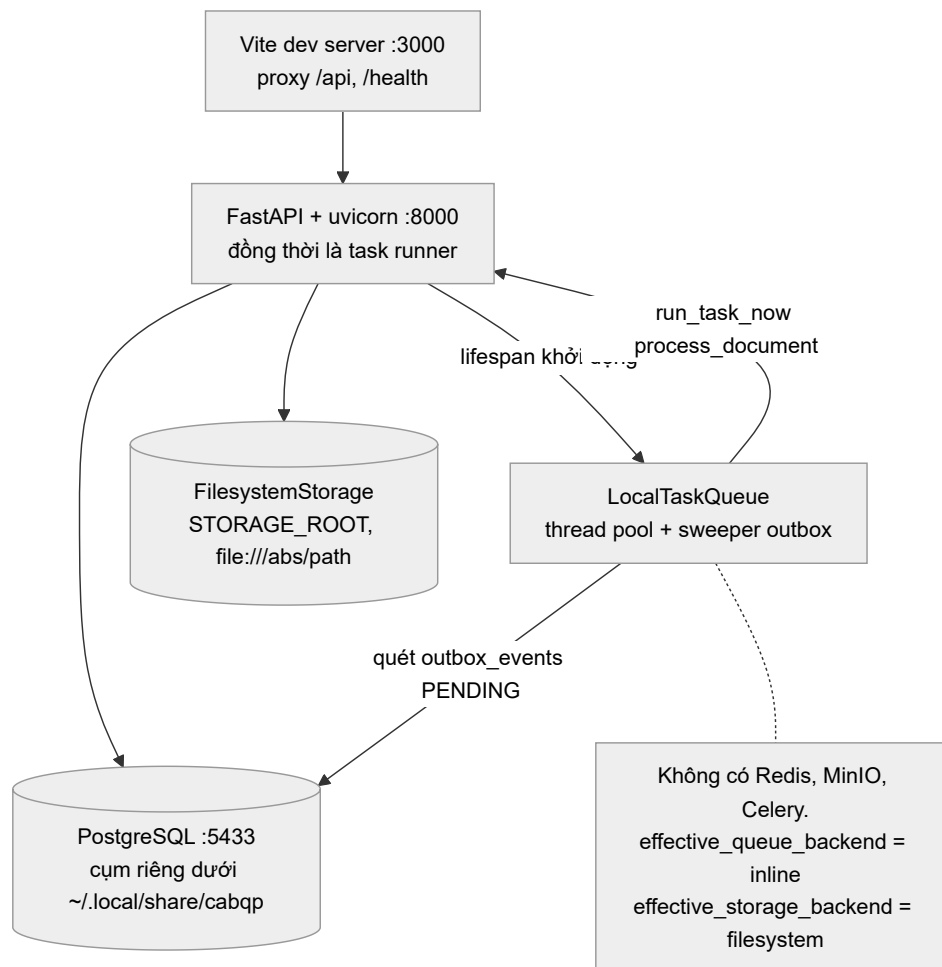
`RUNTIME_PROFILE (local | docker)` chọn backend lưu trữ và hàng đợi qua `Settings.effective_storage_backend / effective_queue_backend`; cả hai có thể bị ghi đè trực tiếp bằng `STORAGE_BACKEND / QUEUE_BACKEND`.

Khía cạnh	local	docker
Phục vụ SPA	Vite dev server	Nginx
Lưu tệp	FilesystemStorage, file:///abs/path	MinioStorage, s3://bucket/key
Hàng đợi	Runner in-process + sweeper	Celery worker + Beat + Redis
Phụ thuộc bắt buộc	Chỉ PostgreSQL	PostgreSQL, Redis, MinIO (ClamAV tùy chọn)
Rate limit	Cửa sổ in-memory	Cùng middleware, có Redis
Readiness	Không kiểm tra Redis	Kiểm tra Redis

Bất biến kiến trúc

`ObjectStorage()` là một *factory*, không phải class, trả về một trong hai hiện thực sau cùng một interface, nên điểm gọi không đổi. Hai backend sinh URI khác scheme, vì vậy nơi nào cần đổi `storage_uri` ngược về key **phải** dùng `storage.key_from_uri()` chứ không cắt chuỗi tiền tố của một scheme — bulk confirm từng làm điều thứ hai và trả 500 trên mọi bảng tính ở profile local.

7.2 Profile local



Hình 7.1. Profile local: một tiến trình, chỉ phụ thuộc PostgreSQL

`ops/local.py` sở hữu trọn vẹn profile local:

1. Khởi động một cụm PostgreSQL 18 *riêng* ở cổng **5433**, dữ liệu dưới `~/.local/share/cabqp/postgres`. Cụm hệ thống ở 5432 không cấp quyền tạo database cho người dùng này, và volume của dự án không thỏa kiểm tra quyền thư mục của PostgreSQL.
2. Ghi tệp `.env` ở gốc repo với mật khẩu sinh ngẫu nhiên trong lần chạy đầu.
3. Chạy migration, seed danh mục và hai tài khoản bàn giao.
4. Khởi động uvicorn và Vite. Log nằm ở `.local/backend.log` và `.local/web.log`.

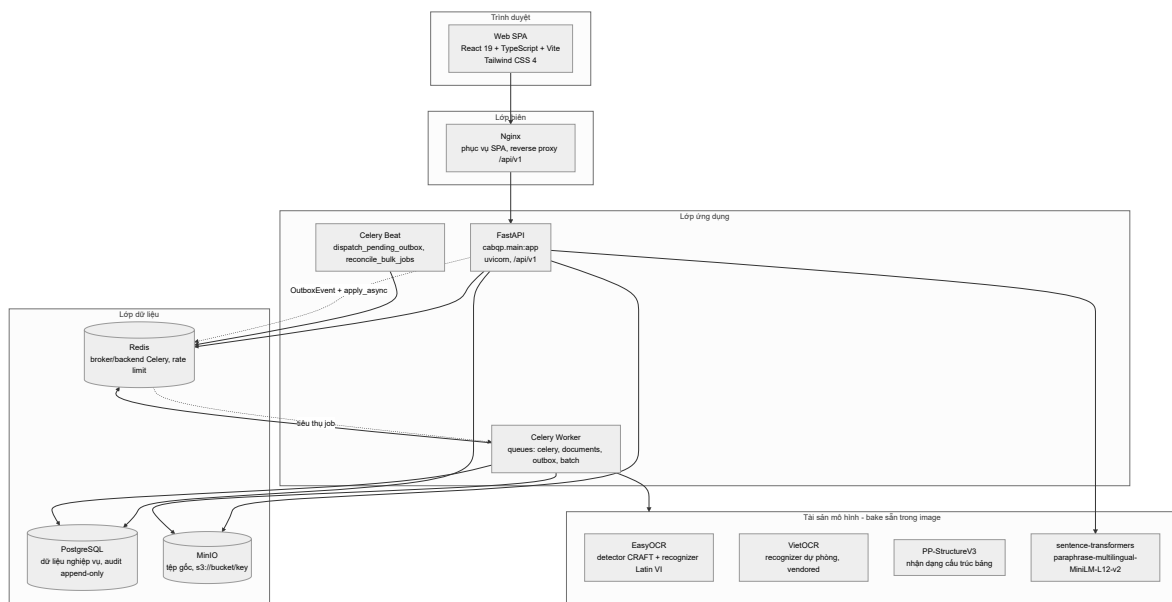
Lý do thiết kế

`ops/local.py` tiêm `.env` ở gốc repo vào môi trường của *mọi* tiến trình con một cách tường minh thay vì dựa vào cơ chế tra cứu tương đối của pydantic: alembic và script seed chạy với thư mục làm việc là `apps/backend`, nơi một `.env` tương đối không phân giải được. Cùng lý do đó, `USE_TF=0USE_TORCH=1` được đặt ở đây — nếu không, việc import

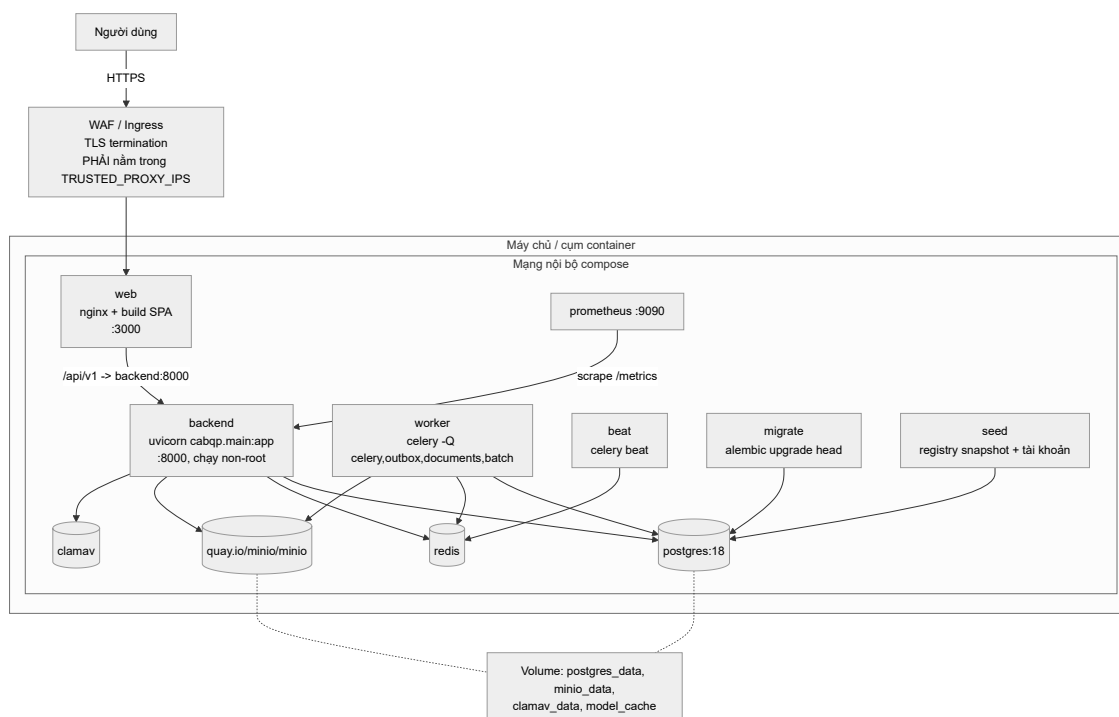
`transformers` sẽ hỏng vì một bản cài TensorFlow cũ.

Vì các giá trị đó trở thành biến môi trường *thật*, chúng thắng cơ chế đọc `.env` của pydantic và không bao giờ được pydantic bỏ ngoặc, nên `read_env_file()` tự bóc một lớp ngoặc bao quanh. Nếu không, một `EMAIL_APP_PASSWORD="abcdefghijklmnop"` sẽ đến Gmail kèm nguyên dấu ngoặc và mọi thư cấp mật khẩu/OTP đều hỏng với `SMTPAuthenticationError`.

7.3 Profile docker



Hình 7.2. Sơ đồ container ở profile docker



Hình 7.3. Sơ đồ triển khai: dịch vụ, volume và biên TLS/WAF

compose.yaml định nghĩa: postgres, redis, minio, clamav, migrate, seed, backend, worker, beat, web, prometheus, cùng các volume postgres_data, minio_data, clamav_data, model_cache.

Giới hạn / cảnh báo

*Docker **chưa** được cài trên máy phát triển hiện tại, nên profile docker chưa được kiểm chứng ở đây: compose.yaml parse được nhưng chưa từng build hay chạy kể từ khi gỡ Keycloak.*

Một số ràng buộc đóng gói đã học được và cần giữ:

- docker.io/minio/minio nay từ chối hoàn toàn việc pull ẩn danh, nên compose.yaml kéo quay.io/minio/minio.
- Lớp apt của image backend **phải** giữ git:passwordgen được ghim vào một nguồn git+https:// trong pyproject.toml (tên trên PyPI là một dự án khác), và pip giải một yêu cầu VCS bằng cách gọi gitclone — việc mà python:3.11-slim không tự làm được.
- Cả worker lẫn uvicorn cần USE_TF=0USE_TORCH=1; Celery trên Windows còn cần --pool=solo.
- Image bake sẵn mô hình EasyOCR, VietOCR, PP-Structure và embedding nên container không bao giờ tải mô hình lúc chạy.

7.4 Phát triển trên host không rebuild image

Trên máy phát triển hiện tại, profile local đã chạy mọi thứ trên host. Khi cần tái hiện profile docker, hãy giữ Docker cho hạ tầng: dockercomposeup-dpostgresredisminio (thêm clamav chỉ khi thay đổi cần đến nó), rồi chạy migrate/seed/uvicorn/celery trên host trở về cổng

localhost. Môi trường conda bqp đã có sẵn toàn bộ stack AI, nên một sửa đổi mã nguồn kiểm tra được bằng cách khởi động lại uvicorn thay vì trả giá cho dockercomposebuildbackend.

Giới hạn / cảnh báo

Trước khi tin bất kỳ máy chủ nào chạy trên host: pipshowcabqp có thể âm thầm trở tới một bản sao khác của dự án trên cùng máy (xem dòng Editableprojectlocation). Một máy chủ gắn nhằm checkout vẫn trả lời request bình thường, nó chỉ chạy mã cũ/khác. Chạy lại pipinstall--no-deps-e. từ đúng apps/backend này để sửa. --reload cũng không bắt được mọi thay đổi trên Windows/WatchFiles. Nếu một bản sửa "không có tác dụng" qua HTTP, hãy so thời điểm khởi động tiến trình với mtime của tệp rồi khởi động lại hẳn uvicorn/Celery thay vì tin vào auto-reload.

7.5 Cổng sức khỏe

Endpoint	Ngữ nghĩa
GET/health	Liveness tương thích ngược.
GET/health/live	Liveness.
GET/health/readiness	Readiness. Chỉ kiểm tra thứ profile đang dùng thật: Redis chỉ được kiểm tra khi QUEUE_BACKEND=celery; "identity" là sự tồn tại của một tài khoản quản trị, không phải một discovery document ngoài. Trả 503 kèm chi tiết từng mục khi có mục không ok.

7.6 Phục vụ SPA và biên mạng

SPA do Nginx phục vụ, không phải tiến trình FastAPI, và mang bản chính sách bảo mật riêng trong apps/web/nginx.conf. Chính sách đó buộc phải lỏng hơn của API (style-src/font-src cho phép Google Fonts) vì index.html nạp các webfont Inter / Be Vietnam Pro / Material Symbols — bỏ chúng sẽ làm hỏng chính các ligature icon mà phần so khớp nhãn trong e2e phụ thuộc vào. vite.config.ts đặt các header chống nhúng khung trên dev server để hai môi trường tương đương nhau.

Bất biến kiến trúc

Một triển khai đặt Nginx, load balancer hoặc WAF trước API **bắt buộc** phải liệt kê hop đó trong TRUSTED_PROXY_IPS, nếu không mọi người dùng sẽ gộp chung vào một bucket rate limit duy nhất.

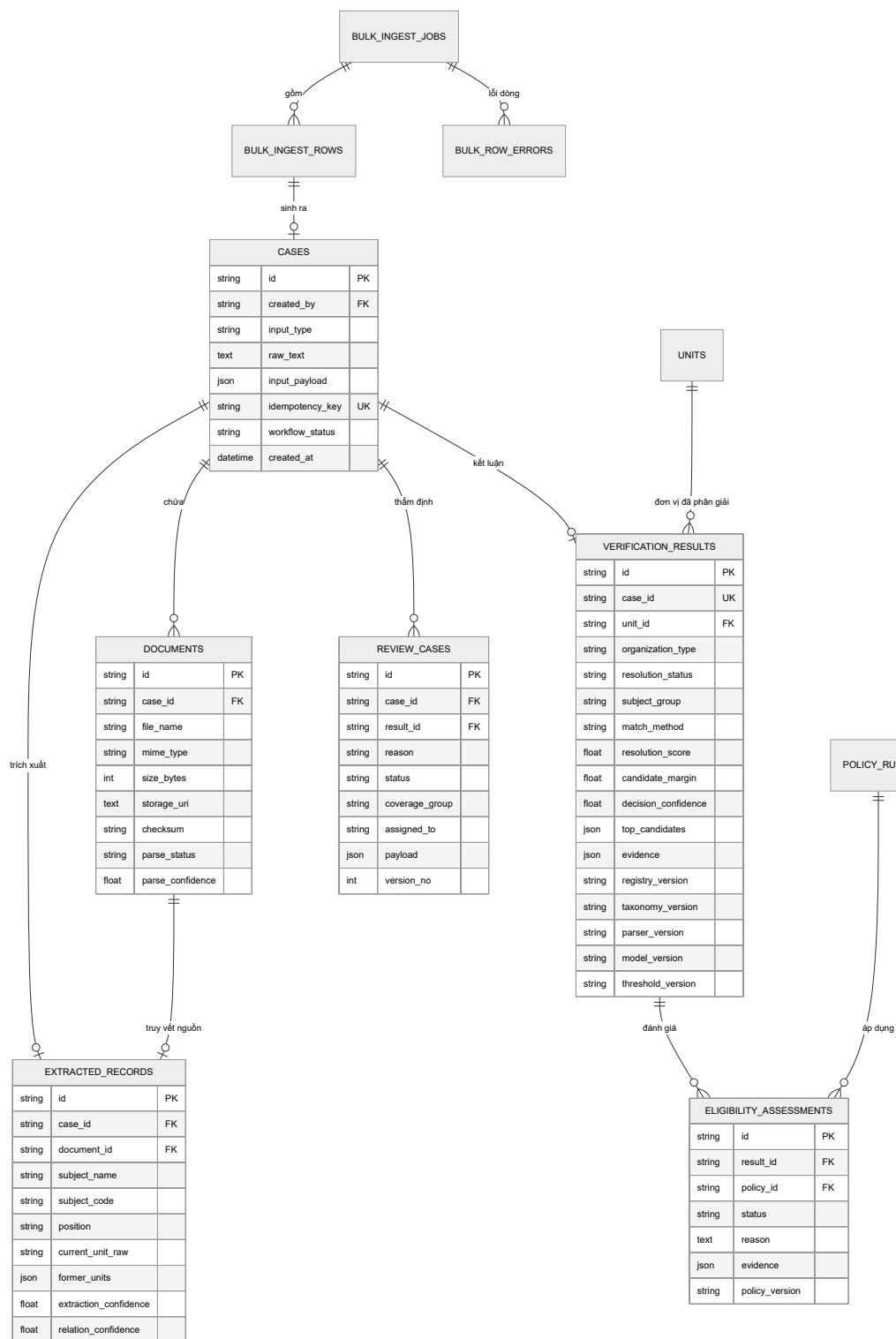
CHƯƠNG 8

Khung nhìn dữ liệu

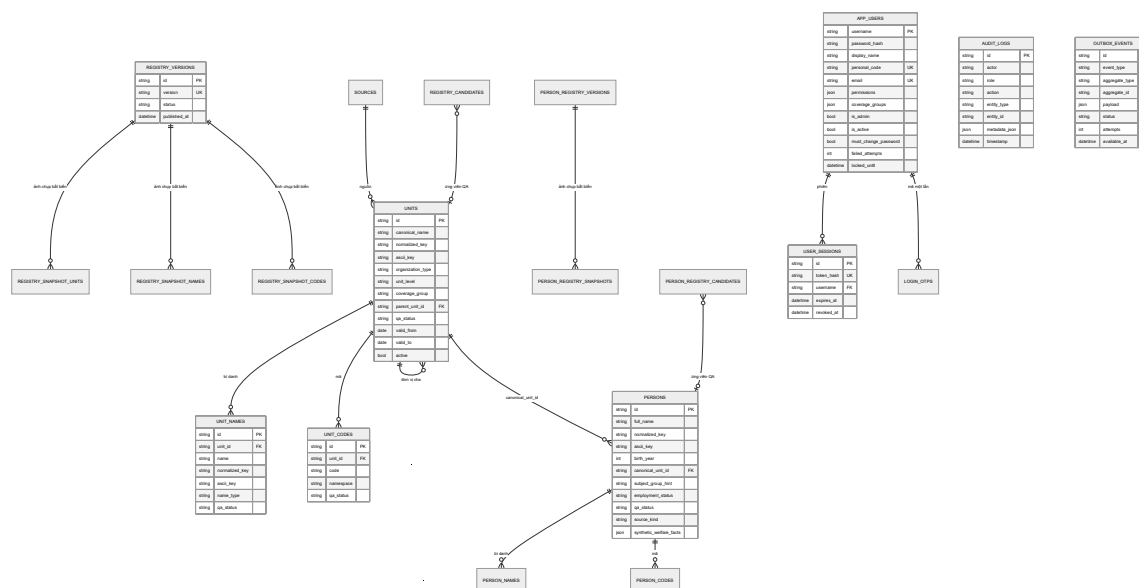
8.1 Nhóm thực thể

Lược đồ chia thành năm nhóm:

1. **Luồng Case:** cases, documents, extracted_records, verification_results, eligibility_assessments, review_cases.
2. **Danh mục đơn vị:** registry_versions, units, unit_names, unit_codes, ba bảng ảnh chụp, registry_candidates, sources.
3. **Danh mục nhân sự:** person_registry_versions, persons, person_names, person_codes, ba bảng ảnh chụp, person_registry_candidates.
4. **Nạp theo lô:** bulk_ingest_jobs, bulk_ingest_rows, bulk_row_errors, header_alias_candidates.
5. **Hạ tầng:** app_users, user_sessions, login_otps, audit_logs, outbox_events, policy_rules.



Hình 8.1. Mô hình dữ liệu luồng Case



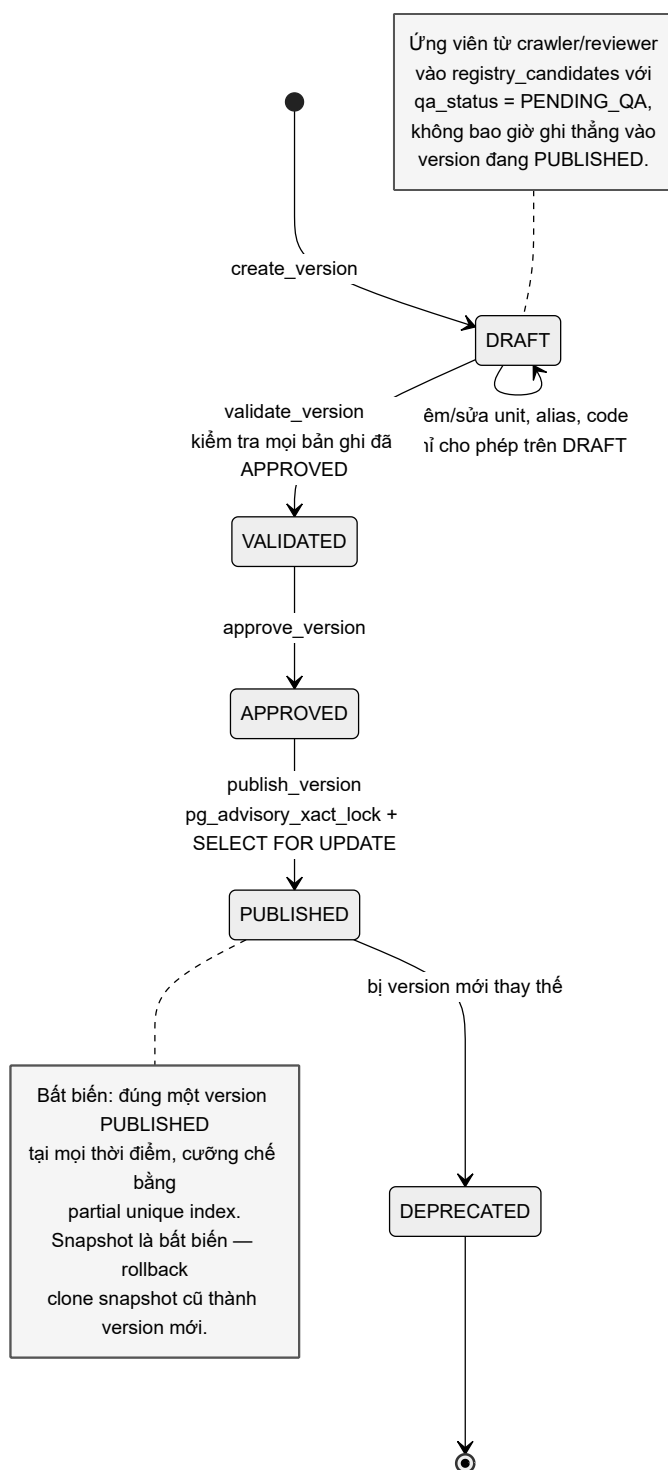
Hình 8.2. Mô hình dữ liệu hai danh mục và hạ tầng dùng chung

8.2 Version hoá và ảnh chụp bất biến

Phân giải *lúc chạy* không đọc bảng units "sống", mà đọc các bảng ảnh chụp của version đang PUBLISHED: registry_snapshot_units, registry_snapshot_names, registry_snapshot_codes. Nhờ đó:

- Một kết quả cũ luôn giải thích được bằng chính dữ liệu đã tạo ra nó.
- Sửa danh mục không thể lạng lẽ làm đổi ý nghĩa của các kết quả đã ghi.
- Rollback không sửa ảnh chụp lịch sử: hệ thống clone ảnh chụp cũ thành một version mới rồi publish nguyên tử.

`Unit.registry_version_id` chỉ là con trỏ version nguồn giữ lại để tương thích khi import; phân giải lúc chạy dùng các bảng ảnh chụp bất biến.



Hình 8.3. Vòng đời version danh mục, áp dụng độc lập cho hai danh mục

8.2.1 Chuẩn hoá khoá

`shared/normalization.py` được dùng chung giữa pipeline ngoại tuyến và runtime, nên khoá không bao giờ trôi lệch giữa hai bên:

- `normalize_text`: chuẩn NFC, quy dấu gạch dài về gạch ngắn, casefold, gộp khoảng

trắng/gạch/underscore, bỏ ký tự không phải chữ–số–dấu tiếng Việt.

- `ascii_key`: thêm bước bỏ dấu (NFD, loại Mn) và ánh xạ Đ/đ → D/d.

Bất biến kiến trúc

`ascii_key` được lưu *cạnh* `normalized_key`, không thay thế nó: cách viết có dấu luôn thẳng, còn việc bỏ dấu chỉ cứu những đầu vào lẽ ra rơi xuống tầng fuzzy. Migration 0003 sinh bản đồ bỏ dấu cho SQL từ `unicodedata` — không bao giờ gõ tay hai chuỗi `translate()` song song, chúng ghép lệch mà không báo lỗi.

8.3 Ràng buộc quan trọng

Ràng buộc	Ý nghĩa
<code>uq_registry_single_published</code>	Partial unique index: đúng <i>một</i> version PUBLISHED. Danh mục nhân sự có index tương đương, riêng biệt.
<code>uq_case_actor_idempotency</code>	(<code>created_by</code> , <code>idempotency_key</code>) duy nhất.
<code>VerificationResult.case_id</code>	Duy nhất: một Case có tối đa một kết quả.
<code>uq_assessment_result_policy</code>	Một rule chỉ đánh giá một lần cho một kết quả.
<code>uq_unit_name</code> , <code>uq_unit_code_namespace</code>	Chống trùng bí danh/mã.
<code>uq_bulk_row_job_index</code>	Mỗi dòng trong một job là duy nhất.
<code>app_users.email</code> , <code>app_users.personal_code</code>	Duy nhất toàn hệ thống (migration 0008); lưu đã chuẩn hoá — email viết thường, mã viết hoa — nên index thường vẫn không phân biệt hoa thường mà không cần collation riêng của CSDL. NULL vẫn khác nhau, nên tài khoản không khai cả hai trường không bị ảnh hưởng.
<code>ck_person_synthetic_welfare_source</code>	<code>synthetic_welfare_facts</code> chỉ được phép tồn tại khi <code>source_kind='SYNTHETIC_DEMO'</code> .
Các CHECK về validity	<code>valid_from<=valid_to</code> ở mọi thực thể có hiệu lực thời gian.
Trigger append-only	<code>audit_logs</code> chỉ cho phép INSERT (migration 0004).

Bất biến kiến trúc

Địa chỉ email duy nhất là một *thuộc tính bảo mật*, không phải sự gọn gàng: mật khẩu cấp phát và mọi mã OTP đều giao tới đó, nên hai tài khoản dùng chung một hộp thư đồng nghĩa trao cho người giữ hộp thư một lối vào với tư cách của cả hai.

8.4 Khối bằng chứng

`VerificationResult.evidence` là hợp đồng truy vết của hệ thống. Nó chứa tối thiểu:

- Đầu vào: `current_unit_raw`, `former_units`, `as_of_date`.
- Phân giải: `canonical_name`, `match_method`, `resolution_score`, `candidate_margin`, `calibration_status`, `resolution_confidence`.
- Định danh: toàn bộ khối `person_resolution` và `person_registry_version`.
- Trích xuất: `extraction_confidence`, `decision_extraction_confidence`, `relation_confidence`.
- Đọc tài liệu: `parse_method`, `parse_confidence`, `parse_quality`, `parse_evidence`.
- Nhóm đối tượng: `subject_group_method`, `subject_group_confidence`.
- Chính sách: `policy_summary`, `policy_as_of_date`.
- Nguồn gốc: `source_kind`, `unit_source_kind`, `bulk_source`, `split_source`, `operator_correction`.
- Sáu trục version: `registry_version`, `taxonomy_version`, `parser_version`, `model_version`, `threshold_version` (và `policy_version` nằm trên từng `assessment`).

`GET/cases/{id}` trả về trường `extracted` (hàng `ExtractedRecord`) *bên cạnh* `case` và `result`. Hãy dùng trường đó để biết pipeline đã rút ra gì, thay vì phân tích `result.evidence`.

`ExtractedRecord.document_id` được ghi bởi `_upsert_extracted_record` qua `_source_document_id`, và *chỉ khi* `Case` có đúng một `Document`. Có nhiều `Document` thì trường này để `None` thay vì đoán tài liệu nào sinh ra trường nào — đoán ở đây là tạo ra bằng chứng mà nền tảng không đứng sau được.

8.5 Dữ liệu danh mục và seed

Thứ tự phân giải nguồn seed:

1. `REGISTRY_SEED_PATH`
2. `/app/registry/master_units_registry.csv` (mount vào dịch vụ seed)
3. `artifacts/registry/published/master_units_registry.csv`
4. `datasets/curated/master_units_baseline43.csv`
5. `/app/bootstrap/master_units_baseline43.csv`

Giới hạn / cảnh báo

`artifacts/` **không** được theo dõi bởi git — nó chứa danh mục đã tuyển chọn (~2.5k đơn vị, trong đó 2.077 bản ghi `APPROVED`) cùng bằng chứng `crawl` thô, và không có trong một bản clone mới. Đừng nhầm `datasets/curated/master_units_baseline43.csv` là danh mục thật; nó chỉ là bản dự phòng khởi động.

Seed tôn trọng cột `qa_status` của chính tệp CSV và bỏ qua mọi bản ghi không phải `APPROVED`; không bao giờ hard-code cột này thành `APPROVED`. Phân giải sản xuất chỉ được dùng đơn

vị/tên/mã đang active và đã QA. Đơn vị tạo tay bắt đầu ở `PENDING_QA`. Giữ lại hiệu lực lịch sử của đơn vị thay vì xoá dữ kiện lịch sử.

Danh mục nhân sự *chưa* có script seed gắn vào `make seed: apps/backend/scripts/seed_persons.py` tồn tại để bootstrap một danh sách nhân sự tổng hợp (khớp người với `canonical_unit_id` đã phải tồn tại ở trạng thái APPROVED) nhưng không chạy tự động — hãy gọi tường minh khi cần.

8.6 Migration

Migration	Nội dung
0001_initial	Lược đồ nền: Case, danh mục, chính sách, thẩm định, audit, outbox.
0002_input_intelligence_bulk	Nạp theo lô và siêu dữ liệu định tuyến đầu vào.
0003_ascii_folded_resolution	<code>ascii_key</code> và bản đồ bỏ dấu sinh từ <code>unicodedata</code> .
0004_audit_append_only	Trigger PostgreSQL chặn UPDATE/DELETE trên audit.
0005_person_registry	Toàn bộ danh mục nhân sự và ảnh chụp của nó.
0006_local_accounts	Tài khoản, phiên, quyền nội bộ (thay cho Keycloak).
0007_login_otp	Bảng <code>login_otps</code> .
0008_unique_account_identity	Unique index cho email và <code>personal_code</code> .

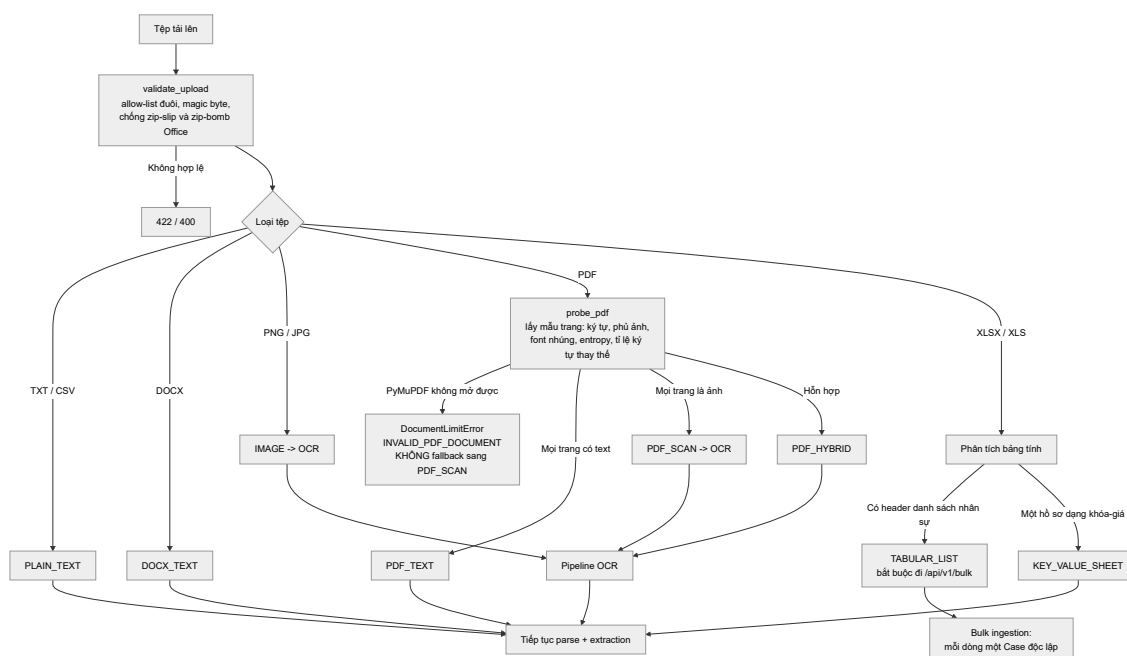
Hiệu lực ngày nghiệp vụ đi qua `_validate_business_date` trong `shared/schemas.py`; nạp theo lô dùng lại đúng hàm đó nên một cột trong bảng tính không thể vượt qua rào chắn.

CHƯƠNG 9

Hiểu tài liệu (Document Intelligence)

Toàn bộ việc đọc tài liệu — parse, OCR, trích xuất, gate chất lượng, tách nhiều chủ thể — nằm trong `modules/document_intelligence/`. Không có gói `cabqp/ocr/` ở mức cao nhất; đường dẫn đó không tồn tại.

9.1 Định tuyến đầu vào



Hình 9.1. `route_input`: quyết định chiến lược đọc *trước* khi parse

`route_input` quyết định chiến lược trước bất kỳ thao tác đọc nào:

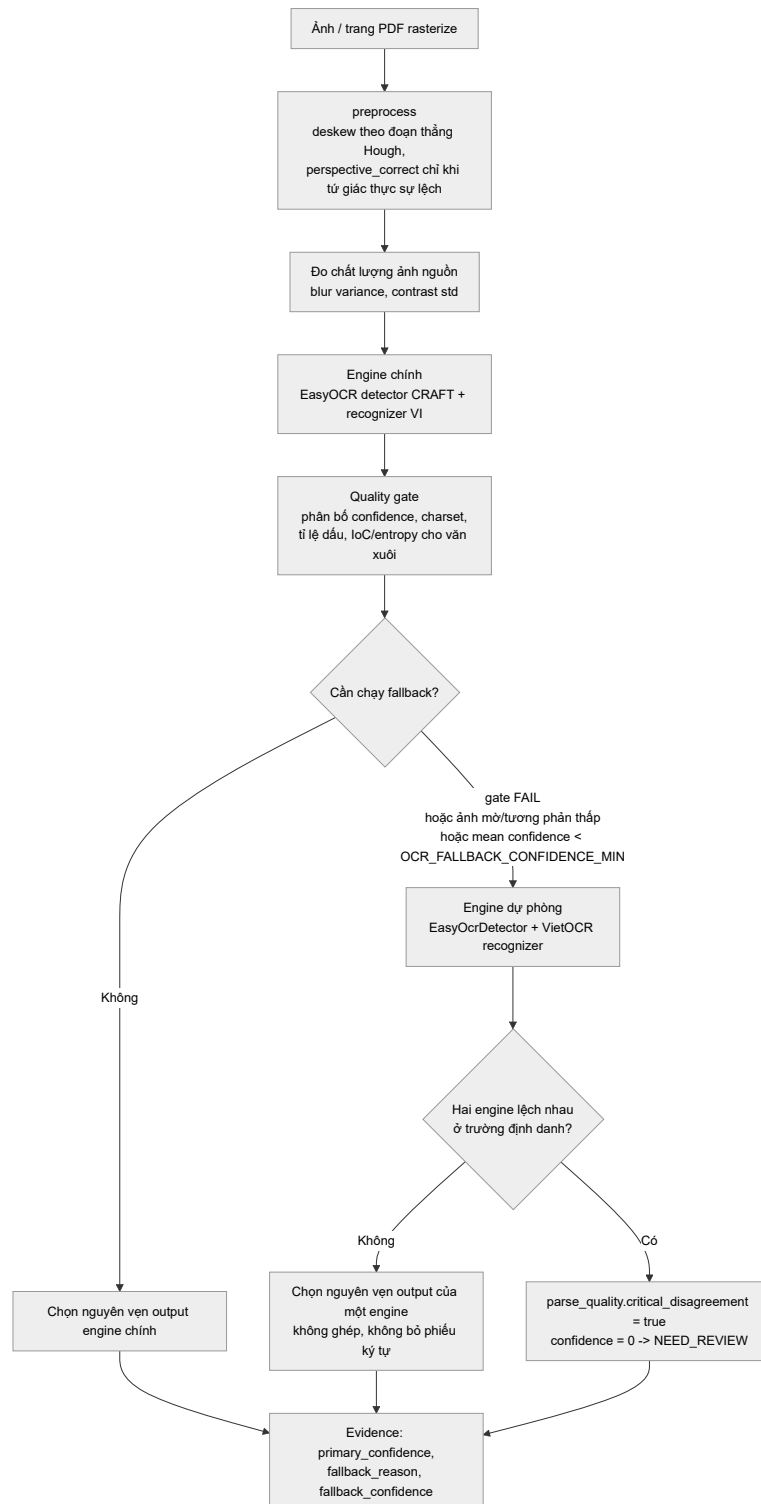
- PDF được thăm dò *theo trang* (`probe_pdf`) trên một mẫu trang, đo số ký tự, độ phủ ảnh, số font nhúng, entropy ký tự và tỉ lệ ký tự thay thế, rồi kết luận `PDF_TEXT` / `PDF_SCAN` / `PDF_HYBRID`.
- Bảng tính được lập hồ sơ thành `TABULAR_LIST` (thuộc về nạp theo lô, mỗi dòng một Case) hoặc `KEY_VALUE_SHEET` (một hồ sơ).
- Mọi tệp tải lên đi qua `validate_upload` trước: allow-list phần mở rộng, khớp magic byte, rào chắn đường dẫn và tỉ lệ giải nén cho ZIP của Office — nên nhánh "nhị phân không rõ" của router chỉ còn với tối được từ bên trong.

Bất biến kiến trúc

Kiểm tra magic byte chỉ đọc phần header, nên một PDF bị cắt cụt hoặc hỏng vẫn tới được `probe_pdf`. Khi PyMuPDF không mở được, đó là `DocumentLimitError("INVALID_PDF_DOCUMENT")`, **không** phải fallback sang `PDF_SCAN`. Mọi đường quét đều rasterize qua chính thư viện đó, nên fallback cũ không bao giờ đọc được tệp — nó chỉ dời lỗi vào OCR rồi báo cáo sai rằng OCR có vấn đề.

Một tài liệu có cấu trúc bảng (bảng có header danh sách nhân sự thật) là một đường *hoàn toàn khác* và bị từ chối khỏi endpoint tạo Case đơn với mã `TABULAR_LIST_REQUIRES_BULK`, chuyển client sang `/api/v1/bulk`. Một bảng chỉ thiếu đường kẻ (bố cục PDF không viền) vẫn đi qua trích xuất bảng của bulk nếu PP-StructureV3 hoặc parse bảng gốc dựng lại được hàng. Một danh sách nhân sự trình bày mỗi trường một dòng, không có cấu trúc hàng/cột dựng lại được, hiện *chưa* được bắt kỳ thành phần nào tách và ở lại như một Case đơn (nhiều khả năng `NEED_REVIEW`).

9.2 OCR



Hình 9.2. Pipeline OCR: một engine chính, đúng một fallback được cấu hình

OCR mặc định dùng recognizer Latin tiếng Việt của **EasyOCR**, với đúng *một* fallback được cấu hình: **VietOCR** (detector EasyOCR + recognizer VietOCR). VietOCR chỉ nhận dạng — nó

phiên âm một dòng đã cắt và không có detector — nên engine fallback dùng lại tầng phát hiện CRAFT của EasyOCR (EasyOcrDetector) và chỉ thay recognizer.

Fallback chạy khi:

1. Gate chất lượng của engine chính không đạt, **hoặc**
2. Ảnh nguồn bị mờ / tương phản thấp, **hoặc**
3. Độ tin cậy nhận dạng trung bình của engine chính thấp hơn `OCR_FALLBACK_CONFIDENCE_MIN` (0.70).

Lý do thiết kế

Điều kiện thứ ba tồn tại vì một bản quét khoá—giá trị ngẫu nhiên quá ít dòng để trượt các chỉ số phân bố, ngay cả khi bản thân recognizer đang rất không chắc. Riêng gate vì thế bỏ sót đúng những lần đọc đáng chạy lại nhất.

`run_ocr` ghi vào bằng chứng `primary_confidence`, `fallback_reason` và `fallback_confidence`. Nó vẫn chọn *trọn vẹn* output của một engine và không bao giờ ghép hay bỏ phiếu theo ký tự.

Bất biến kiến trúc

Khi hai engine bất đồng ở các trường định danh, gate hỏng (`parse_quality.critical_disagreement=true`) và độ tin cậy về 0. Đây là đường fail-closed có chủ đích, không phải lỗi: một bản quét nhòe mà cả hai engine đọc sai theo hai kiểu khác nhau phải trừu xuất, không được lấy trung bình hai phỏng đoán.

Tham số EasyOCR/VietOCR là *cấu hình*, không phải thứ thay thế theo từng tài liệu; Docker bake mọi tài sản mô hình để runtime luôn ngoại tuyến.

9.2.1 Các engine đã bị gỡ bỏ

Hai engine Paddle và Tesseract đã bị gỡ (PaddleEngine, PaddleDetector, PaddleRecognizer, TesseractEngine, preprocess_for_tesseract, **phụ thuộc** `pytesseract` và các gói `apt tesseract-ocr`). `paddleocr` vẫn là **phụ thuộc** nhưng **chỉ** cho nhận dạng cấu trúc bảng PP-StructureV3 (`document_intelligence/table.py`) — một đường khác hẳn nhận dạng văn bản. Không đưa nó trở lại làm engine OCR.

9.2.2 VietOCR được vendor

VietOCR nằm tại `apps/backend/vendor/vietocr` (trong build context của Docker; nguồn upstream `pbcquoc/vietocr`, giấy phép MIT) và được cài bằng `pip install --no-deps -e apps/backend/vendor/vietocr`, *không bao giờ* từ PyPI: `sdist` công bố ghim cứng `pillow==10.2.0`, điều mà `pdfplumber` (`Pillow>=12.2`) loại trừ. Các nhu cầu runtime của nó (`einops`, `gdown>=5.2`, `torch/torchvision`, `PyYAML`) được khai báo trong `apps/backend/pyproject.toml`; `gdown<5` không dùng được vì `vietocr.tool.utils` import nó ngay lúc nạp module và nó lại import `pkg_resources`, thứ mà `setuptools>=81` không còn ship.

`engines.py` cũng từ chối `Cfg.load_config_from_name()` của upstream: hàm đó tải cả hai tệp YAML từ `vocr.vn` ngay lúc khởi tạo. Engine đọc

`vendor/vietocr/config/base.yml` cộng tệp kiến trúc từ đĩa (`VIETOCR_CONFIG_DIR` ghi đè vị trí; image đặt `/opt/vietocr/config`). Checkpoint là `VIETOCR_WEIGHTS` khi tệp đó tồn tại (bake vào image, kèm `VIETOCR_DOWNLOAD_ENABLED=false`), ngược lại được tải một lần vào `~/.cache/cabqp/vietocr/` — ghi ra `.partial` rồi đổi tên, nên một lần tải dở dang không bao giờ bị nhận nhầm là checkpoint hợp lệ.

9.2.3 Tiền xử lý ảnh

Giới hạn / cảnh báo

`deskew()` đo góc xoay chỉ từ các đoạn thẳng Hough. Trước đây nó còn trung bình thêm một góc dựa trên `minAreaRect` của mọi điểm tối trên trang; với một trang gồm nhiều dòng văn bản rời, hình chữ nhật đó chính là bounding box thẳng trục của trang, nên trả về một góc vô nghĩa (đo được đúng 90° trên một bản quét nhiều dòng không hề xoay), kéo trung bình về một góc sai và cắt cụt đầu mỗi dòng theo mức tăng dần. **Không** đưa lại góc `minAreaRect` toàn trang; nếu cần thêm bộ dò nghiêng, hãy kiểm chứng trước rằng nó không kích hoạt trên một bản quét nhiều dòng thẳng.

Tương tự, `perspective_correct()` chỉ warp khi tứ giác phát hiện được thực sự nghiêng (đối chiếu với các góc thẳng trục): khung viền trang trí của một ảnh chụp màn hình hay bản quét phẳng là một hình chữ nhật thẳng nhưng vẫn thỏa tìm kiếm "contour 4 điểm lớn nhất", và warp theo nó sẽ cắt mất lề thật của tài liệu.

9.3 Trích xuất

`extraction.py` so khớp nhãn (`label_values`, `spatial_key_values`, `_is_label`) bằng `rapidfuzz.fuzz.ratio` với ngưỡng 75 trên tập alias `LABELS` — không phải so chính xác hay so chuỗi con.

Lý do thiết kế

Bản quét thật làm rơi hoặc méo từng ký tự một cách khó đoán: lúc ở đầu dòng, lúc là một nguyên âm giữa từ ("chc v" cho "chức vụ", "CCCO" cho "CCCD"). Không quy tắc cố định theo vị trí nào bắt được mọi trường hợp.

`_inline_label_value` còn thử các tiền tố dòng ngắn khi OCR mất dấu hai chấm hoặc dính các từ của nhãn (`ChucvChuyenvien`); nó giữ nguyên phần văn bản còn lại của mô hình và không bao giờ thay bằng một giá trị gán cứng theo hồ sơ. `spatial_key_values` thử dạng inline này *trước* khi tìm sang các hộp OCR lân cận, nhờ đó nhãn kế tiếp không bị gán nhầm thành giá trị của nhãn hiện tại.

Giá trị Nhóm đối tượng khai tường minh được giữ lại cho bộ kiểm tra nhóm đối tượng; các nhãn quá rộng như BQP vẫn trừu xuất vì chúng không phân biệt được quân nhân với cơ yếu.

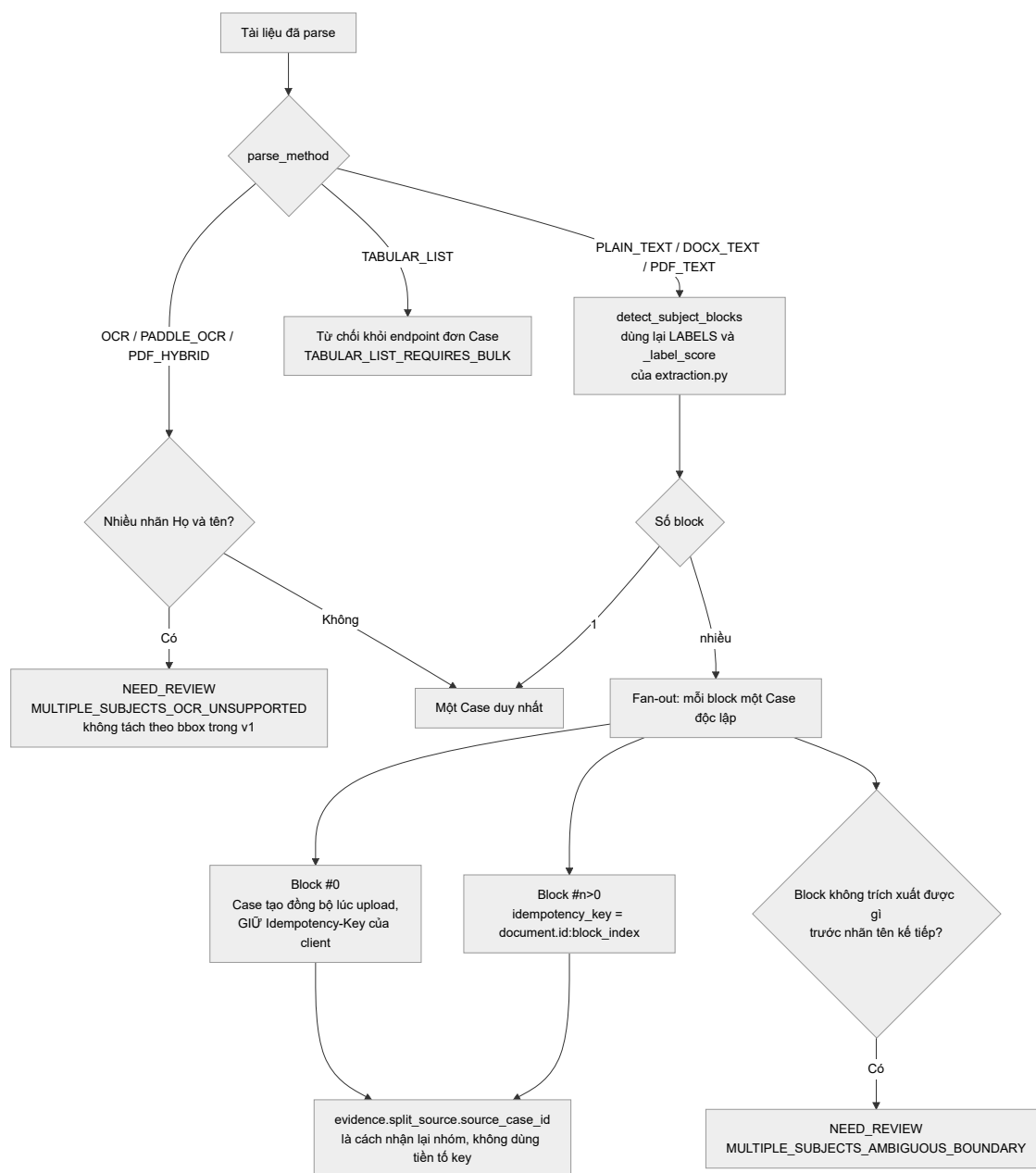
Bất biến kiến trúc

Nhãn lịch sử có định ngữ là một trường `LABELS` riêng (`former_unit_name`: "Đơn vị công tác cũ", "Đơn vị công tác trước đây", "Nguyên đơn vị công tác", ...), không bao giờ là biến thể của `unit_name`. Ở ngưỡng 75, các cách viết đó đạt khoảng 91 điểm so với alias

của `unit_name`, nên trước kia chúng bị đọc thành `CURRENT_WORK_UNIT`; và vì nhánh `CURRENT_MARKERS` theo văn tường thuật chỉ chạy khi *không* tìm được gì, đơn vị hiện tại thật của tài liệu bị mất hẳn. `former_unit_name` được liệt kê *sau* `unit_name` để một nhãn đơn vị hiện tại chính xác vẫn thắng khi hai bên bằng điểm.

Trích xuất theo văn tường thuật cắt theo *cả hai* chiều tại marker kế tiếp (`_cut_at_markers`): `_clean_unit` chỉ tách theo dấu câu, nên một bản quét mất dấu ngắt câu từng sinh ra một "đơn vị cũ" trải dài qua cả hai mệnh đề ("Cục A hiện đang công tác tại Cục B"). `_earliest_marker` chọn marker *được nhắc đến sớm nhất* chứ không phải marker đứng đầu danh sách, vì `hiệnđangcôngtáctại` chứa `đangcôngtáctại` và thứ tự danh sách có thể cắt giữa một marker.

9.4 Tách nhiều chủ thể



Hình 9.3. Tách nhiều chủ thể và fan-out thành các Case anh em

Một tài liệu văn bản tự do (PLAIN_TEXT/DOCX_TEXT/PDF_TEXT) chứa nhiều hơn một khối gắn nhãn "**Họ và tên**" **không** bị âm thầm rút gọn về người đầu tiên. `detect_subject_blocks` tách thành mỗi người một block và `process_document` fan-out thành các Case độc lập — phản chiếu đúng mẫu một-Case-một-dòng của pipeline lô, mỗi Case anh em được khoá bởi một idempotency key ổn định `f" {document.id} : {block_index}"` nên việc chạy lại không bao giờ nhân bản Case.

`detect_subject_blocks` dùng lại chính bộ máy LABELS/_label_score của `extraction.py` cho việc dò ranh giới thay vì nhân bản ngưỡng fuzzy — hãy giữ hai bên đồng bộ nếu `_LABEL_FUZZY_THRESHOLD` thay đổi.

Bất biến kiến trúc

Tài liệu OCR/hybrid (OCR/PADDLE_OCR/PDF_HYBRID) có nhiều nhãn tên **không** được tách theo hình học ở v1 — dò ranh giới dựa trên bbox theo thứ tự dòng OCR không đủ đáng tin, nên cả tài liệu trồi xuất về NEED_REVIEW (MULTIPLE_SUBJECTS_OCR_UNSUPPORTED) thay vì đoán. Một block đã phát hiện nhưng không trích xuất được gì trước nhãn tên kế tiếp cũng trồi xuất (MULTIPLE_SUBJECTS_AMBIGUOUS_BOUNDARY) chứ không nhập vào block bên cạnh.

Bản thân `extract()` không bao giờ được gọi với văn bản của nhiều hơn một người và không bị thay đổi bởi cơ chế này: việc tách xảy ra *trước* trích xuất, mỗi block một lời gọi.

9.5 Gate chất lượng và ngưỡng

Bất biến kiến trúc

Ngưỡng chất lượng nằm ở `apps/backend/config/document_quality.yaml` và được đóng dấu lên mọi kết quả dưới tên `threshold_version`. Đổi một ngưỡng là đổi các quyết định đã ghi nhận, nên phải tăng version trong *cùng một lần sửa*.

Hai lưu ý về ngữ nghĩa gate:

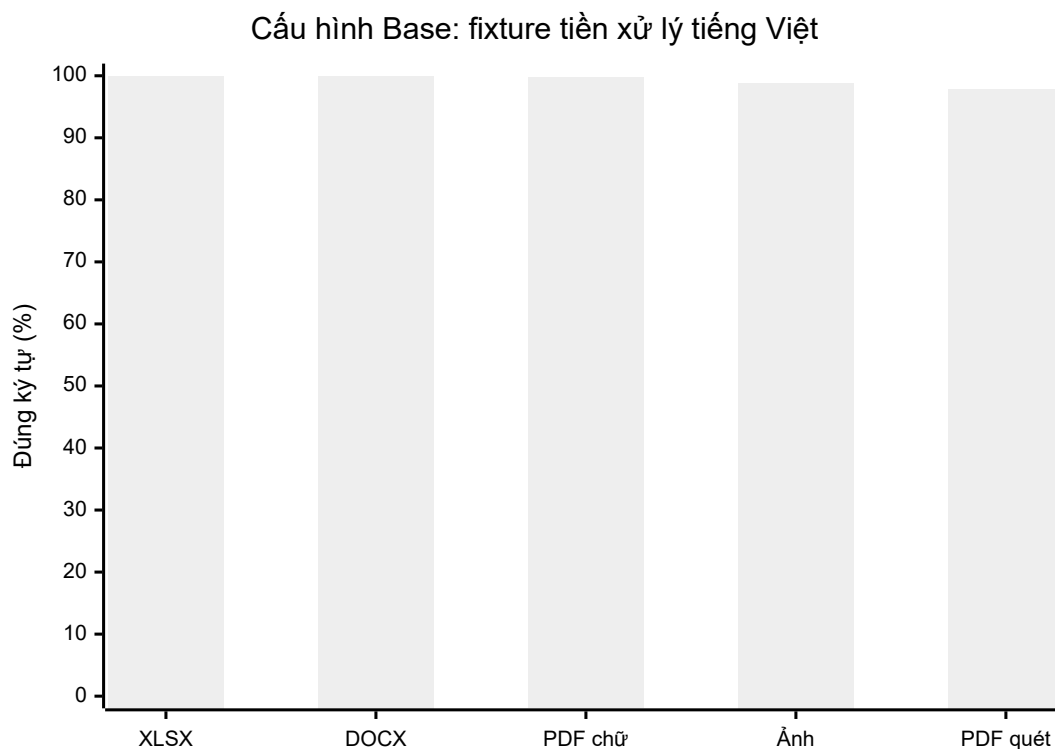
- IoC và entropy chỉ áp dụng cho văn bản *dạng văn xuôi*; áp thống kê phân bố ngôn ngữ lên các dạng khoá-giá trị ngắn sinh ra thất bại giả.
- Chất lượng OCR bao gồm cả độ mờ và độ tương phản của *ảnh nguồn*, độc lập với độ tin cậy của recognizer, vì một recognizer có thể rất tự tin mà vẫn sai trên bản quét kém.

9.6 Baseline đo lường của pipeline tiền xử lý

Trong tài liệu này, **Base** tương ứng với **tier Lite** của `_final_staging/BENCHMARK_REPORT.md`, phiên bản `bench-1.0.0` trên 2 GPU V100 32 GB. Phạm vi gồm 7 tệp nghiệp vụ thật có nhãn vàng, 9 fixture tiếng Việt, 40 tài liệu E2E, 100 bảng WikiTableQuestions và 200 trang OmniDocBench. Không dùng số Mini, Pro hoặc Max để đại diện cho cấu hình đã chọn.

Bảng 9.1. Độ chính xác tiền xử lý của cấu hình Base

Định dạng	Tệp thật: đúng ký tự	Tệp thật: đúng ô	Fixture Việt: đúng ký tự	Fixture Việt: đúng ô
XLSX	100,00%	100,00%	100,00%	100,00%
DOCX	99,84%	86,57%	100,00%	100,00%
PDF có lớp chữ	99,93%	99,42%	99,72%	100,00%
Ảnh	56,95%	0,00%	98,86%	90,48%
PDF quét	53,95%	0,00%	97,87%	95,24%

**Hình 9.4.** Độ chính xác fixture tiền xử lý tiếng Việt của cấu hình Base**Bảng 9.2.** Kết quả tổng hợp của cấu hình Base

Nhóm đo	Kết quả	Diễn giải
E2E: độ bao phủ/độ chính xác người	95,26% / 86,17%	Trung bình 5 định dạng
E2E: đúng đơn vị hiện tại	50,00%	Còn là điểm cần cải thiện
E2E: cần thẩm định	72,80%	Pipeline ưu tiên trừu xuất khi chưa đủ căn cứ
OmniDocBench: khoảng cách sửa	0,3213	Thấp hơn là tốt hơn
OmniDocBench: BLEU/TEDS cấu trúc	0,4787 / 0,5228	Phép tính giản lược của <code>benchmark.py</code>
OmniDocBench: độ trễ trung bình	3,74 giây/trang	Chưa phải p50/p95

Giới hạn / cảnh báo

Kết quả structured và PDF có lớp chữ tốt, nhưng ảnh/PDF quét nghiệp vụ thật còn yếu; đúng ô bằng 0% trên cỡ mẫu hiện tại. WikiTableQuestions và OmniDocBench dùng tiếng Anh/Latin, không thay thế benchmark nghiệp vụ tiếng Việt. Cỡ mẫu 7 tệp thật cũng không đủ để tuyên bố capacity hoặc chất lượng production; cần báo cáo p50/p95/p99, throughput, peak VRAM và raw artifact khi nghiệm thu.

9.7 Hạn mức tài nguyên và an toàn tệp

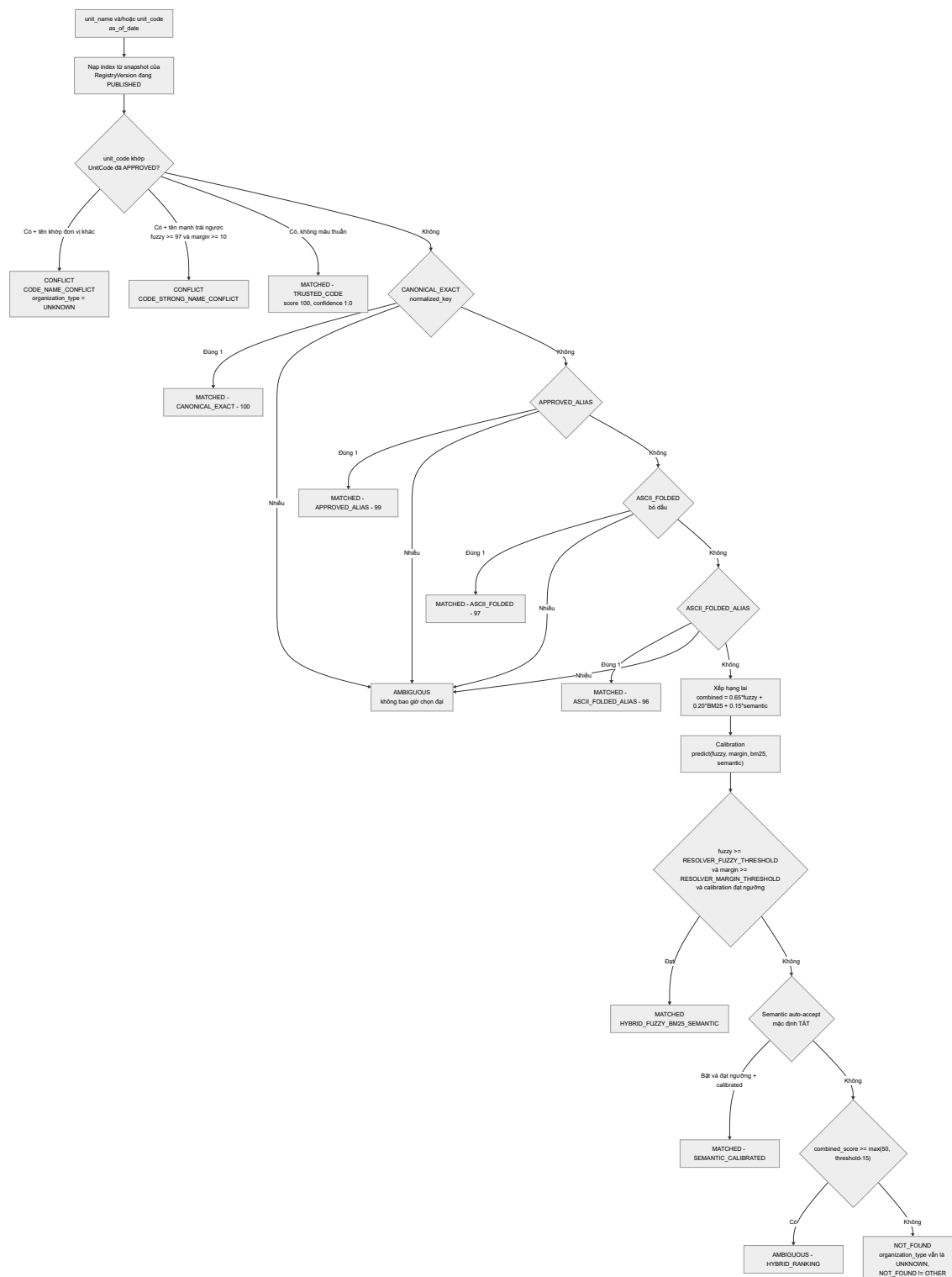
Cấu hình	Ý nghĩa
MAX_UPLOAD_MB (25)	Kích thước tệp tối đa.
DOCUMENT_MAX_PDF_PAGES (200)	Số trang PDF tối đa.
DOCUMENT_MAX_OCR_PAGES (20)	Số trang chạy OCR tối đa.
DOCUMENT_MAX_IMAGE_PIXELS (40M)	Chặn ảnh bom.
DOCUMENT_MAX_RASTER_PIXELS (200M)	Chặn rasterize quá lớn.
DOCUMENT_MAX_TABLES (200), DOCUMENT_MAX_WORKSHEETS (50)	Chặn tài liệu bệnh lý.
DOCUMENT_MAX_SPREADSHEET_ROWS (100.000)	Rào chắn bộ nhớ của parser.
DOCUMENT_SOFT_TIMEOUT_SECONDS (240)	Giới hạn mềm cho task tài liệu.
ANTIMALWARE_REQUIRED	Ở production bắt buộc bật (fail-closed).

Tệp gốc lưu ở object storage; CSDL chỉ giữ siêu dữ liệu và checksum. `FilesystemStorage` ghi `.partial` rồi đổi tên, và từ chối mọi key có thể thoát khỏi `STORAGE_ROOT`.

CHƯƠNG 10

Phân giải và quản trị danh mục

10.1 Cascade phân giải đơn vị



Hình 10.1. Cascade phân giải tên/mã đơn vị

Thứ tự bằng chứng, mạnh nhất trước:

TRUSTED_CODE (100) → CANONICAL_EXACT (100) → APPROVED_ALIAS (99) → A

SCII_FOLDED (97) →
ASCII_FOLDED_ALIAS (96) → fuzzy/BM25/semantic → ABSTAIN

Bất biến kiến trúc

Bỏ dấu là tầng *chính xác cuối cùng*: cách viết có dấu luôn thắng, và việc bỏ dấu chỉ cứu những đầu vào lẽ ra đã rơi xuống so khớp mờ. Điểm số phản ánh đúng thứ tự đó: cách viết chuẩn mạnh nhất, rồi đến bí danh do con người duyệt, rồi đến so khớp bỏ dấu (do máy suy ra, nên xếp thấp nhất trong các tầng chính xác).

10.1.1 Xử lý mã đơn vị

Mã đơn vị được coi là bằng chứng tin cậy nhất, nhưng không được phép che mâu thuẫn:

- Mã và tên trở về hai đơn vị khác nhau → CONFLICT (CODE_NAME_CONFLICT), organization_type=UNKNOWN.
- Mã hợp lệ nhưng tên (dù không phải alias chính xác) khớp *rất mạnh* với một đơn vị khác (fuzzy ≥ 97 và margin ≥ 10) → CONFLICT (CODE_STRONG_NAME_CONFLICT).
- Không mâu thuẫn → MATCHED / TRUSTED_CODE, điểm 100, độ tin cậy 1.0.

10.1.2 Xếp hạng lai

Khi không có tầng chính xác nào khớp, hệ thống xếp hạng ứng viên bằng tổ hợp tuyến tính:

$$combined = 0,65 \cdot \frac{fuzzy}{100} + 0,20 \cdot BM25 + 0,15 \cdot semantic$$

Lý do thiết kế

Tầng semantic *không* được tính khi bằng chứng từ vựng đã đủ dứt khoát (fuzzy vượt ngưỡng cộng 3 điểm và margin vượt ngưỡng cộng 4 điểm). Đây là tối ưu chi phí có chủ đích: nạp một transformer là thao tác đắt nhất trong toàn bộ đường phân giải và không thay đổi kết quả trong những trường hợp đó.

Điều kiện tự động chấp nhận:

Nhánh	Điều kiện
HYBRID_FUZZY_BM25_SEMANTIC	fuzzy ≥ RESOLVER_FUZZY_THRESHOLD (91) và margin ≥ RESOLVER_MARGIN_THRESHOLD (6) và yêu cầu calibration được thoả (RESOLVER_REQUIRE_CALIBRATION_FOR_FUZZY_AUTOACCEPT mặc định bật, ngưỡng xác suất RESOLVER_CALIBRATED_ACCEPT_THRESHOLD = 0,90).
SEMANTIC_CALIBRATE D	Chỉ khi RESOLVER_ALLOW_SEMANTIC_AUTOACCEPT được bật — mặc định tắt — và điểm semantic cùng margin semantic đều vượt ngưỡng và đã calibrated.

Nhánh	Điều kiện
AMBIGUOUS	$\text{combined_score} \geq \max(50, \text{ngngfuzzy} - 15)$ nhưng không đủ để chấp nhận.
NOT_FOUND	Dưới mức đó.

Bất biến kiến trúc

UNKNOWN vẫn là UNKNOWN. **Không bao giờ** ánh xạ NOT_FOUND thành OTHER khi không có bằng chứng dương từ danh mục. Cả nhánh AMBIGUOUS lẫn NOT_FOUND đều trả `organization_type=UNKNOWN` và `unit_id=None`.

10.1.3 Calibration

`modules/resolution/calibration.py` nhận bốn đặc trưng (`fuzzy`, `margin`, `bm25`, `semantic`) và trả về một xác suất kèm phương pháp/version, được ghi vào `calibration_status` trên bằng chứng. Artifact mặc định nằm ở `artifacts/resolver_calibration.json` (`RESOLVER_CALIBRATION_PATH`).

Giới hạn / cảnh báo

Artifact hiện tại được fit từ nhiều mention tổng hợp xác định của danh mục trong repo. Nó phù hợp cho cổng chất lượng ở mức dự án/demo và **phải** được fit lại trên dữ liệu kiểm chứng production có nhãn trước khi triển khai thật.

10.2 Phân giải nhân sự

`PersonResolver` chạy một cascade song song nhưng *khác biệt*:

1. `personal_code` tin cậy trước tiên; mã khớp nhiều hoặc mâu thuẫn thì trừu xuất.
2. Bốn tầng tên chính xác, chấm điểm giống hệt cascade đơn vị.
3. Fallback mờ *chỉ* bằng `rapidfuzz.fuzz.WRatio`.

Bất biến kiến trúc

`PersonResolver` **không** có tầng BM25, **không** có tầng semantic và **không** có mô hình calibration — khác hẳn xếp hạng lai của `Resolver` đơn vị. Một kết quả `PersonResolution` chỉ bao giờ sinh ra một `canonical_unit_id`; `Resolver` đơn vị được gọi lại trên ID đó để lấy `organization_type` trước khi bất kỳ Case nào được đánh dấu MATCHED cho phạm vi BCA/BQP.

`PersonRegistryService` phản chiếu một-đối-một hình dạng endpoint và vòng đời version DRAFT → VALIDATED → APPROVED → PUBLISHED của Unit Registry, với trạng thái QA và advisory lock riêng — được version hoá độc lập hoàn toàn.

10.3 Quản trị danh mục

10.3.1 Luồng dữ liệu vào danh mục

Nguồn chính thức/được cung cấp → ứng viên từ crawler/import → chuẩn hoá + khử trùng → PENDING_QA → QA của quản trị viên → ảnh chụp DRAFT → VALIDATED → APPROVED → PUBLISHED (bất biến)

pipelines/registry_ingestion/ thực hiện phần ngoại tuyến (crawl, extract, normalize, dedupe, coverage) và dùng chung cabqp.shared.normalization nên khoá không bao giờ lệch giữa pipeline và runtime. pipelines/synthetic_data/ chỉ sinh artifact tổng hợp và không bao giờ được âm thầm nâng lên thành dữ liệu có thẩm quyền.

10.3.2 Cường chế vòng đời

Thao tác	Ràng buộc do dịch vụ cường chế
Sửa đơn vị/alias/mã	Chỉ cho phép trên version DRAFT (409 nếu khác).
validate_version	Chỉ từ DRAFT; kiểm tra mọi bản ghi trong ảnh chụp đã APPROVED.
approve_version	Chỉ từ VALIDATED.
publish_version	Chỉ từ APPROVED; lấy pg_advisory_xact_lock, khoá các version PUBLISHED hiện có bằng SELECT...FORUPDATE, hạ chúng xuống, rồi nâng version mới lên.
rollback	Clone ảnh chụp cũ thành version mới rồi publish nguyên tử; <i>không</i> sửa ảnh chụp lịch sử.
decide_candidate	Duyệt ứng viên tạo đơn vị/alias mới ở trạng thái đã QA của version DRAFT, không tác động tới version đang phục vụ.

Bất biến kiến trúc

Chỉnh sửa của người thẩm định *có thể* tạo RegistryCandidate, nhưng **không được** tự tạo alias đã APPROVED. Đây là ranh giới ngăn hệ thống "tự học" từ các phán đoán đơn lẻ của con người thành dữ liệu sản xuất.

10.4 Tra cứu đơn vị trực tiếp

POST/lookup/unit là một lớp bọc mỏng quanh Resolver, phục vụ giao diện tra cứu trực tiếp và bộ chọn đơn vị trong màn hình thẩm định.

Giới hạn / cảnh báo

Đây là một lời gọi Resolver.resolve() đầy đủ, không phải endpoint autocomplete chuyên dụng: nó có thể mất vài giây khi cache lạnh (nạp mô hình embedding) và khoảng 2–3 giây khi đã ấm. Hãy debounce tương ứng và đừng kỳ vọng gợi ý dưới một giây.

CHƯƠNG 11

Quyết định, chính sách và thẩm định

11.1 Phân loại nhóm đối tượng

`classify_subject_group` là bộ phân loại theo luật, bảo thủ có chủ đích. Ba nhóm được công nhận: CAND, QUAN_NHAN, CO_YEU_HUONG_LUONG_NHU_QUAN_NHAN.

Thứ tự quyết định:

1. **Nhãn tường minh** trong tài liệu (`EXPLICIT_FIELD`, độ tin cậy 1.0). Giá trị không thuộc `KNOWN_GROUPS` trả về *không có nhóm* kèm phương pháp `INVALID_EXPLICIT_GROUP` — không phải một phỏng đoán.
2. **Luật theo văn bản** tùy theo `organization_type` đã phân giải: với BCA là các từ khoá công an; với BQP, “*cơ yếu*” được kiểm tra trước các từ khoá quân nhân.
3. Ngược lại: None, độ tin cậy 0, `INSUFFICIENT_EVIDENCE`.

Lý do thiết kế

Tiếng Việt chèn hệ từ và định ngữ giữa phủ định và danh từ (“*không còn là sĩ quan*”, “*không phải một quân nhân*”). Neo phủ định trực tiếp vào từ khoá đã bỏ sót những trường hợp đó và báo cáo chủ thể là quân nhân đang tại ngũ. Vì vậy `_NEGATION_FILLER` cho phép một chuỗi ngắn các từ đệm — có giới hạn, để phép khớp không trải qua cả một mệnh đề.

`_positive_not_negated` quét *mọi* lần xuất hiện của từ khoá và chỉ kết luận dương khi có ít nhất một lần không bị phủ định trong 40 ký tự trước đó.

Bất biến kiến trúc

Nhóm từ danh mục nhân sự (`subject_group_hint`) chỉ là gợi ý: `classify_subject_group` vẫn đối chiếu với `KNOWN_GROUPS`, nên một gợi ý lạ không bao giờ trở thành kết luận nghiệp vụ.

11.2 Bộ máy chính sách

`PolicyEngine.evaluate` chạy trên các `PolicyRule` đang active, lọc theo `effective_from/effective_to` so với `as_of`, rồi với mỗi rule quyết định:

Kết quả	Khi nào
NOT_APPLICABLE	<code>subject_groups</code> rỗng (lỗi cấu hình, không phải wildcard); hoặc nhóm của chủ thể không nằm trong phạm vi rule.

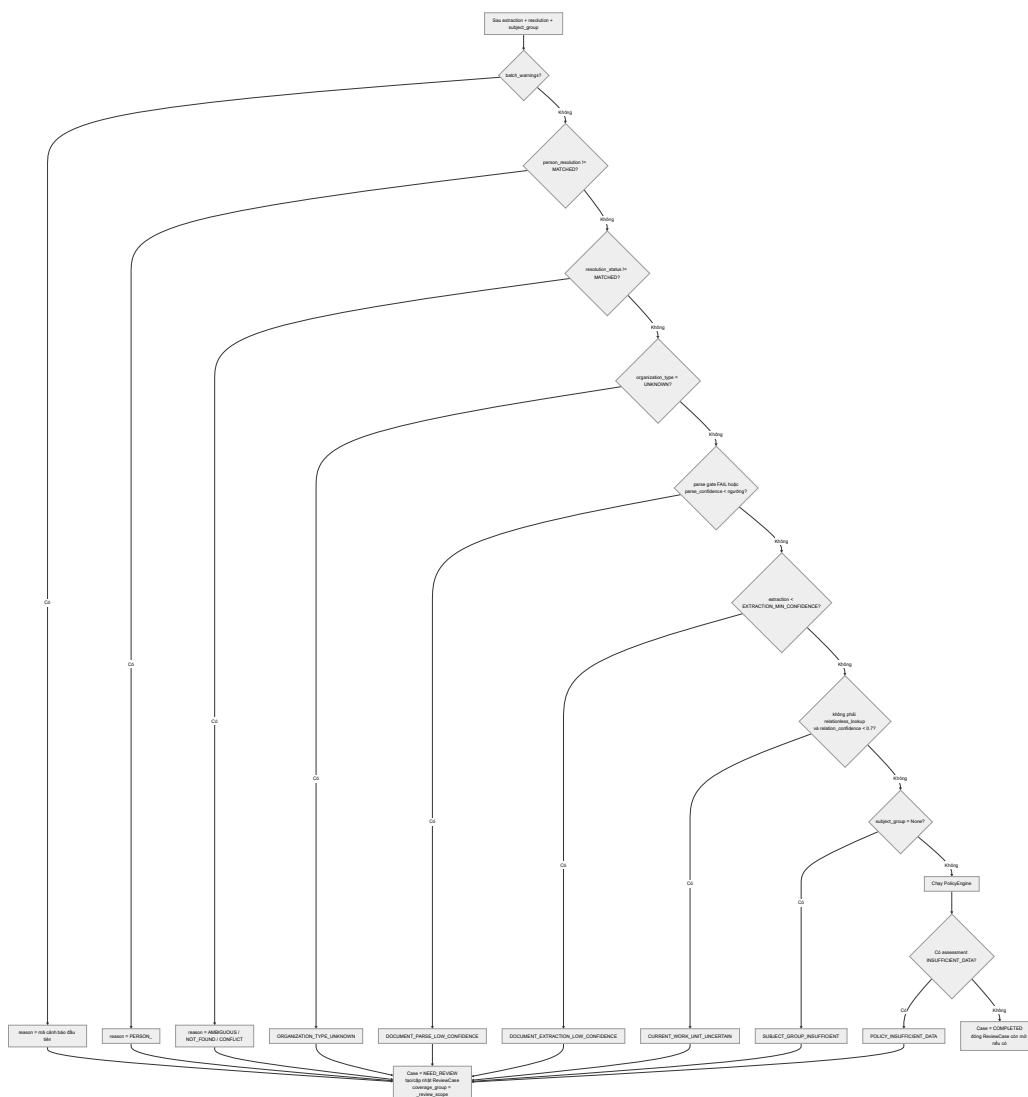
Kết quả	Khi nào
INSUFFICIENT_DATA	Thiếu <code>subject_group</code> ; hoặc thiếu trường bắt buộc trong <code>required_fields</code> .
ELIGIBLE	Đủ dữ kiện để xác định thuộc phạm vi áp dụng của rule đã version hoá.

Bất biến kiến trúc

`PolicyRule.subject_groups=[]` là *không hợp lệ* với tư cách wildcard ngầm. Rule toàn cục phải khai báo `['*']`. Đầu ra chính sách lúc chạy chỉ là *phạm vi/khả năng áp dụng*, trừ khi có công thức chính thức đã version hoá và đủ mọi dữ kiện bắt buộc; hệ thống **không** tự nghĩ ra số tiền.

Mỗi `EligibilityAssessment` lưu `facts_used` (đúng các trường mà rule yêu cầu), `source_ref`, `as_of_date` và cờ `scope_only`, cùng `policy_version` và `source_kind` của rule.

11.3 Thang điều kiện thẩm định



Hình 11.1. Thang điều kiện đưa Case sang NEED_REVIEW và lý do tương ứng

Thứ tự trong thang không tùy tiện: nó đi từ dữ kiện *cụ thể nhất* mà người thẩm định cần nhìn thấy đến dữ kiện tổng quát nhất. Lý do được ghi vào `ReviewCase.reason` và là thứ định hướng người thẩm định.

Bất biến kiến trúc

Cảnh báo lô tường minh là dữ kiện chất lượng dữ liệu hướng tới người vận hành và **không** được bị che bởi một nhãn AMBIGUOUS/NOT_FOUND chung chung ở hạ nguồn — vì vậy chúng đứng đầu thang. `reason` chỉ giữ mã của cảnh báo đầu tiên, nên payload mang đầy đủ `batch_warnings` kèm thông điệp tiếng Việt của từng cảnh báo.

`ReviewCase.payload` được dựng để người thẩm định không phải mở thêm màn hình nào khác:

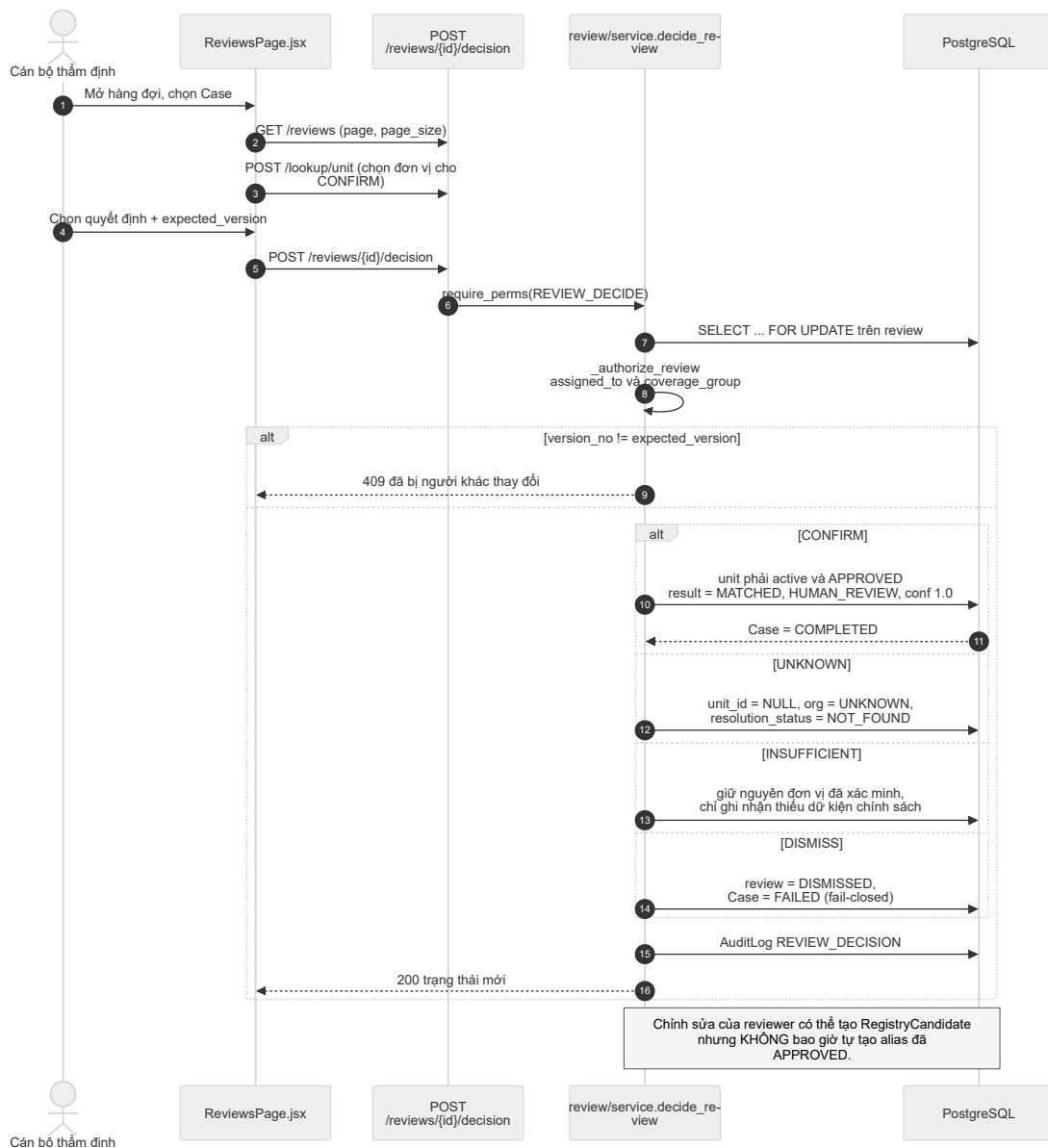
- `subject`: tên, mã, chức vụ, năm sinh — nếu thiếu, người mở hàng đợi chỉ thấy một Case id

và phải tra Case riêng để biết việc này nói về ai.

- resolution, top_candidates, person_resolution.
- confidence: từng thành phần độ tin cậy.
- thresholds: *ngưỡng* mà mỗi độ tin cậy đã bị so sánh với, kèm threshold_version — để người thẩm định thấy Case hụt bao nhiêu, không chỉ thấy rằng nó hụt.
- parse_quality, field_confidence, field_evidence.
- bulk_source, split_source, policy_assessments, as_of_date.

_review_scope gắn coverage_group: lấy nhóm của đơn vị đã phân giải; nếu chưa có, lấy nhóm chung duy nhất của tối đa 5 ứng viên đầu; nếu vẫn không rõ thì UNASSIGNED.

11.4 Quy trình thẩm định thủ công



Hình 11.2. Trình tự ra quyết định thẩm định

11.4.1 Ủy quyền

`_authorize_review` kiểm tra hai điều độc lập nhau: review đã được giao cho người khác chưa, và `coverage_group` của review có nằm trong phạm vi của người gọi không. Tài khoản quản trị vượt qua cả hai.

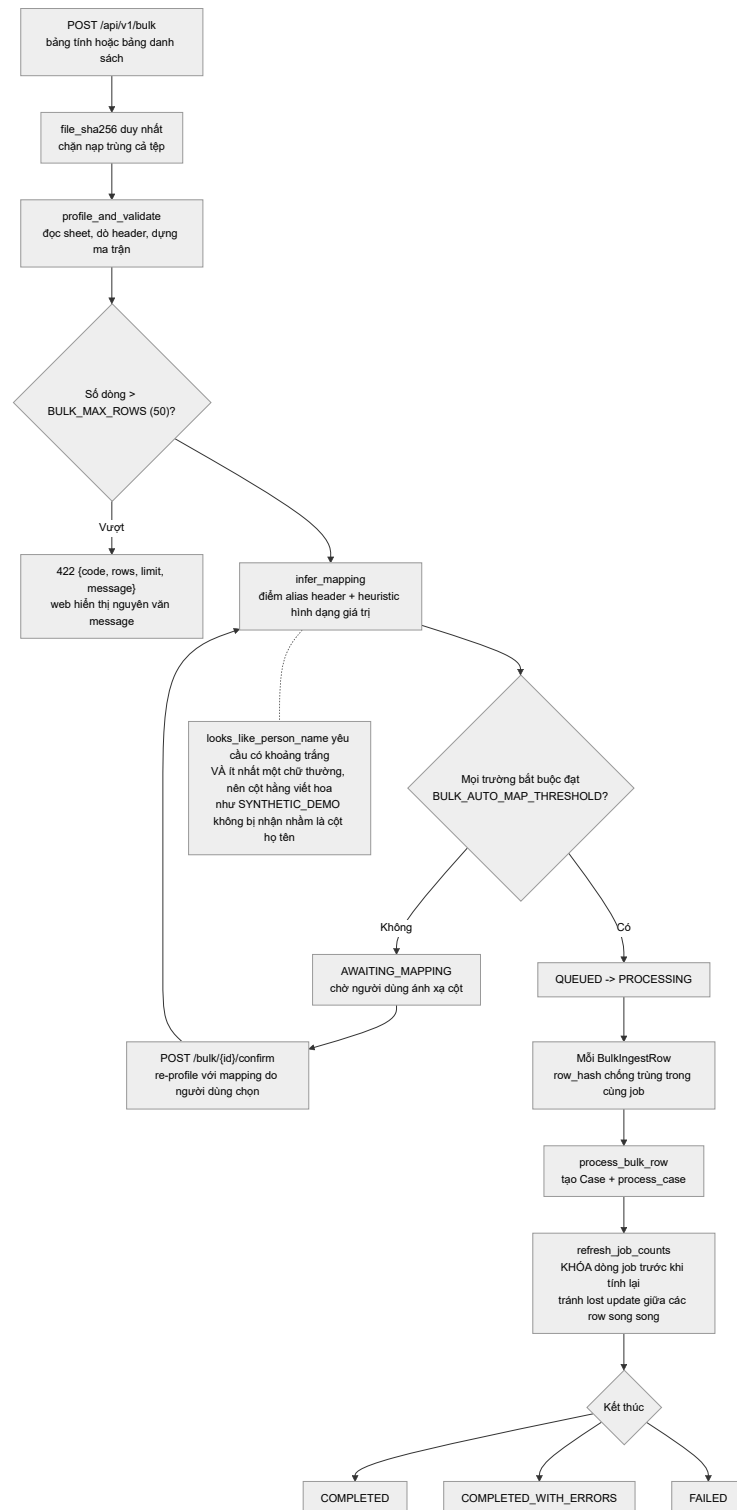
11.4.2 Bốn quyết định

Quyết định	Ngữ nghĩa
CONFIRM	Xác nhận đơn vị. Đơn vị đích phải <i>active</i> và APPROVED, và phải nằm trong phạm vi của người thẩm định. Kết quả thành MATCHED / HUMAN_REVIEW, độ tin cậy 1.0, <code>source_kind</code> lấy từ nguồn của đơn vị. Có thể xác nhận các dữ kiện phi-đơn-vị mà không phải nhập lại <code>unit_id</code> khi Case đã có đơn vị đáng tin.
UNKNOWN	Đây <i>riêng</i> là một kết luận về phân giải đơn vị: bằng chứng có thẩm quyền không đủ để xác định CURRENT_WORK_UNIT, nên đơn vị đã phân giải bị xoá, tổ chức về UNKNOWN, trạng thái phân giải về NOT_FOUND.
INSUFFICIENT	Thiếu dữ kiện <i>nghiệp vụ/chính sách</i> . Nó không được xoá một đơn vị đã xác minh: phân giải đơn vị và tính đủ của chính sách là hai trục khác nhau.
DISMISS	Đóng vận hành, không phải một kết luận nghiệp vụ dương. Fail-closed: Case chuyển FAILED để không bao giờ còn ở NEED_REVIEW mà không có việc nào mở.

11.4.3 Đồng thời

Khoá lạc quan bằng `expected_version` *cộng với* SELECT...FORUPDATE: hai người thẩm định cùng mở một Case thì người thứ hai nhận 409 kèm thông điệp yêu cầu tải lại trước khi quyết định. Mọi quyết định đều ghi AuditLog.

11.5 Nạp danh sách theo lô



Hình 11.3. Luồng nạp danh sách theo lô

11.5.1 Suy luận ánh xạ cột

`infer_mapping/_value_score` tự dò cột nào của bảng ứng với trường chuẩn nào (`subject_name`, `subject_code`, `unit_name`, ...) bằng cách kết hợp điểm fuzzy của alias tiêu đề với heuristic *hình dạng giá trị*.

Lý do thiết kế

Heuristic giá trị cho `subject_name` yêu cầu có từ cách nhau bởi khoảng trắng và ít nhất một chữ thường (`looks_like_person_name`). Lý do rất cụ thể: một cột hằng viết hoa toàn bộ — ví dụ cột `source_kind/Nguồn dữ liệu` mang giá trị `"SYNTHETIC_DEMO"` ở mọi dòng — không được bị nhận nhầm là cột họ tên. Một regex đếm từ ngẫu thơ coi dấu gạch dưới là ranh giới từ và sẽ chấm nó như một cái tên hai từ hợp lý, va chạm với cột tên thật và đẩy cả job vào `AWAITING_MAPPING`.

Giới hạn / cảnh báo

Nếu một job lô bất ngờ kẹt ở `AWAITING_MAPPING` với một tệp danh sách nhân sự hợp lệ, hãy kiểm tra `BulkIngestJob.mapping_json` xem có tiêu đề thứ hai bị ánh xạ vào cùng một trường đích không, trước khi kết luận tệp bị lỗi.

11.5.2 Hạn mức và tính nhất quán

Bất biến kiến trúc

Một lần import tạo tối đa `bulk_max_rows` Case (mặc định 50). Đây là *hạn mức khối lượng công việc*, không phải rào chắn của parser — `document_max_spreadsheet_rows` đã lo việc từ chối tệp đủ lớn để cạn bộ nhớ. Nó được cưỡng chế trong `profile_and_validate` *trước khi* ghi bất kỳ dòng nào, nên nó phủ luôn cả lần re-profile mà thao tác confirm thực hiện.

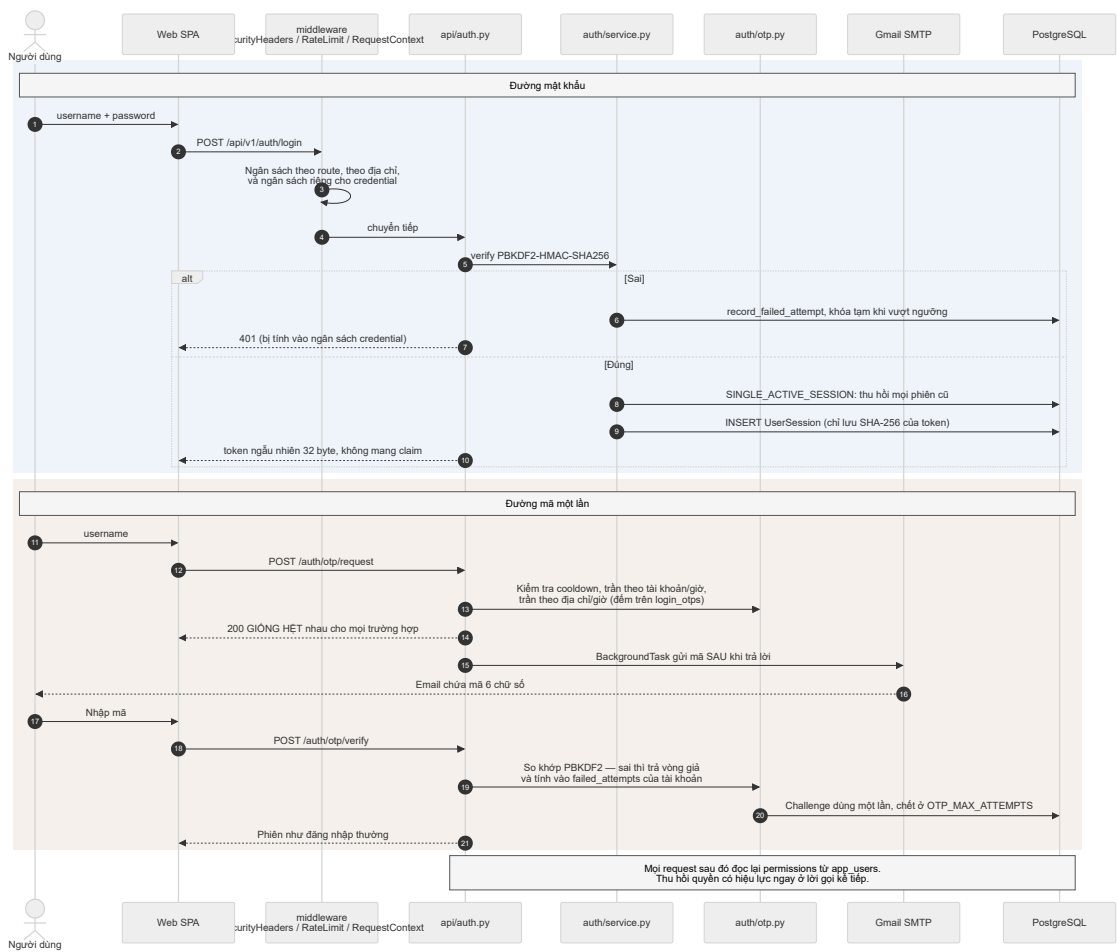
`GET/bulk/{id}` trả về `organization_type/resolution_status/workflow_status` đã quyết định của từng dòng dưới *đúng* các tên mà `GET/cases/{id}/subjects` dùng, nhờ đó web client render kết quả của một lần import và của một tài liệu nhiều chủ thể bằng cùng một danh sách.

Cảnh báo lô mức `WARNING` (ví dụ `AMBIGUOUS_UNIT`) vẫn tạo Case nhưng bắt buộc Case đó đi qua thẩm định thủ công.

CHƯƠNG 12

Kiến trúc bảo mật

12.1 Định danh và phiên



Hình 12.1. Đăng nhập bằng mật khẩu và bằng mã một lần

Tài khoản nằm trong chính CSDL của nền tảng. **Không có** nhà cung cấp định danh ngoài: realm Keycloak, việc xác thực JWT/JWKS và cơ chế phát token dev-login của frontend đều đã bị gỡ.

Thành phần

auth/permissions.py

Vai trò

Danh mục quyền — *thẩm quyền duy nhất* về việc quyền nào tồn tại. `normalize()` loại mã không có trong danh mục, nên một quyền đã nghỉ hưu không thể tiếp tục cấp quyền qua một hàng dữ liệu cũ.

Thành phần	Vai trò
<code>auth/service.py</code>	Mật khẩu PBKDF2-HMAC-SHA256 lưu dạng <code>pbkdf2_sha256\$iter\$salt\$hash</code> , nên có thể nâng số vòng lặp mà không vô hiệu các hàng hiện có; sinh tên đăng nhập; vòng đời phiên.
<code>auth/dependencies.py</code>	<code>current_principal</code> phân giải tài khoản người gọi từ CSDL ở mỗi request ; <code>require_perms(...)</code> đối chiếu với danh mục.

Bất biến kiến trúc

Token phiên không mang claim nào. Nó là 32 byte ngẫu nhiên và chỉ có SHA-256 của nó được lưu. Quyền được đọc lại ở mỗi request, nên thu hồi một quyền có hiệu lực ngay ở lời gọi kế tiếp của người gọi, và vô hiệu hoá một tài khoản làm chết các phiên đang sống của nó. Đây chính là lý do việc ẩn menu ở frontend thuần tuý là thẩm mỹ — đừng thêm bất kỳ kiểm tra nào mà client có thể tự thoả mãn, và đừng bắt đầu mã hoá quyền vào token “để tiết kiệm một truy vấn”.

`Principal.roles` là một phép chiếu *dẫn xuất* từ `permissions`, giữ lại để cột `role` của audit log và các phép đọc kiểu `"ADMIN" in p.roles` còn ý nghĩa. Nó không bao giờ là nguồn thẩm quyền thứ hai, và không có gì đọc ngược role từ client.

`enforce_password_policy=False` chỉ tồn tại cho seeder cài hai bộ thông tin đăng nhập demo đã ghi trong tài liệu — **không bao giờ** cho một mật khẩu do con người chọn đi qua cửa này.

12.1.1 Một phiên hoạt động

`SINGLE_ACTIVE_SESSION` (mặc định bật) khiến một lần đăng nhập thành công thu hồi mọi phiên tài khoản đó đang giữ, nên cùng một bộ thông tin không thể sống trên hai thiết bị. Cơ chế này hoạt động được *chính* vì phiên không mang claim và được đọc lại mỗi request: thiết bị kia ngừng hoạt động ngay ở lời gọi tiếp theo chứ không phải lúc hết hạn.

Giới hạn / cảnh báo

*Đây **không** phải SSO. Bản build này cố ý không có nhà cung cấp định danh ngoài, không OAuth 2.0 và không OIDC; “một thiết bị tại một thời điểm” được hiện thực bên trong tầng phiên nội bộ. Đưa một IdP trở lại là một quyết định kiến trúc, không phải một tác vụ gia cố — đừng thêm nó như tác dụng phụ của một thay đổi khác.*

12.1.2 Tính duy nhất của định danh tài khoản

`email` và `personal_code` là duy nhất toàn hệ thống, cưỡng chế ở hai nơi: `ensure_identity_available` sinh ra thông điệp mà quản trị viên đọc (route trả **409**, không phải 400), và `unique index` của migration 0008 là lớp chặn cuối mà một lần insert song song không vượt qua được. Cả hai được lưu đã chuẩn hoá qua `normalize_email / normalize_personal_code` — hãy ghi qua các hàm đó thay vì gán trực tiếp trường thô.

12.2 Mã đăng nhập một lần

POST/auth/otp/request gửi một mã sáu chữ số qua email; POST/auth/otp/verify đổi mã lấy một phiên thông thường; GET/auth/methods cho frontend biết có nên hiển thị tab này không. Các tính chất cần giữ, mỗi tính chất có một test tương ứng trong tests/test_login_otp.py:

- Mã **không bao giờ** xuất hiện trong thân phản hồi, trong tiêu đề thư, hay trong preheader của hộp thư. Nó đi bằng email và không đi đâu khác; chỉ băm PBKDF2 của nó được lưu.
- /auth/otp/request trả lời *giống hệt nhau* cho tài khoản thật, tên đăng nhập không tồn tại, tài khoản không có địa chỉ, tài khoản bị khoá và ngân sách đã cạn — và cuộc hội thoại SMTP chạy trong một BackgroundTask *sau* phản hồi nên thời gian đáp ứng cũng không phân biệt được.
- /auth/otp/verify dùng *một* thông điệp chung cho mã sai và cho tên đăng nhập không tồn tại, và trả một vòng PBKDF2 giả ở nhánh không tồn tại để hai bên tốn chi phí như nhau.
- Mã sai tính vào bộ đếm đăng nhập thất bại thông thường của tài khoản (record_failed_attempt), nên đoán mã khoá tài khoản y hệt đoán mật khẩu. Thiếu điều này, endpoint trở thành lối vòng qua cơ chế khoá tài khoản.
- Một challenge dùng một lần, chết ở OTP_MAX_ATTEMPTS lần đoán sai kể cả khi còn hạn, và việc phát mã mới tiêu huỷ mọi mã còn hiệu lực — nhiều mã cùng sống sẽ nhân xác suất trúng của người đoán lên theo số mã.
- Khối lượng thư bị chặn bởi các *đếm bên vũng trên hàng login_otps*, không phải bộ đếm trong tiến trình: cooldown theo tài khoản, trần theo tài khoản mỗi giờ, và trần theo địa chỉ mỗi giờ. Trần theo địa chỉ mới là thứ chặn kẻ rả mã theo danh sách tên đăng nhập. Vì vậy các hàng được giữ đến OTP_RETENTION_HOURS (dài hơn cửa sổ một giờ) — xoá lúc hết hạn sẽ trả lại ngân sách cho kẻ tấn công. Địa chỉ không phân giải được bị tính vào một bucket chung "unknown" thay vì bỏ qua kiểm tra.

Giới hạn / cảnh báo

Không thêm nhánh "không tìm thấy tài khoản", không hồi âm email đã che dấu, không thông báo gửi thất bại: mỗi thứ trong số đó biến endpoint này thành một oracle liệt kê tài khoản.

Settings.otp_login_available là OTP_LOGIN_ENABLED and mail_configured: một mã không thể giao được thì không bao giờ được phát, và cả hai endpoint trả 503 ở trạng thái đó. production_guards từ chối một cooldown ngắn hơn chính TTL của mã. Tab và đồng hồ đếm ngược ở frontend chỉ để hiển thị — máy chủ giữ hạn và ngân sách có thẩm quyền.

12.3 Vận chuyển và chống lạm dụng

shared/middleware.py giữ ba mối quan tâm, áp cho mọi request:

12.3.1 Header bảo mật

SecurityHeadersMiddleware được đăng ký *cuối cùng* nên nằm *ngoài cùng* — nhờ đó cả một 429 từ rate limiter lẫn một 500 từ exception handler đều mang đủ header. API chỉ trả JSON

và không trả gì khác, nên chính sách của nó là tuyệt đối:

- `Content-Security-Policy:default-src'none';...;frame-ancestors'none'`
- `X-Frame-Options:DENY` (giữ song song với CSP cho trình duyệt chỉ đọc header này)
- `X-Content-Type-Options:nosniff`, `Referrer-Policy`, `Permissions-Policy`
- `Cache-Control:no-store`
- HSTS *chỉ* khi production thực sự phục vụ HTTPS

`Settings.production_guards` từ chối khởi động production với `SECURITY_HEADERS_ENABLED=false`.

12.3.2 Giới hạn tần suất

Ba ngân sách theo địa chỉ client, *tất cả* đều phải thoả:

Ngân sách	Vì sao cần
<code>RATE_LIMIT_PER_MINUTE (120)</code>	Theo <i>hình dạng route</i> .
<code>RATE_LIMIT_IP_PER_MINUTE (600)</code>	Xuyên mọi route — không có nó, một đợt tấn công chỉ cần rải yêu cầu qua nhiều đường dẫn và luôn nằm dưới hạn mức của từng đường.
<code>RATE_LIMIT_AUTH_PER_MINUTE (10)</code>	Riêng cho các lần thử thông tin đăng nhập <i>bị từ chối</i> : kiểm tra trước lời gọi nhưng chỉ <i>tính phí</i> khi kết quả là 401/403, nên một cán bộ đăng nhập bình thường không bao giờ tiêu đến nó.

Cơ chế khoá tài khoản trong `auth/service.py` là một kiểm soát *khác*: nó bảo vệ một tài khoản, và không làm được gì trước kẻ rải mật khẩu theo danh sách tên đăng nhập. Hai cơ chế bổ sung cho nhau, không thay thế nhau.

12.3.3 Xác định địa chỉ client

Bất biến kiến trúc

`client_address()` chỉ tôn trọng `X-Forwarded-For` khi *peer của socket* khớp `TRUSTED_PROXY_IPS` (địa chỉ hoặc dải CIDR); ngược lại dùng địa chỉ socket. Tin header một cách vô điều kiện — như một phiên bản trước từng làm — cho phép bất kỳ ai tự reset bucket của mình ở mọi request chỉ bằng cách gửi một giá trị mới.

Hệ quả bất buộc: một triển khai đặt Nginx, load balancer hay WAF trước API **phải** liệt kê hop đó, nếu không toàn bộ người dùng gộp vào một bucket duy nhất. Các ngân sách OTP cũng lấy địa chỉ từ chính `middleware.client_address`, nên `TRUSTED_PROXY_IPS` chi phối luôn cả chúng.

Giới hạn / cảnh báo

Một WAF ngoài (F5 BIG-IP ASM hoặc tương đương) mới là nơi lọc chữ ký L7, nhận dạng bot và chống tấn công theo khối lượng; nó không nằm trong repository này và không thể cài từ đây. Thứ ứng dụng bảo đảm là nó hoạt động đúng khi đứng sau một thiết bị như vậy: địa chỉ client thật sống sót qua hop nhờ TRUSTED_PROXY_IPS, và các giới hạn nội tại vẫn còn hiệu lực nếu thiết bị bị đi vòng.

12.4 Thư đi

`modules/notifications/email.py` có đúng một transport: Gmail SMTP (`smtp.gmail.com:587`, STARTTLS) xác thực bằng *app password* của Google trên `GMAIL_USER / GMAIL_APP_PASSWORD`.

Giới hạn / cảnh báo

Các thiết lập SMTP tự quản (`SMTP_HOST/SMTP_PORT/SMTP_USERNAME/SMTP_PASSWORD/SMTP_USE_TLS`) và bộ ghi tệp `.eml` ngoại tuyến (`MAIL_OUTBOX_DIR`) đã bị gỡ: một host cấu hình được cho phép triển khai mở mật khẩu cấp phát tới một relay không xác thực, còn transport ghi tệp để lại mật khẩu trong một thư mục cục bộ. Đừng đưa lại; host và port là hằng số có chủ đích.

Gmail viết lại một `From` mà nó không sở hữu, nên địa chỉ người gửi luôn dẫn xuất từ `GMAIL_USER` và `MAIL_FROM_NAME` chỉ đặt tên hiển thị. `Settings.mail_configured` đúng khi cả hai nửa của thông tin đăng nhập có mặt, và `production_guards` từ chối khởi động production khi thiếu chúng. `_credential_response` vẫn giữ lại mật khẩu khi gửi thành công và chỉ trả về khi việc gửi thất bại.

Thư là MIME multipart: multipart/related bọc một multipart/alternative (text thuần + HTML) cộng favicon của ứng dụng dưới dạng image/png nội tuyến mà HTML tham chiếu bằng `cid:`. Markup trong `templates.py` cố tình dựa trên bảng với style nội tuyến: client thư loại bỏ khối `<style>` và Outlook render qua Word, nên flexbox/grid và stylesheet ngoài không sống sót. Logo ship bên trong package (`modules/notifications/assets/logo.png`, khai trong `package-data`) chú không đọc từ `apps/web` — image backend không có bản sao của cây web. Giữ bản text thuần tự đọc được: test lấy mật khẩu và mã OTP ra từ chính nó.

12.5 Mã hoá đầu ra và hình dạng phản hồi

Máy chủ không render HTML ở bất kỳ đâu: mọi route trả `application/json` và email tài khoản là `text/plain`. Việc escape vì thế thuộc về *điểm render*, và điểm đó là React — nó tự escape `<`, `>`, `&` và dấu nhảy trong text node.

Bất biến kiến trúc

Không escape HTML trên đường ghi vào CSDL hay vào phản hồi JSON: nó làm hỏng dữ liệu (một cái tên chứa dấu nhảy sẽ được lưu và về sau kết xuất thành `"`) và vẫn không cứu được một sink chủ động bỏ qua escape. Bảo đảm thật nằm ở chỗ *không có sink*

nào tồn tại: `tests/test_security_hardening.py` fail nếu `dangerouslySetInnerHTML`, `innerHTML`, `document.write` hoặc `eval` (xuất hiện ở bất cứ đâu dưới `apps/web/src`).

Mọi truy cập CSDL đi qua cấu trúc ORM của SQLAlchemy, vốn tham số hoá; `text()` duy nhất trong toàn bộ mã nguồn là `SELECT1` của `health probe` và hai lời gọi `advisory lock`, và các lời gọi đó *bind* khoá thay vì nội suy vào câu lệnh.

12.6 Kiểm toán và nhật ký

- `audit_logs` chỉ ghi-thêm, cường chế bằng trigger PostgreSQL (migration 0004). Chỉ INSERT.
- Log kỹ thuật là JSON có cấu trúc; `shared/logging.py` che dữ liệu *theo tên khoá* (`__REDACT_KEYS`). Không bao giờ log PII thô, nội dung tài liệu, bearer token hay bí mật.
- `AUTH_DISABLED` chỉ dành cho phát triển; `production_guards` từ chối khởi động production khi nó bật.

12.7 Rào chắn production

`Settings.production_guards` từ chối khởi động môi trường production khi:

1. `AUTH_DISABLED` bật;
2. `DATABASE_URL` hoặc `MINIO_SECRET_KEY` còn chứa thông tin mặc định/đã biết (`change_me`, `change_me_minio`, `admin`);
3. Quét mã độc không bật ở chế độ fail-closed;
4. Rate limiting tắt;
5. Header bảo mật tắt;
6. Thiếu `GOOGLE_USER/GOOGLE_APP_PASSWORD`;
7. Các tham số OTP nằm ngoài dải hợp lệ, hoặc cooldown ngắn hơn TTL.

CHƯƠNG 13

Chiến lược chất lượng và kiểm thử

13.1 Nguyên tắc kiểm thử

Bất biến kiến trúc

Mục tiêu chất lượng là **precision cộng với trù xuất đúng lúc**, không phải tối ưu accuracy tổng bằng cách ép nhần. Một thay đổi chạm vào bất biến nghiệp vụ phải bổ sung hoặc cập nhật kiểm thử hồi quy; hãy ưu tiên một test *fail* vì *đúng lý do đã nêu* hơn một test chỉ ghim lại đầu ra hiện tại.

Kiểm thử cần engine hoặc máy chủ thật (suy luận OCR, trigger audit) được *skip* hoặc *stub* một cách tường minh, chứ không âm thầm pass.

13.2 Golden suites

`apps/backend/tests/test_golden_suites.py` khoá tám suite nghiệp vụ cộng một suite tách nhiều chủ thể:

Suite	Nội dung được khoá
GOLDEN-CLEAN	Hồ sơ sạch: khớp chính xác mã và tên chuẩn.
GOLDEN-NOISY	Đầu vào nhiễu: sai chính tả, thiếu dấu, viết tắt.
GOLDEN-OCR	Đường OCR và gate chất lượng.
GOLDEN-CONTEXT	Nhiều tổ chức trong một tài liệu; xác định CURRENT_WORK_UNIT.
GOLDEN-CONFLICT	Mã và tên mâu thuẫn.
GOLDEN-UNKNOWN	Trù xuất: NOT_FOUND, AMBIGUOUS, UNKNOWN.
GOLDEN-POLICY	Hiệu lực theo ngày và INSUFFICIENT_DATA.
GOLDEN-E2E	Toàn tuyến từ đầu vào đến bằng chứng.
Multi-subject splitting	Fan-out, trù xuất ranh giới, đường OCR không hỗ trợ tách.

Lý do thiết kế

Golden suite theo quy ước *gọi trực tiếp*: nó gọi `extract()` / `process_case()` / `detect_subject_blocks()` thẳng chứ không đi qua route FastAPI hay task Celery, và dựng session SQLite in-memory bằng các helper `db_session()` / `seed_unit()` / `seed_policy()` của chính tệp đó. Quy ước này giữ cho suite chạy nhanh và chỉ fail vì logic nghiệp vụ, chứ không vì hạ tầng. Hãy theo đúng mẫu này cho test golden mới thay vì dựng API/worker.

13.3 Cấu hình kiểm thử backend

`tests/conftest.py` trở `DATABASE_URL` vào một CSDL SQLite **có tệp** trong thư mục tạm theo từng lần chạy, và tắt `auth`, `embedding`, `antimalware`, `rate limiting` — nên `pytest` không cần chuẩn bị gì.

Lý do thiết kế

CSDL là *file-backed* chứ không phải `:memory:` vì API nay đọc tài khoản và phiên qua `SessionLocal` ở phạm vi tiến trình, còn một SQLite in-memory không được chia sẻ giữa các kết nối mà một request của `TestClient` đi qua. Phần lớn test vẫn tự dựng engine riêng và không bao giờ chạm vào nó; những test cần lược đồ ứng dụng thì phụ thuộc fixture `app_database`.

Bảng 13.2. Kết quả bộ test theo backend CSDL

Backend	Kết quả	Ghi chú
SQLite (mặc định)	376 passed, 5 skipped	4 skip là test trigger append-only; skip thứ năm cần khoá hàng thật (<code>test_bulk_job_finalisation.py</code> , đưa lost-update).
PostgreSQL thật	380 passed, 1 skipped	Trở <code>DATABASE_URL</code> vào PostgreSQL để chạy thêm cả các test trigger.

`tests/test_ocr_pipeline_and_structure.py` giả lập cả hai engine OCR. Lưu ý `get_settings` được `lru_cache`, nên một lần ghi đè biến môi trường cần fixture `setting_s_env` của chính tệp đó (fixture này xoá cache) — riêng `monkeypatch.setenv` không thay đổi được gì.

`tests/test_local_auth.py` phủ tầng tài khoản/phiên/quyền, một nửa ở mức dịch vụ và một nửa trên bề mặt HTTP thật với phiên thật.

Bất biến kiến trúc

Khi thay đổi bất kỳ thứ gì trong `modules/auth/`, hãy bổ sung hồi quy ở đó thay vì chỉ `assert` trên dataclass `Principal`: các tính chất đáng bảo vệ — một quyền bị thu hồi có hiệu lực giữa phiên, phiên của tài khoản bị vô hiệu hoá chết theo, tài khoản quản trị không sửa được — chỉ lộ ra khi chạy đầu-cuối.

Test không bao giờ mở socket: `tests/mail_stub.py` thay `smtpplib.SMTP` qua một fixture `autouse` trong `conftest.py` và ghi lại chính `EmailMessage` lẽ ra đã gửi đi — đó là thứ mà `_issued_password` và `_sent_code` đọc.

13.4 Kiểm thử frontend và E2E

- `npmruntypecheck`, `npmrunbuild` (build chạy typecheck trước).

- E2E: `npxplaywrighttest` chạy `apps/web/e2e/*.spec.ts` với một backend thật đang chạy cộng `npmrundev`.

Giới hạn / cảnh báo

Không còn đường tắt dev-login. Mọi spec đăng nhập qua form thật bằng `signInAs()` trong `e2e/helpers.ts`, dùng tài khoản `user/admin` đã seed. `signInAs(page, 'REVIEWER')` tự cấp một tài khoản `e2e_reviewer` qua API quản trị thật trước, vì thẩm định là một bộ quyền do quản trị viên cấp chứ không phải một role bật sẵn. Các spec này ghi vào CSDL local thật (tài khoản, Case, review), điều được chấp nhận với một POC nhưng có nghĩa CSDL sẽ tích tụ dữ liệu test. `playwright.config.ts` ghim `workers: 1`: một tiến trình `uvicorn` chạy ở chế độ `--reload` không phục vụ nổi các lời gọi `POST/cases/text` đồng thời đủ nhanh cho mức song song mặc định, và spec sẽ timeout (chứ không phải fail assertion) ở mức đồng thời cao hơn.

Playwright là devDependency (@playwright/test); chạy `npxplaywrightinstallchromium` một lần trên mỗi máy để tải binary trình duyệt.

13.5 Pipeline CI

Job	Nội dung
backend-quality	pytest + golden suites có đo coverage, Ruff, MyPy, Bandit, pip-audit (<code>--strict</code>).
frontend-react	Cài phụ thuộc, typecheck TypeScript, build production, kiểm tra JSON của artifact calibration.
migration-drill	upgrade → seed → downgrade → re-upgrade trên PostgreSQL thật.
Docker build + Trivy	Build image và quét lỗ hổng.
release.yml	SBOM + SHA256 + chứng thực provenance của GitHub.

Các target trong Makefile phản chiếu đúng những gì CI chạy: `maketest`, `makegolden`, `makelinttypesecurity`, `makemigration-drill`.

13.6 Cổng phát hành

Theo `docs/quality/RELEASE_GATES.md`, một bản phát hành bị **chặn** nếu bất kỳ điều nào sau đây xảy ra:

1. Một trong 8 golden suite fail ở mức critical.
2. Hồi quy chính sách fail.
3. Hồi quy hợp đồng audit/evidence.
4. Kiểm tra ảnh chụp danh mục fail.
5. Hồi quy hành vi false-positive / conflict / unknown.

6. Drill migration upgrade/downgrade fail.
7. Chưa có biên bản drill backup/restore gần thời điểm phát hành.
8. Phát hiện bảo mật critical chưa được xử lý hoặc chấp nhận chính thức.
9. Quét container/phụ thuộc fail theo chính sách CI.
10. Readiness của phụ thuộc fail ở staging.

Bất biến kiến trúc

Không mô tả một bản build là "sẵn sàng production" trong khi còn tồn tại lỗi critical về bảo mật, audit, migration, rollback hoặc golden gate.

CHƯƠNG 14

Vận hành và giám sát

14.1 Lệnh vận hành

14.1.1 Profile local

Lệnh	Tác dụng
<code>condarun-nbqppythonops/local.py start</code>	PostgreSQL + migrate + seed + API + web.
<code>condarun-nbqppythonops/local.py status</code>	Trạng thái, gồm cả kiểm tra <code>/health/ready</code> .
<code>condarun-nbqppythonops/local.py stop</code>	Dừng tiến trình ứng dụng, giữ PostgreSQL.
<code>ops/local.py backup / restore-drill</code>	Backup và diễn tập phục hồi.

Giao diện: `http://127.0.0.1:3000`. OpenAPI: `http://127.0.0.1:8000/docs`. Tài khoản bàn giao: `admin/admin` và `user/user`.

14.1.2 Target Makefile

Target	Tác dụng
<code>makelocal</code>	<code>ops/local.py start</code> .
<code>makeup / down / logs</code>	Vòng đời docker compose.
<code>makemigrate / seed</code>	<code>alembic upgrade head</code> ; nạp ảnh chụp danh mục và siêu dữ liệu chính sách.
<code>maketest / golden</code>	Bộ test backend; riêng golden suite.
<code>makelinttypesecurity</code>	Ruff, MyPy, Bandit.
<code>makemigration-drill</code>	<code>upgrade / seed / downgrade / re-upgrade</code> .
<code>makeapi / web</code>	<code>uvicorn</code> có reload; <code>npm install && vitedev</code> .

14.2 Số liệu Prometheus

`/metrics` được bật theo `METRICS_ENABLED`. Các counter được định nghĩa trong `shared/metrics.py`:

Counter	Nhãn
<code>cabqp_case_transitions_total</code>	<code>status</code>
<code>cabqp_resolution_results_total</code>	<code>status, method, organization_type</code>
<code>cabqp_review_events_total</code>	<code>event, reason</code>
<code>cabqp_document_parse_total</code>	<code>status, method</code>
<code>cabqp_policy_assessments_total</code>	<code>status, policy_type</code>
<code>cabqp_outbox_events_total</code>	<code>status, event_type</code>
<code>cabqp_input_routing_total</code>	<code>kind, decision</code>
<code>cabqp_ocr_quality_total</code>	<code>engine, gate_result</code>
<code>cabqp_bulk_ingest_total</code>	<code>status</code>

Các counter này được thiết kế để trả lời những câu hỏi vận hành cụ thể:

- *Tỉ lệ trừu xuất có đang tăng bất thường không?* — so `cabqp_resolution_results_total` theo `status` và `cabqp_case_transitions_total{status="NEED_REVIEW"}`.
- *Một lần publish danh mục mới có làm xấu chất lượng phân giải không?* — theo dõi phân bố `method` trước và sau thời điểm publish.
- *OCR fallback có đang chạy quá nhiều không?* — `cabqp_ocr_quality_total` theo `engine`.
- *Có job nào bị kẹt không?* — `cabqp_outbox_events_total` theo `status` và `cabqp_bulk_ingest_total`.

Cảnh báo mẫu nằm ở `ops/monitoring/alerts.yml`; cấu hình scrape ở `ops/prometheus/prometheus.yml`.

14.3 Khả năng mở rộng, độ trễ và độ tin cậy

Profile Docker tách API khỏi ba nhóm hàng đợi `documents`, `outbox` và `batch`; mỗi nhóm có thể tăng số Celery worker độc lập theo tải. Tập gốc nằm ở MinIO, trạng thái nghiệp vụ ở PostgreSQL và Redis làm broker/rate limit, nên API không giữ trạng thái xử lý dài hạn trong bộ nhớ tiến trình. Đây là *đường mở rộng ngang của kiến trúc*, không phải bằng chứng capacity đã đo.

Hai mốc độ trễ hiện có không được trộn lẫn:

- cấu hình Base đạt trung bình 3,74 giây/trang trên 200 trang OmniDocBench;
- phép tải nhẹ API ngày 16/09/2026 đạt p95 265 ms, 135/135 checks và 0% lỗi với 3 người dùng ảo trong 20 giây, nhưng không bao gồm upload/OCR nặng.

Pipeline bảo vệ độ tin cậy bằng idempotency key, checksum, durable outbox, upsert kết quả theo Case, retry có backoff, trạng thái `FAILED` cuối cùng, DLQ và circuit breaker. Drill

backup/restore ngày 16/09 đã đối chiếu chính xác 5 bảng với số dòng 210/210/191/48/495. Chưa có soak test, saturation, backlog, throughput OCR, failover dài hạn hoặc SLA/RPO/RTO được ký; vì vậy không diễn giải các mốc trên thành cam kết sẵn sàng production.

14.4 Nhật ký

Log kỹ thuật là JSON có cấu trúc với trường `event` lồng bên trong. Việc che dữ liệu diễn ra theo tên khoá qua `_REDACT_KEYS`. Các sự kiện quan trọng khi chẩn đoán:

Sự kiện	Ý nghĩa
<code>local_task_runner_started</code>	Profile local đã bật sweeper outbox.
<code>outbox_immediate_dispatch_failed</code>	Bản giao ngay thất bại; outbox bền vững giữ lại yêu cầu và Beat sẽ thử lại khi broker hồi phục.
<code>inline_document_task_terminal_failure</code>	Task đã tiêu hết retry của chính nó và chạy nhánh terminal.
<code>document_processing_failed (audit)</code>	Case chuyển FAILED.
<code>dead_letter_publish_failed</code>	Không đẩy được message vào <code>document_dlq</code> .
<code>semantic_resolver_unavailable</code>	Không nạp được mô hình embedding; xếp hạng lại chạy thiếu tầng semantic.
<code>case_upload_persist_failed</code>	Transaction ghi Case/Document/Outbox bị rollback.
<code>unhandled_request_exception</code>	Handler toàn cục; phản hồi 500 dạng <code>INTERNAL_ERROR</code> .

14.5 Runbook

Runbook	Phạm vi
<code>ops/runbooks/BACKUP_RESTORE</code>	Sạc lưu và phục hồi PostgreSQL; script ở <code>ops/scripts/backup_postgres.sh</code> và <code>restore_postgres.sh</code> .
<code>ops/runbooks/DEPLOY_ROLLBACK</code>	Triển khai và quay lui, gồm drill migration (<code>ops/scripts/migration_rollback_drill.sh</code>).
<code>ops/runbooks/REGISTRY_POLICY</code>	Cập nhật danh mục và quy tắc chính sách.
<code>ops/runbooks/INCIDENT_RESPONSE</code>	Ứng phó sự cố.
<code>ops/scripts/validate_release</code>	Kiểm tra công phát hành.
<code>ops/loadtest/verify_smoke</code>	Kiểm thử tải dạng smoke.

14.6 Chẩn đoán các trục trặc đã gặp

Triệu chứng	Nguyên nhân đã xác định
Upload đứng mãi ở RECEIVE, không lỗi	Worker khởi động thiếu <code>-Qcelery</code> , <code>outbox</code> , <code>documents</code> , <code>batch</code> : nó chỉ nhận hàng đợi mặc định và âm thầm bỏ mọi thứ được định tuyến đi nơi khác. Hoặc <code>include=["cabqp.workers.tasks"]</code> bị bỏ khỏi <code>celery_app.py</code> .
Job lô kẹt ở PROCESSING	<code>refresh_job_counts</code> không khoá hàng job (lost update), và <code>profile local</code> không có Beat chạy <code>reconcile_bulk_jobs</code> để kết thúc hộ.
Job lô kẹt ở AWAITING_MAPPING với tệp hợp lệ	Hai tiêu đề bị ánh xạ vào cùng một trường đích; kiểm tra <code>mapping.json</code> .
Bulk confirm trả 500 ở profile local	Cắt chuỗi tiền tố <code>scheme</code> thay vì dùng <code>storage.key_from_uri()</code> .
Mọi thư OTP/mật khẩu hỏng với <code>SMTPAuthenticationError</code>	App password mang theo dấu ngoặc từ <code>.env</code> ; <code>read_env_file()</code> bóc một lớp ngoặc bao quanh.
<code>Import transformers raise RuntimeError</code>	Thiếu <code>USE_TF=0</code> <code>USE_TORCH=1</code> trong môi trường tiến trình (bản TensorFlow cũ).
Sửa mã "không có tác dụng" qua HTTP	<code>pipshowcabqp</code> trở nhằm checkout, hoặc <code>--reload</code> không bắt được thay đổi — khởi động lại tiến trình hẳn.
E2E timeout thay vì fail assertion	Chạy Playwright với nhiều worker; <code>workers:1</code> là bắt buộc.

Giới hạn / cảnh báo

Trên Windows, bản mingw của `curl (/mingw64/bin/curl` trong `Git Bash`) có thể làm hỏng ký tự không-ASCII truyền qua `-d'...'` trên dòng lệnh, ngay cả khi byte thô nhìn qua pipe vẫn đúng. Một `unit_name` tiếng Việt gửi kiểu này có thể âm thầm phân giải như thể nó là một chuỗi khác gần-ASCII. Hãy dùng một script Python gắn với `httpx/requests (json=gửi UTF-8 đúng)` thay cho `curl-d` với mọi phép thử API tay có tiếng Việt.

CHƯƠNG 15

Rủi ro, nợ kỹ thuật và ranh giới

15.1 Ranh giới production

Repository cung cấp các kiểm soát hướng production nhưng **không** tự biến dữ liệu demo thành dữ liệu nghiệp vụ thật. Nguồn gốc và `source_kind` phải được giữ đúng; kết quả tổng hợp phải mang `SYNTHETIC_DEMO`. Trình quản lý bí mật, WAF và TLS termination nên do hạ tầng triển khai cung cấp (Kubernetes/Ingress/dịch vụ secret của cloud hoặc tương đương), không hard-code vào repo.

15.2 Những gì chưa được kiểm chứng

Hạng mục	Trạng thái
Profile Docker	Chưa kiểm chứng trên máy phát triển hiện tại vì Docker chưa được cài. <code>compose.yaml</code> parse được nhưng chưa build hay chạy kể từ khi gỡ Keycloak.
Artifact calibration	Fit từ nhiều mention tổng hợp xác định của danh mục trong repo. Phù hợp cổng chất lượng mức dự án/demo; phải fit lại trên dữ liệu kiểm chứng có nhãn thật.
Ngưỡng OCR	Đã có baseline Base/Lite, nhưng tập thật chỉ 7 tệp; phải mở rộng corpus và chốt theo p50/p95, raw artifact.
Danh mục nhân sự	Chưa có script seed gắn vào <code>makeseed</code> ; <code>scripts/seed_persons.py</code> phải chạy tường minh.
Hàng đợi <code>document_dlq</code>	Không có consumer mặc định: message ở lại chờ người vận hành kiểm tra và phát lại thủ công.
<code>AppErrorBoundary.jsx</code>	Tồn tại nhưng <i>chưa</i> được nối vào <code>main.tsx/App.tsx</code> ; đừng giả định lỗi render đang được bắt.

15.3 Nợ kỹ thuật và giới hạn đã biết

- Tách nhiều chủ thể trên tài liệu OCR:** chưa hỗ trợ ở v1. Dò ranh giới theo bbox trên thứ tự dòng OCR không đủ tin cậy, nên cả tài liệu trừu xuất. Đây là một quyết định *có chủ đích* (fail-closed), nhưng vẫn là một khoảng trống chức năng.
- Danh sách nhân sự dạng mỗi trường một dòng:** không có cấu trúc hàng/cột dựng lại được nên hiện không được thành phần nào tách; nó ở lại như một Case đơn, nhiều khả năng `NEED_REVIEW`.

3. **CABQVerification.jsx** khoảng 2.9k dòng: là giao diện thật đang phục vụ, nhưng kích thước đó khiến việc kiểm tra bất biến hiển thị (Mục ngữ nghĩa trạng thái ở Chương 4) phụ thuộc nhiều vào kỷ luật của người sửa. Trước khi tách nhỏ, cần có test cho `historyStatusCategory` và các hàm dẫn xuất của nó.
4. **Lexicon hành chính trong gate chất lượng** (`COMMON_VI`) cố tình nhỏ và không giả vờ là một từ điển đầy đủ; gate từ điển mặc định *tắt* (`QUALITY_DICTIONARY_GATE_ENABLED=false`).
5. **Spec E2E ghi vào CSDL local thật**: chấp nhận được với POC nhưng làm CSDL tích tụ dữ liệu test.
6. **POST/lookup/unit** là một lời gọi resolver đầy đủ, không phải endpoint autocomplete: trải nghiệm gợi ý sẽ không bao giờ dưới một giây nếu không tách một đường index riêng.
7. **workers:1** trong **Playwright** là cách đi vòng quanh giới hạn của unicorn chế độ reload, không phải một đặc tính về năng lực hệ thống.

15.4 Rủi ro kiến trúc

Rủi ro	Tác động	Giảm thiểu hiện có
Calibration chưa fit trên dữ liệu thật	Ngưỡng tự động chấp nhận có thể lệch, sinh false accept hoặc trừu xuất thừa	Mặc định yêu cầu calibration cho auto-accept mờ; semantic auto-accept mặc định tắt; golden suite khoá hành vi trừu xuất
Danh mục thiếu độ phủ	Tỉ lệ <code>NOT_FOUND</code> cao, dồn tải lên hàng đợi thẩm định	Pipeline ingestion + QA; số liệu <code>resolution_results</code> theo status
Giao diện vi phạm bất biến hiển thị	Hiển thị sai bản chất kết luận	Hợp đồng hiển thị được ghi rõ; hàm dẫn xuất trạng thái dùng chung; test chống sink XSS
Cấu hình worker sai ở docker	Việc bị bỏ âm thầm	Tài liệu hoá rõ; <code>/health/ready</code> kiểm tra Redis khi profile dùng Celery
Proxy không khai trong <code>T RUSTED_PROXY_IPS</code>	Mọi người dùng chung một bucket rate limit	Rào chắn cấu hình; tài liệu hoá ở chương triển khai và bảo mật
Đưa IdP ngoài trở lại như "tác vụ gia cố"	Phá vỡ tính chất token-không-claim	Ghi rõ đây là quyết định kiến trúc, không phải gia cố
Docker chưa kiểm chứng	Rủi ro triển khai lần đầu	Profile local đầy đủ chức năng; drill migration trong CI trên PostgreSQL thật

15.5 Hướng phát triển

1. Fit lại calibration trên dữ liệu kiểm chứng có nhãn từ môi trường thật, đồng thời tăng `model_version` và `threshold_version`.
2. Mở rộng benchmark Base trên tập quét thật, lưu raw artifact/model revision/VRAM và chốt ngưỡng theo từng nhóm định dạng.
3. Bổ sung khả năng tách nhiều chủ thể cho tài liệu OCR, kèm chứng minh rằng bộ dò ranh giới không kích hoạt sai trên bản quét một người.
4. Đưa seed danh mục nhân sự vào quy trình chính thức (hoặc ghi rõ nó là thao tác thủ công vĩnh viễn).
5. Chạy và kiểm chứng profile Docker đầu-cuối trên một máy có Docker.
6. Tách `CABQPVerification.jsx` sau khi đã có test bao phủ các hàm dẫn xuất trạng thái.
7. Cân nhắc một endpoint gợi ý đơn vị nhẹ, tách khỏi `Resolver` đầy đủ, cho trải nghiệm nhập liệu.

PHỤ LỤC A

Phụ lục A — Danh mục API

Mọi route nằm dưới tiền tố `/api/v1` (biến `API_PREFIX`), trừ nhóm `health`. Cột **Quyền** ghi yêu cầu do `require_perms(...)` khai báo; *principal* nghĩa là route chỉ cần người gọi đã đăng nhập rồi tự giới hạn phạm vi dữ liệu.

A.1 Xác thực

Route	Quyền	Ghi chú
POST/auth/login	—	Trả token phiên opaque.
GET/auth/methods	—	Cho biết có hiển thị tab OTP không.
POST/auth/otp/request	—	Luôn trả 200 giống hệt nhau.
POST/auth/otp/verify	—	Nằm trong <code>_AUTH_PATHS</code> của middleware.
POST/auth/logout	principal	
GET/auth/me	principal	
GET/auth/permissions	principal	Trả danh mục quyền hiệu lực.
POST/auth/change-password	principal	

A.2 Hồ sơ và tra cứu

Route	Quyền	Ghi chú
POST/cases/text	CASE_CREATE	Hỗ trợ Idempotency-Key, as_of_date, fan-out nhiều chủ thể.
POST/cases/file	CASE_CREATE	202; từ chối TABULAR_LIST với 422.
GET/cases	principal	Trả items và totals đếm trên toàn hàng đợi trong phạm vi người gọi.
GET/cases/{id}	principal	Trả case, extracted, result, eligibility, documents.
GET/cases/{id}/subjects	principal	Các Case anh em của một tài liệu nhiều chủ thể.

Route	Quyền	Ghi chú
POST/lookup/unit	LOOKUP_UNIT	Lời gọi <code>Resolver.resolve()</code> đầy đủ.

A.3 Nạp theo lô và thẩm định

Route	Quyền	Ghi chú
POST/bulk	CASE_BULK	202; lập hồ sơ và suy luận ánh xạ cột.
POST/bulk/{id}/confirm	CASE_BULK	202; re-profile với ánh xạ người dùng chọn.
GET/bulk/{id}	principal	Trạng thái job và từng dòng, dùng chung tên trường với <code>/cases/{id}/subjects</code> .
GET/reviews	REVIEW_QUEUE	Phân trang <code>page/page_size</code> , sắp xếp cũ nhất trước.
POST/reviews/{id}/decision	REVIEW_DECIDE	Cần <code>expected_version</code> .
POST/reviews/{id}/assign	REVIEW_DECIDE	Gán lại bởi quản trị viên bị từ chối độc lập ở máy chủ.

A.4 Quản trị danh mục đơn vị

Tiền tố `/admin/registry`; toàn bộ yêu cầu `REGISTRY_ADMIN`.

Route	Ghi chú
GET/units, GET/units/{id}	Tra cứu đơn vị.
POST/units, PATCH/units/{id}	Chỉ trên version DRAFT.
POST/units/{id}/qa	Quyết định QA cho đơn vị.
POST/units/{id}/aliases, POST/aliases/{id}/qa	Bí danh và QA của bí danh.
POST/units/{id}/codes, POST/codes/{id}/qa	Mã và QA của mã.
GET/candidates, POST/candidates/{id}/decision	Ứng viên từ crawler/reviewer.
GET/versions, POST/versions	Danh sách và tạo version.
POST/versions/{v}/validate	Chỉ từ DRAFT.
POST/versions/{v}/approve	Chỉ từ VALIDATED.
POST/versions/publish	Chỉ từ APPROVED; advisory lock.

Route	Ghi chú
GET/versions/diff	So sánh hai version.
POST/versions/rollback	Clone ảnh chụp cũ thành version mới.

A.5 Quản trị danh mục nhân sự

Tiền tố `/admin/person-registry`, quyền `PERSON_REGISTRY_ADMIN`. Hình dạng endpoint phản chiếu *một-đối-một* danh mục đơn vị, với `persons` thay cho `units` — kể cả `/versions/publish`, `/versions/diff` và `/versions/rollback`. Lock publish và bất biến "đúng một version `PUBLISHED`" là *riêng biệt*.

A.6 Quản trị tài khoản và kiểm toán

Route	Quyền	Ghi chú
GET/admin/users	USER_ADMIN	
POST/admin/users	USER_ADMIN	201; is_admin=False hard-code.
POST/admin/users/suggest-username	USER_ADMIN	Sinh tên đăng nhập gợi ý.
GET/admin/users/{u}, PATCH/admin/users/{u}	USER_ADMIN	<code>_guard_admin_target</code> từ chối sửa tài khoản quản trị.
POST/admin/users/{u}/reset-password	USER_ADMIN	Mật khẩu chỉ trả về khi gửi mail thất bại.
POST/admin/users/{u}/unlock	USER_ADMIN	
POST/admin/users/{u}/revoked-sessions	USER_ADMIN	
DELETE/admin/users/{u}	USER_ADMIN	
GET/admin/audit	AUDIT_VIEW	

A.7 Sức khoẻ và số liệu

Route	Ghi chú
GET/health, GET/health/live	Liveness (ngoài <code>API_PREFIX</code>).
GET/health/ready	Readiness theo profile; 503 kèm chi tiết khi có mục không ok.

Route	Ghi chú
GET/metrics	Định dạng Prometheus, bật theo METRICS_ENABLED, không nằm trong OpenAPI schema.

A.8 Header được chấp nhận

CORS cho phép Authorization, Content-Type, Idempotency-Key, X-Request-ID với các phương thức GET, POST, PATCH, PUT, DELETE, OPTIONS. Origin lấy từ CORS_ORIGINS.

PHỤ LỤC B

Phụ lục B — Cấu hình

Mọi thiết lập được khai báo trong `shared/settings.py` (pydantic-settings, đọc `.env`) và có thể ghi đè bằng biến môi trường viết hoa cùng tên.

B.1 Profile và hạ tầng

Thiết lập	Mặc định	Ý nghĩa
APP_ENV	development	prod/production bật rào chắn.
API_PREFIX	/api/v1	
RUNTIME_PROFILE	local	local docker.
STORAGE_BACKEND	auto	auto filesystem minio.
QUEUE_BACKEND	auto	auto inline celery.
STORAGE_ROOT	.local/documents	Gốc lưu tệp ở profile local.
DATABASE_URL	PostgreSQL	
REDIS_URL	redis://localhost:6379/0	Chỉ dùng ở profile docker.
CORS_ORIGINS	http://localhost:3000	
METRICS_ENABLED	true	

B.2 Xác thực, phiên và OTP

Thiết lập	Mặc định	Ý nghĩa
AUTH_DISABLED	false	Chỉ dành cho phát triển; production từ chối khởi động.
SESSION_HOURS	12	
SINGLE_ACTIVE_SESSION	true	Một phiên hoạt động cho mỗi tài khoản.
OTP_LOGIN_ENABLED	true	Tự tắt nếu mail chưa cấu hình.
OTP_TTL_SECONDS	60	Hợp lệ trong 20–900.
OTP_CODE_LENGTH	6	Hợp lệ 6–10.

Thiết lập	Mặc định	Ý nghĩa
OTP_MAX_ATTEMPTS	5	Hợp lệ 1–10.
OTP_RESEND_COOLDOWN_SECONDS	60	Phải $\geq$ OTP_TTL_SECONDS.
OTP_MAX_PER_ACCOUNT_PER_HOUR	5	
OTP_MAX_PER_IP_PER_HOUR	20	Chặn kẻ rả theo danh sách tên đăng nhập.
OTP_RETENTION_HOURS	24	Dài hơn cửa sổ một giờ, nếu không ngân sách bị trả lại.

B.3 Vận chuyển và chống lạm dụng

Thiết lập	Mặc định	Ý nghĩa
RATE_LIMIT_ENABLED	true	Production bắt buộc bật.
RATE_LIMIT_PER_MINUTE	120	Theo hình dạng route.
RATE_LIMIT_IP_PER_MINUTE	600	Xuyên mọi route.
RATE_LIMIT_AUTH_PER_MINUTE	10	Chỉ tính khi bị từ chối.
RATE_LIMIT_FAIL_OPEN	true	
TRUSTED_PROXY_IPS	rỗng	Bắt buộc khai khi có proxy phía trước.
SECURITY_HEADERS_ENABLED	true	Production bắt buộc bật.
FRAME_ANCESTORS	'none'	
HSTS_MAX_AGE_SECONDS	31536000	Chỉ phát khi production phục vụ HTTPS.

B.4 Ngưỡng phân giải

Thiết lập	Mặc định	Ý nghĩa
RESOLVER_FUZZY_THRESHOLD	91.0	Ngưỡng điểm fuzzy để auto-accept.
RESOLVER_MARGIN_THRESHOLD	6.0	Khoảng cách top1–top2.
RESOLVER_CANDIDATE_LIMIT	10	Số ứng viên trả về.
RESOLVER_CALIBRATED_ACCEPT_THRESHOLD	0.90	Xác suất calibration tối thiểu.

Thiết lập	Mặc định	Ý nghĩa
RESOLVER_REQUIRE_CALIBRATION_FOR_FUZZY_AUTOACCEPT	true	
RESOLVER_SEMANTIC_THRESHOLD	0.82	
RESOLVER_SEMANTIC_MARGIN_THRESHOLD	0.08	
RESOLVER_ALLOW_SEMANTIC_AUTOACCEPT	false	Mặc định tắt.
RESOLVER_CACHE_TTL_SECONDS	60	
RESOLVER_CALIBRATION_PATH	artifacts/resolver_calibration.json	
EMBEDDING_MODEL_NAME	MiniLM đa ngữ	
EMBEDDING_ENABLED	true	
EMBEDDING_LOCAL_FILES_ONLY	true	Giữ runtime ngoại tuyến.

B.5 Ngưỡng tài liệu và OCR

Thiết lập	Mặc định	Ý nghĩa
OCR_MIN_CONFIDENCE	0.70	Ngưỡng cho đường OCR.
DOCUMENT_PARSE_MIN_CONFIDENCE	0.60	Ngưỡng cho đường parse cấu trúc.
EXTRACTION_MIN_CONFIDENCE	0.75	Ngưỡng độ đầy đủ trích xuất.
DOCUMENT_FAILED_GATE_CONFIDENCE_CAP	0.50	Trần độ tin cậy khi gate hỏng.
OCR_FALLBACK_ENABLED	true	
OCR_FALLBACK_CONFIDENCE_MIN	0.70	Dưới mức này thì chạy VietOCR.
OCR_DETECTOR / OCR_RECOGNIZER	easyocr	Engine chính.
OCR_FALLBACK_DETECTOR / _RECOGNIZER	easyocr / vietocr	
VIETOCR_ARCHITECTURE	vgg_transformer	
VIETOCR_DEVICE	cpu	

Thiết lập	Mặc định	Ý nghĩa
QUALITY_CONFIG_PATH	config/document_quality.yaml	Nguồn ngưỡng có version.
QUALITY_IMAGE_BLUR_VARIANCE_MIN	50.0	Chất lượng ảnh nguồn.
QUALITY_IMAGE_CONTRAST_STD_MIN	12.0	
QUALITY_DICTIONARY_GATE_ENABLED	false	Lexicon nội tại cố tình nhỏ.
SCAN_TABLE_DETECTION_ENABLED	true	

Bất biến kiến trúc

Biến môi trường là *ghi đè triển khai tường minh*; ngoài ra chính tệp YAML có version mới sở hữu ngưỡng (`quality._threshold` kiểm tra `os.getenv` trước khi tra YAML). Đổi ngưỡng phải đi kèm tăng `THRESHOLD_VERSION`.

B.6 Nạp theo lô

Thiết lập	Mặc định	Ý nghĩa
BULK_MAX_ROWS	50	Số Case tối đa một lần import tạo ra.
BULK_AUTO_MAP_THRESHOLD	0.90	Ngưỡng tự động chấp nhận ánh xạ cột.
BULK_HEADER_ALIAS_MIN	0.75	
BULK_RECORD_NAME_VALUE_MIN	0.60	
BULK_TABLE_LABEL_ANTI_SIGNAL_MAX	0.25	
BULK_OCR_TABLE_CONFIDENCE_MIN	0.80	
HEADER_ALIAS_PATH	config/header_aliases.yaml	

B.7 Phiên bản đóng dấu lên kết quả

Trục version	Giá trị hiện tại
PARSER_VERSION	parser-2026.09-production.3
MODEL_VERSION	hybrid-rule-ner-2026.09-production.1
THRESHOLD_VERSION	threshold-calibrated-2026.09-v2
TAXONOMY_VERSION	taxonomy-2026.09
registry_version	Lấy từ version danh mục đang PUBLISHED.
policy_version	Lấy từ từng PolicyRule, ghi trên mỗi assessment.

B.8 Mail và quét mã độc

Thiết lập	Mặc định	Ý nghĩa
EMAIL_USER	—	Bắt buộc ở production.
EMAIL_APP_PASSWORD	—	App password của Google, bắt buộc ở production.
EMAIL_FROM_NAME	Hệ thống truy cập uCA/BQP	Chỉ đặt tên hiển thị.
SMTP_TIMEOUT_SECONDS	15.0	
ANTIMALWARE_ENABLED	true	
ANTIMALWARE_REQUIRED	false	Production bắt buộc true (fail-closed).
CLAMAV_HOST / CLAMAV_PORT	localhost / 3310	
ANTIMALWARE_TIMEOUT_SECONDS	8.0	

PHỤ LỤC C

Phụ lục C — Mã nguồn sơ đồ Mermaid

Toàn bộ sơ đồ trong tài liệu này được viết bằng **Mermaid** và lưu dưới dạng tệp `.mmd` độc lập trong thư mục `diagrams/`. Chúng được dựng thành PDF bằng `@mermaid-js/mermaid-cli` và chèn vào tài liệu qua `\includegraphics`, nên hình vẽ trong bản in *chính là* mã nguồn dưới đây, không phải một bản vẽ tay riêng biệt.

Cách dựng lại:

```
cd SoftwareArchitectureDocument
make diagrams      # dựng *.mmd -> diagrams/out/*.pdf
make               # biên dịch main.tex bằng xelatex
make all           # dựng sơ đồ rồi biên dịch
```

Mã nguồn đầy đủ của từng sơ đồ, theo thứ tự xuất hiện:

C.0.1 01-context.mmd — Sơ đồ ngữ cảnh

```
1 %% Sơ đồ ngữ cảnh hệ thống (C4 mức 1)
2 flowchart TB
3     classDef person fill:#d9d9d9,stroke:#333333,color:#000000
4     classDef system fill:#f2f2f2,stroke:#000000,color:#000000
5     classDef ext fill:#fffff,stroke:#666666,color:#000000
6
7     CB["Cán bộ tra cứu<br/>CASE_CREATE, CASE_BULK, LOOKUP_UNIT"]:::person
8     TD["Cán bộ thẩm định<br/>REVIEW_QUEUE, REVIEW_DECIDE"]:::person
9     QT["Quản trị viên<br/>USER_ADMIN, REGISTRY_ADMIN, AUDIT_VIEW"]:::person
10
11     SYS["Nền tảng xác minh CA/BQP<br/>ca-bqp-verification-platform"]:::system
12
13     GMAIL["Gmail SMTP<br/>smtp.gmail.com:587 STARTTLS"]:::ext
14     CLAMAV["ClamAV<br/>quét mã độc tệp tải lên"]:::ext
15     PROM["Prometheus<br/>thu thập /metrics"]:::ext
16     WAF["WAF / Reverse proxy<br/>F5 BIG-IP ASM hoặc tương đương"]:::ext
17     CRAWL["Nguồn dữ liệu danh mục<br/>văn bản chính thức, crawl offline"]:::ext
18
19     CB -->|"Nộp text/tệp, xem kết quả"| SYS
20     TD -->|"Xử lý hàng đợi thẩm định"| SYS
21     QT -->|"QA danh mục, quản trị tài khoản"| SYS
22
23     WAF -->|"Lọc L7, chuyển tiếp X-Forwarded-For"| SYS
24     SYS -->|"Gửi mật khẩu cấp phát và mã OTP"| GMAIL
25     SYS -->|"Quét tệp trước khi lưu"| CLAMAV
26     PROM -->|"Thu thập số liệu"| SYS
27     CRAWL -->|"Ứng viên danh mục, luôn qua QA"| SYS
```

C.0.2 02-containers.mmd — Sơ đồ container, profile docker

```
1 %% Sơ đồ container - profile docker (C4 mức 2)
2 flowchart TB
3     subgraph CLIENT["Trình duyệt"]
4         SPA["Web SPA<br/>React 19 + TypeScript + Vite<br/>Tailwind CSS 4"]
```

```

5     end
6
7     subgraph EDGE["Lớp biên"]
8         NGINX["Nginx<br/>phục vụ SPA, reverse proxy /api/v1"]
9     end
10
11    subgraph APP["Lớp ứng dụng"]
12        API["FastAPI<br/>cabqp.main:app<br/>uvicorn, /api/v1"]
13        WORKER["Celery Worker<br/>queues: celery, documents, outbox, batch"]
14        BEAT["Celery Beat<br/>dispatch_pending_outbox, reconcile_bulk_jobs"]
15    end
16
17    subgraph DATA["Lớp dữ liệu"]
18        PG["PostgreSQL<br/>dữ liệu nghiệp vụ, audit append-only"]
19        REDIS["Redis<br/>broker/backend Celery, rate limit"]
20        MINIO["MinIO<br/>tập gốc, s3://bucket/key"]
21    end
22
23    subgraph AI["Tài sản mô hình - bake sẵn trong image"]
24        EASY["EasyOCR<br/>detector CRAFT + recognizer Latin VI"]
25        VIET["VietOCR<br/>recognizer dự phòng, vendored"]
26        PPS["PP-StructureV3<br/>nhận dạng cấu trúc bảng"]
27        EMB["sentence-transformers<br/>paraphrase-multilingual-MiniLM-L12-v2"]
28    end
29
30    SPA --> NGINX
31    NGINX --> API
32    API --> PG
33    API --> REDIS
34    API --> MINIO
35    WORKER --> PG
36    WORKER --> REDIS
37    WORKER --> MINIO
38    BEAT --> REDIS
39    API -.->|"OutboxEvent + apply_async"| REDIS
40    REDIS -.->|"tiêu thụ job"| WORKER
41    WORKER --> AI
42    API --> EMB

```

C.0.3 03-local-profile.mmd — Profile local

```

1  %% Profile local - một tiến trình, chỉ phụ thuộc PostgreSQL
2  flowchart TB
3      SPA["Vite dev server :3000<br/>proxy /api, /health"]
4      API["FastAPI + uvicorn :8000<br/>đồng thời là task runner"]
5      SWEEP["LocalTaskQueue<br/>thread pool + sweeper outbox"]
6      FS["FilesystemStorage<br/>STORAGE_ROOT, file:///abs/path"]
7      PG["PostgreSQL :5433<br/>cụm riêng dưới ~/.local/share/cabqp"]
8
9      SPA --> API
10     API --> PG
11     API --> FS
12     API -->|"lifespan khởi động"| SWEEP
13     SWEEP -->|"quét outbox_events PENDING"| PG
14     SWEEP -->|"run_task_now process_document"| API
15
16     NOTE["Không có Redis, MinIO, Celery.<br/>effective_queue_backend = inline<br/>effective_storage_backend"]
17     SWEEP -. NOTE

```

C.0.4 04-backend-components.mmd — Thành phần backend

```

1  %% Thành phần backend - cabqp
2  flowchart LR
3      subgraph APIL["cabqp.api - router mỏng, /api/v1"]
4          A1["auth.py"]
5          A2["cases.py"]
6          A3["bulk.py"]

```

```

7      A4["reviews.py"]
8      A5["lookup.py"]
9      A6["admin_registry.py"]
10     A7["admin_person_registry.py"]
11     A8["admin_users.py"]
12     A9["audit.py"]
13     A10["health.py"]
14 end
15
16 subgraph MOD["cabqp.modules - dịch vụ nghiệp vụ"]
17     M1["auth<br/>permissions, service, dependencies, otp"]
18     M2["document_intelligence<br/>router, parsers, ocr, extraction,<br/>quality, subject_split,
19     M3["resolution<br/>Resolver, calibration"]
20     M4["person_resolution<br/>PersonResolver, PersonRegistryService"]
21     M5["registry<br/>RegistryService"]
22     M6["subject_group"]
23     M7["decisioning<br/>PolicyEngine"]
24     M8["cases<br/>process_case"]
25     M9["review"]
26     M10["bulk"]
27     M11["audit"]
28     M12["notifications<br/>email, templates"]
29 end
30
31 subgraph SHARED["cabqp.shared"]
32     S1["models.py - ORM"]
33     S2["settings.py"]
34     S3["middleware.py"]
35     S4["normalization.py"]
36     S5["schemas.py"]
37     S6["logging.py, metrics.py, resilience.py"]
38 end
39
40 subgraph WORK["cabqp.workers"]
41     W1["celery_app.py"]
42     W2["tasks.py"]
43     W3["local_queue.py"]
44 end
45
46 A2 --> M8
47 A3 --> M10
48 A4 --> M9
49 A5 --> M3
50 A6 --> M5
51 A7 --> M4
52 A8 --> M1
53 A8 --> M12
54 A1 --> M1
55 M8 --> M2
56 M8 --> M3
57 M8 --> M4
58 M8 --> M6
59 M8 --> M7
60 M8 --> M11
61 M9 --> M7
62 M10 --> M8
63 W2 --> M2
64 W2 --> M8
65 W2 --> M10
66 MOD --> SHARED
67 WORK --> SHARED

```

C.0.5 05-case-sequence .mmd — Trình tự xử lý một Case

```

1 %% Luồng xử lý một Case từ tệp tải lên
2 sequenceDiagram
3     autonumber
4     actor U as Cán bộ tra cứu
5     participant W as Web SPA
6     participant API as FastAPI /cases/file

```

```

7 participant V as validate_upload + scan_upload
8 participant R as route_input
9 participant ST as ObjectStorage
10 participant DB as PostgreSQL
11 participant Q as Outbox dispatcher
12 participant T as process_document
13 participant P as process_case
14
15 U->>W: Chọn tệp hồ sơ
16 W->>API: POST /cases/file (Idempotency-Key)
17 API->>DB: Tra Case theo (created_by, idempotency_key)
18 alt Đã tồn tại
19 API->>W: 200 idempotent_replay = true
20 else Chưa tồn tại
21 API->>V: Kiểm tra đuôi, magic byte, ClamAV
22 V->>API: OK hoặc 422/400
23 API->>R: Quyết định chiến lược đọc
24 alt TABULAR_LIST
25 API->>W: 422 TABULAR_LIST_REQUIRES_BULK
26 else Hồ sơ đơn
27 API->>ST: put(key, content)
28 API->>DB: INSERT Case + Document + OutboxEvent + AuditLog
29 API->>W: 202 case_id, document_id
30 API->>Q: enqueue dispatch_outbox_event
31 end
32 end
33
34 Q->>T: Giao việc (Celery hoặc inline)
35 T->>ST: get(key)
36 T->>T: parse_document -> text, structured, parse_quality
37 T->>T: detect_subject_blocks
38 alt Nhiều chủ thể, nguồn text
39 T->>DB: Tạo Case anh em cho mỗi block
40 else OCR nhiều chủ thể
41 T->>DB: ReviewCase MULTIPLE_SUBJECTS_OCR_UNSUPPORTED
42 end
43 T->>P: process_case(case)
44 P->>P: extract -> resolve -> subject_group -> policy
45 P->>DB: UPSERT ExtractedRecord + VerificationResult
46 alt Đủ căn cứ
47 P->>DB: Case = COMPLETED
48 else Thiếu căn cứ
49 P->>DB: Case = NEED_REVIEW + ReviewCase
50 end
51 P->>DB: AuditLog CASE_DECISION
52 W->>API: GET /cases/{id} (polling)
53 API->>W: case, extracted, result, eligibility

```

C.0.6 06-input-routing.mmd — Định tuyến đầu vào

```

1 %% route_input - quyết định chiến lược đọc trước khi parse
2 flowchart TB
3   IN["Tệp tải lên"] --> VAL["validate_upload<br/>allow-list đuôi, magic byte,<br/>chống zip-slip v
4   VAL -->|"Không hợp lệ"| REJ["422 / 400"]
5   VAL --> TYPE{"Loại tệp"}
6
7   TYPE -->|"TXT / CSV"| PT["PLAIN_TEXT"]
8   TYPE -->|"DOCX"| DX["DOCX_TEXT"]
9   TYPE -->|"PNG / JPG"| IMG["IMAGE -> OCR"]
10
11  TYPE -->|"PDF"| PROBE["probe_pdf<br/>lấy mẫu trang: ký tự, phủ ảnh,<br/>font nhúng, entropy, tỉ
12  PROBE -->|"PyMuPDF không mở được"| INV["DocumentLimitError<br/>INVALID_PDF_DOCUMENT<br/>KHÔNG f
13  PROBE -->|"Mọi trang có text"| PTX["PDF_TEXT"]
14  PROBE -->|"Mọi trang là ảnh"| PSC["PDF_SCAN -> OCR"]
15  PROBE -->|"Hỗn hợp"| PHY["PDF_HYBRID"]
16
17  TYPE -->|"XLSX / XLS"| SHEET["Phân tích bảng tính"]
18  SHEET -->|"Có header danh sách nhân sự"| TAB["TABULAR_LIST<br/>bắt buộc đi /api/v1/bulk"]
19  SHEET -->|"Một hồ sơ dạng khóa-giá trị"| KV["KEY_VALUE_SHEET"]
20

```

```

21 PT --> OK["Tiếp tục parse + extraction"]
22 DX --> OK
23 PTX --> OK
24 KV --> OK
25 IMG --> OCRP["Pipeline OCR"]
26 PSC --> OCRP
27 PHY --> OCRP
28 OCRP --> OK
29 TAB --> BULK["Bulk ingestion:<br/>mỗi dòng một Case độc lập"]

```

C.0.7 07-ocr-pipeline.mmd — Pipeline OCR

```

1 %% Pipeline OCR - engine chính và một fallback duy nhất
2 flowchart TB
3   SRC["Ảnh / trang PDF rasterize"] --> PRE["preprocess<br/>deskew theo đoạn thẳng Hough,<br/>persp"]
4   PRE --> IMGQ["Đo chất lượng ảnh nguồn<br/>blur variance, contrast std"]
5   IMGQ --> E1["Engine chính<br/>EasyOCR detector CRAFT + recognizer VI"]
6   E1 --> GATE1["Quality gate<br/>phân bố confidence, charset,<br/>tỉ lệ dấu, IoC/entropy cho văn x"]
7
8   GATE1 --> COND{"Cần chạy fallback?"}
9   COND -->|"gate FAIL<br/>hoặc ảnh mờ/tương phản thấp<br/>hoặc mean confidence < OCR_FALLBACK_CONF"]
10  COND -->|"Không"| PICK["Chọn nguyên vẹn output engine chính"]
11
12  E2 --> CMP{"Hai engine lệch nhau<br/>ở trường định danh?"}
13  CMP -->|"Có"| FAILC["parse_quality.critical_disagreement = true<br/>confidence = 0 -> NEED_REVIEW"]
14  CMP -->|"Không"| PICK2["Chọn nguyên vẹn output của một engine<br/>không ghép, không bỏ phiếu ký"]
15
16  PICK --> EV["Evidence: primary_confidence,<br/>fallback_reason, fallback_confidence"]
17  PICK2 --> EV
18  FAILC --> EV

```

C.0.8 08-subject-split.mmd — Tách nhiều chủ thể

```

1 %% Tách nhiều chủ thể trong một tài liệu
2 flowchart TB
3   DOC["Tài liệu đã parse"] --> KIND{"parse_method"}
4
5   KIND -->|"PLAIN_TEXT / DOCX_TEXT / PDF_TEXT"| SPLIT["detect_subject_blocks<br/>dùng lại LABELS v"]
6   KIND -->|"OCR / PADDLE_OCR / PDF_HYBRID"| OCRM{"Nhiều nhân Họ và tên?"}
7   KIND -->|"TABULAR_LIST"| BULKR["Tù chối khỏi endpoint đơn Case<br/>TABULAR_LIST_REQUIRES_BULK"]
8
9   OCRM -->|"Có"| ABST1["NEED_REVIEW<br/>MULTIPLE_SUBJECTS_OCR_UNSUPPORTED<br/>không tách theo bbo"]
10  OCRM -->|"Không"| ONE["Một Case duy nhất"]
11
12  SPLIT --> N{"Số block"}
13  N -->|"1"| ONE
14  N -->|"nhiều"| FAN["Fan-out: mỗi block một Case độc lập"]
15
16  FAN --> B0["Block #0<br/>Case tạo đồng bộ lúc upload,<br/>GIỮ Idempotency-Key của client"]
17  FAN --> BN["Block #n>0<br/>idempotency_key = document.id:block_index"]
18  FAN --> EMPTY{"Block không trích xuất được gì<br/>trước nhận tên kế tiếp?"}
19  EMPTY -->|"Có"| ABST2["NEED_REVIEW<br/>MULTIPLE_SUBJECTS_AMBIGUOUS_BOUNDARY"]
20
21  B0 --> GRP["evidence.split_source.source_case_id<br/>là cách nhận lại nhóm, không dùng tiền tố"]
22  BN --> GRP

```

C.0.9 09-resolution-cascade.mmd — Cascade phân giải đơn vị

```

1 %% Cascade phân giải tên đơn vị - modules/resolution/service.py
2 flowchart TB
3   IN["unit_name và/hoặc unit_code<br/>as_of_date"] --> IDX["Nạp index từ snapshot của<br/>Registry"]
4   IDX --> CODE{"unit_code khớp<br/>UnitCode đã APPROVED?"}
5
6   CODE -->|"Có + tên khớp đơn vị khác"| CONF1["CONFLICT<br/>CODE_NAME_CONFLICT<br/>organization_ty"]

```

```

7 CODE -->|"Có + tên mạnh trái ngược<br/>fuzzy >= 97 và margin >= 10"| CONF2["CONFLICT<br/>CODE_S"]
8 CODE -->|"Có, không mâu thuẫn"| M1["MATCHED - TRUSTED_CODE<br/>score 100, confidence 1.0"]
9 CODE -->|"Không"| N1{"CANONICAL_EXACT<br/>normalized_key"}
10
11 N1 -->|"Đúng 1"| M2["MATCHED - CANONICAL_EXACT - 100"]
12 N1 -->|"Nhiều"| AMB["AMBIGUOUS<br/>không bao giờ chọn đại"]
13 N1 -->|"Không"| N2{"APPROVED_ALIAS"}
14
15 N2 -->|"Đúng 1"| M3["MATCHED - APPROVED_ALIAS - 99"]
16 N2 -->|"Nhiều"| AMB
17 N2 -->|"Không"| N3{"ASCII_FOLDED<br/>bỏ dấu"}
18
19 N3 -->|"Đúng 1"| M4["MATCHED - ASCII_FOLDED - 97"]
20 N3 -->|"Nhiều"| AMB
21 N3 -->|"Không"| N4{"ASCII_FOLDED_ALIAS"}
22
23 N4 -->|"Đúng 1"| M5["MATCHED - ASCII_FOLDED_ALIAS - 96"]
24 N4 -->|"Nhiều"| AMB
25 N4 -->|"Không"| RANK["Xếp hạng lai<br/>combined = 0.65*fuzzy + 0.20*BM25 + 0.15*semantic"]
26
27 RANK --> CAL["Calibration<br/>predict(fuzzy, margin, bm25, semantic)"]
28 CAL --> STRONG["fuzzy >= RESOLVER_FUZZY_THRESHOLD<br/>và margin >= RESOLVER_MARGIN_THRESHOLD<br/>"]
29 STRONG -->|"Đạt"| M6["MATCHED<br/>HYBRID_FUZZY_BM25_SEMANTIC"]
30 STRONG -->|"Không"| SEM{"Semantic auto-accept<br/>mặc định TẮT"}
31 SEM -->|"Bật và đạt ngưỡng + calibrated"| M7["MATCHED - SEMANTIC_CALIBRATED"]
32 SEM -->|"Không"| SCORE["combined_score >= max(50, threshold-15)"]
33 SCORE -->|"Có"| AMB2["AMBIGUOUS - HYBRID_RANKING"]
34 SCORE -->|"Không"| NF["NOT_FOUND<br/>organization_type vẫn là UNKNOWN,<br/>NOT_FOUND != OTHER"]

```

C.0.10 10-person-resolution.mmd — Cascade phân giải nhân sự

```

1 %% PersonResolver - chỉ định danh, không tự quyết định tổ chức
2 flowchart TB
3     IN["full_name / personal_code<br/>birth_year, unit_id, as_of_date"] --> PC{"personal_code<br/>KHOẢNG trống"}
4     PC -->|"Nhiều hoặc mâu thuẫn"| ABS1["ABSTAIN - AMBIGUOUS/CONFLICT"]
5     PC -->|"Đúng 1"| MATCH["MATCHED - TRUSTED_CODE"]
6     PC -->|"Không"| NAME["Bổn tăng tên chính xác<br/>CANONICAL_EXACT 100 / APPROVED_ALIAS 99<br/>AS"]
7
8     NAME --> NARROW["Thu hẹp bằng birth_year, unit_id"]
9     NARROW -->|"Đúng 1"| MATCH
10    NARROW -->|"Nhiều"| ABS2["AMBIGUOUS"]
11    NARROW -->|"Không"| FUZ["Fuzzy rapidfuzz WRatio<br/>KHÔNG có BM25, KHÔNG có semantic,<br/>KHÔNG có"]
12    FUZ -->|"Đủ mạnh và duy nhất"| MATCH
13    FUZ -->|"Còn lại"| ABS3["AMBIGUOUS / NOT_FOUND"]
14
15    MATCH --> CU["Trả về canonical_unit_id"]
16    CU --> REU["process_case gọi lại Resolver đơn vị<br/>trên canonical_unit_id"]
17    REU --> ORG["organization_type BCA/BQP/OTHER/UNKNOWN<br/>chỉ có một nguồn thẩm quyền duy nhất"]

```

C.0.11 11-decision-ladder.mmd — Thang điều kiện thẩm định

```

1 %% Thang điều kiện đưa Case sang NEED_REVIEW - modules/cases/service.py
2 flowchart TB
3     START["Sau extraction + resolution + subject_group"] --> G1{"batch_warnings?"}
4     G1 -->|"Có"| R1["reason = mã cảnh báo đầu tiên"]
5     G1 -->|"Không"| G2{"person_resolution != MATCHED?"}
6     G2 -->|"Có"| R2["reason = PERSON_<status>"]
7     G2 -->|"Không"| G3{"resolution_status != MATCHED?"}
8     G3 -->|"Có"| R3["reason = AMBIGUOUS / NOT_FOUND / CONFLICT"]
9     G3 -->|"Không"| G4{"organization_type = UNKNOWN?"}
10    G4 -->|"Có"| R4["ORGANIZATION_TYPE_UNKNOWN"]
11    G4 -->|"Không"| G5{"parse gate FAIL hoặc<br/>parse_confidence < ngưỡng?"}
12    G5 -->|"Có"| R5["DOCUMENT_PARSE_LOW_CONFIDENCE"]
13    G5 -->|"Không"| G6{"extraction < EXTRACTION_MIN_CONFIDENCE?"}
14    G6 -->|"Có"| R6["DOCUMENT_EXTRACTION_LOW_CONFIDENCE"]
15    G6 -->|"Không"| G7{"không phải relationless_lookup<br/>và relation_confidence < 0.7?"}
16    G7 -->|"Có"| R7["CURRENT_WORK_UNIT_UNCERTAIN"]

```

```

17      G7 -->|"Không"| G8{"subject_group = None?"}
18      G8 -->|"Có"| R8["SUBJECT_GROUP_INSUFFICIENT"]
19      G8 -->|"Không"| POL["Chạy PolicyEngine"]
20      POL --> G9{"Có assessment INSUFFICIENT_DATA?"}
21      G9 -->|"Có"| R9["POLICY_INSUFFICIENT_DATA"]
22      G9 -->|"Không"| DONE["Case = COMPLETED<br/>đóng ReviewCase còn mở nếu có"]
23
24      R1 --> NR["Case = NEED_REVIEW<br/>tạo/cập nhật ReviewCase<br/>coverage_group = _review_scope"]
25      R2 --> NR
26      R3 --> NR
27      R4 --> NR
28      R5 --> NR
29      R6 --> NR
30      R7 --> NR
31      R8 --> NR
32      R9 --> NR

```

C.0.12 12-case-state.mmd — Máy trạng thái Case

```

1  %% Vòng đời Case
2  stateDiagram-v2
3      [*] --> RECEIVED : POST /cases/text hoặc /cases/file
4      RECEIVED --> PROCESSING : worker nhận job
5      PROCESSING --> COMPLETED : đủ căn cứ, không rule nào thiếu dữ kiện
6      PROCESSING --> NEED_REVIEW : trượt bất kỳ cổng nào trong thang điều kiện
7      PROCESSING --> FAILED : hết ngân sách retry, dead-letter
8      RECEIVED --> FAILED : parse/limit lỗi cứng
9
10     NEED_REVIEW --> COMPLETED : reviewer CONFIRM / UNKNOWN / INSUFFICIENT
11     NEED_REVIEW --> FAILED : reviewer DISMISS (đóng fail-closed)
12     NEED_REVIEW --> NEED_REVIEW : xử lý lại, version_no tăng
13
14     COMPLETED --> [*]
15     FAILED --> [*]
16
17     note right of NEED_REVIEW
18         Luôn tồn tại đúng một ReviewCase OPEN.
19         Xử lý lại thành công sẽ DISMISS review cũ.
20     end note
21     note right of COMPLETED
22         COMPLETED là trạng thái quy trình,
23         không phải kết luận nghiệp vụ.
24         Reviewer chọn UNKNOWN vẫn đóng COMPLETED
25         trong khi result giữ NOT_FOUND/UNKNOWN.
26     end note

```

C.0.13 13-registry-lifecycle.mmd — Vòng đời version danh mục

```

1  %% Vòng đời version danh mục - áp dụng độc lập cho Unit Registry và Person Registry
2  stateDiagram-v2
3      [*] --> DRAFT : create_version
4      DRAFT --> DRAFT : thêm/sửa unit, alias, code<br/>chỉ cho phép trên DRAFT
5      DRAFT --> VALIDATED : validate_version<br/>kiểm tra mọi bản ghi đã APPROVED
6      VALIDATED --> APPROVED : approve_version
7      APPROVED --> PUBLISHED : publish_version<br/>pg_advisory_xact_lock + SELECT FOR UPDATE
8      PUBLISHED --> DEPRECATED : bị version mới thay thế
9      DEPRECATED --> [*]
10
11     note right of PUBLISHED
12         Bất biến: đúng một version PUBLISHED
13         tại mọi thời điểm, cường chế bằng
14         partial unique index.
15         Snapshot là bất biến - rollback
16         clone snapshot cũ thành version mới.
17     end note
18     note left of DRAFT
19         Ứng viên từ crawler/reviewer

```

```

20         vào registry_candidates với
21         qa_status = PENDING_QA,
22         không bao giờ ghi thẳng vào
23         version đang PUBLISHED.
24     end note

```

C.0.14 14-erd-core.mmd — Mô hình dữ liệu luồng Case

```

1  %% Mô hình dữ liệu - luồng Case
2  erDiagram
3      CASES ||--o{ DOCUMENTS : "chứa"
4      CASES ||--o{ EXTRACTED_RECORDS : "trích xuất"
5      CASES ||--o{ VERIFICATION_RESULTS : "kết luận"
6      CASES ||--o{ REVIEW_CASES : "thẩm định"
7      DOCUMENTS ||--o{ EXTRACTED_RECORDS : "truy vết nguồn"
8      VERIFICATION_RESULTS ||--o{ ELIGIBILITY_ASSESSMENTS : "đánh giá"
9      POLICY_RULES ||--o{ ELIGIBILITY_ASSESSMENTS : "áp dụng"
10     UNITS ||--o{ VERIFICATION_RESULTS : "đơn vị đã phân giải"
11     BULK_INGEST_JOBS ||--o{ BULK_INGEST_ROWS : "gồm"
12     BULK_INGEST_ROWS ||--o{ CASES : "sinh ra"
13     BULK_INGEST_JOBS ||--o{ BULK_ROW_ERRORS : "lỗi dòng"
14
15     CASES {
16         string id PK
17         string created_by FK
18         string input_type
19         text raw_text
20         json input_payload
21         string idempotency_key UK
22         string workflow_status
23         datetime created_at
24     }
25     DOCUMENTS {
26         string id PK
27         string case_id FK
28         string file_name
29         string mime_type
30         int size_bytes
31         text storage_uri
32         string checksum
33         string parse_status
34         float parse_confidence
35     }
36     EXTRACTED_RECORDS {
37         string id PK
38         string case_id FK
39         string document_id FK
40         string subject_name
41         string subject_code
42         string position
43         string current_unit_raw
44         json former_units
45         float extraction_confidence
46         float relation_confidence
47     }
48     VERIFICATION_RESULTS {
49         string id PK
50         string case_id UK
51         string unit_id FK
52         string organization_type
53         string resolution_status
54         string subject_group
55         string match_method
56         float resolution_score
57         float candidate_margin
58         float decision_confidence
59         json top_candidates
60         json evidence
61         string registry_version
62         string taxonomy_version

```

```

63     string parser_version
64     string model_version
65     string threshold_version
66 }
67 ELIGIBILITY_ASSESSMENTS {
68     string id PK
69     string result_id FK
70     string policy_id FK
71     string status
72     text reason
73     json evidence
74     string policy_version
75 }
76 REVIEW_CASES {
77     string id PK
78     string case_id FK
79     string result_id FK
80     string reason
81     string status
82     string coverage_group
83     string assigned_to
84     json payload
85     int version_no
86 }

```

C.0.15 15-erd-registry.mmd — Mô hình dữ liệu danh mục và hạ tầng

```

1 %% Mô hình dữ liệu - hai danh mục độc lập và hạ tầng chung
2 erDiagram
3     REGISTRY_VERSIONS ||--o{ REGISTRY_SNAPSHOT_UNITS : "ảnh chụp bất biến"
4     REGISTRY_VERSIONS ||--o{ REGISTRY_SNAPSHOT_NAMES : "ảnh chụp bất biến"
5     REGISTRY_VERSIONS ||--o{ REGISTRY_SNAPSHOT_CODES : "ảnh chụp bất biến"
6     UNITS ||--o{ UNIT_NAMES : "bí danh"
7     UNITS ||--o{ UNIT_CODES : "mã"
8     UNITS ||--o{ UNITS : "đơn vị cha"
9     SOURCES ||--o{ UNITS : "nguồn"
10    REGISTRY_CANDIDATES }o--o| UNITS : "ứng viên QA"
11
12    PERSON_REGISTRY_VERSIONS ||--o{ PERSON_REGISTRY_SNAPSHOTS : "ảnh chụp bất biến"
13    PERSONS ||--o{ PERSON_NAMES : "bí danh"
14    PERSONS ||--o{ PERSON_CODES : "mã"
15    UNITS ||--o{ PERSONS : "canonical_unit_id"
16    PERSON_REGISTRY_CANDIDATES }o--o| PERSONS : "ứng viên QA"
17
18    APP_USERS ||--o{ USER_SESSIONS : "phiên"
19    APP_USERS ||--o{ LOGIN_OTPS : "mã một lần"
20
21    UNITS {
22        string id PK
23        string canonical_name
24        string normalized_key
25        string ascii_key
26        string organization_type
27        string unit_level
28        string coverage_group
29        string parent_unit_id FK
30        string qa_status
31        date valid_from
32        date valid_to
33        bool active
34    }
35    UNIT_NAMES {
36        string id PK
37        string unit_id FK
38        string name
39        string normalized_key
40        string ascii_key
41        string name_type
42        string qa_status
43    }

```

```

44     UNIT_CODES {
45         string id PK
46         string unit_id FK
47         string code
48         string namespace
49         string qa_status
50     }
51     REGISTRY_VERSIONS {
52         string id PK
53         string version UK
54         string status
55         datetime published_at
56     }
57     PERSONS {
58         string id PK
59         string full_name
60         string normalized_key
61         string ascii_key
62         int birth_year
63         string canonical_unit_id FK
64         string subject_group_hint
65         string employment_status
66         string qa_status
67         string source_kind
68         json synthetic_welfare_facts
69     }
70     APP_USERS {
71         string username PK
72         string password_hash
73         string display_name
74         string personal_code UK
75         string email UK
76         json permissions
77         json coverage_groups
78         bool is_admin
79         bool is_active
80         bool must_change_password
81         int failed_attempts
82         datetime locked_until
83     }
84     USER_SESSIONS {
85         string id PK
86         string token_hash UK
87         string username FK
88         datetime expires_at
89         datetime revoked_at
90     }
91     AUDIT_LOGS {
92         string id PK
93         string actor
94         string role
95         string action
96         string entity_type
97         string entity_id
98         json metadata_json
99         datetime timestamp
100    }
101    OUTBOX_EVENTS {
102        string id PK
103        string event_type
104        string aggregate_type
105        string aggregate_id
106        json payload
107        string status
108        int attempts
109        datetime available_at
110    }

```

C.0.16 16-review-sequence.mmd — Trình tự thẩm định

```

1 %% Quyết định thẩm định thủ công
2 sequenceDiagram
3     autonumber
4     actor TD as Cán bộ thẩm định
5     participant W as ReviewsPage.jsx
6     participant API as POST /reviews/{id}/decision
7     participant SVC as review/service.decide_review
8     participant DB as PostgreSQL
9
10    TD->>W: Mở hàng đợi, chọn Case
11    W->>API: GET /reviews (page, page_size)
12    W->>API: POST /lookup/unit (chọn đơn vị cho CONFIRM)
13    TD->>W: Chọn quyết định + expected_version
14    W->>API: POST /reviews/{id}/decision
15    API->>SVC: require_perms(REVIEW_DECIDE)
16    SVC->>DB: SELECT ... FOR UPDATE trên review
17    SVC->>SVC: _authorize_review<br/>assigned_to và coverage_group
18    alt version_no != expected_version
19        SVC->>W: 409 đã bị người khác thay đổi
20    else
21        alt CONFIRM
22            SVC->>DB: unit phải active và APPROVED<br/>result = MATCHED, HUMAN_REVIEW, conf 1.0
23            DB-->>SVC: Case = COMPLETED
24        else UNKNOWN
25            SVC->>DB: unit_id = NULL, org = UNKNOWN,<br/>resolution_status = NOT_FOUND
26        else INSUFFICIENT
27            SVC->>DB: giữ nguyên đơn vị đã xác minh,<br/>chỉ ghi nhận thiếu dữ kiện chính sách
28        else DISMISS
29            SVC->>DB: review = DISMISSED,<br/>Case = FAILED (fail-closed)
30        end
31        SVC->>DB: AuditLog REVIEW_DECISION
32        SVC-->>W: 200 trạng thái mới
33    end
34
35    note over SVC,DB: Chính sửa của reviewer có thể tạo RegistryCandidate<br/>nhưng KHÔNG bao giờ t

```

C.0.17 17-bulk-flow.mmd — Luồng nạp theo lô

```

1 %% Nạp danh sách theo lô - mỗi dòng một Case độc lập
2 flowchart TB
3     UP["POST /api/v1/bulk<br/>bảng tính hoặc bảng danh sách"] --> HASH["file_sha256 duy nhất<br/>chặt"]
4     HASH --> PROF["profile_and_validate<br/>đọc sheet, dò header, dựng ma trận"]
5     PROF --> CAP["Số dòng > BULK_MAX_ROWS (50)?"]
6     CAP -->|"Vượt"| E422["422 {code, rows, limit, message}<br/>web hiển thị nguyên văn message"]
7     CAP --> MAP["infer_mapping<br/>điểm alias header + heuristic hình dạng giá trị"]
8
9     MAP --> SCORE["Mọi trường bắt buộc đạt<br/>BULK_AUTO_MAP_THRESHOLD?"]
10    SCORE -->|"Không"| AWAIT["AWAITING_MAPPING<br/>chờ người dùng ánh xạ cột"]
11    AWAIT --> CONF["POST /bulk/{id}/confirm<br/>re-profile với mapping do người dùng chọn"]
12    CONF --> MAP
13    SCORE -->|"Có"| QUEUE["QUEUED -> PROCESSING"]
14
15    QUEUE --> ROWS["Mỗi BulkIngestRow<br/>row_hash chống trùng trong cùng job"]
16    ROWS --> TASK["process_bulk_row<br/>tạo Case + process_case"]
17    TASK --> CNT["refresh_job_counts<br/>KHÓA dòng job trước khi tính lại<br/>tránh lost update giữa"]
18    CNT --> FIN["Kết thúc"]
19    FIN --> OKJ["COMPLETED"]
20    FIN --> ERRJ["COMPLETED_WITH_ERRORS"]
21    FIN --> FJ["FAILED"]
22
23    NOTE["looks_like_person_name yêu cầu có khoảng trắng<br/>VÀ ít nhất một chữ thường, nên cột hân"]
24    MAP -.- NOTE

```

C.0.18 18-auth-sequence.mmd — Trình tự xác thực

```

1 %% Đăng nhập bằng mật khẩu và bằng mã một lần
2 sequenceDiagram
3     autonumber
4     actor U as Người dùng
5     participant W as Web SPA
6     participant MW as middleware<br/>SecurityHeaders / RateLimit / RequestContext
7     participant API as api/auth.py
8     participant S as auth/service.py
9     participant O as auth/otp.py
10    participant M as Gmail SMTP
11    participant DB as PostgreSQL
12
13    rect rgb(238, 244, 250)
14    note over U,DB: Đường mật khẩu
15    U->>W: username + password
16    W->>MW: POST /api/v1/auth/login
17    MW->>MW: Ngân sách theo route, theo địa chỉ,<br/>và ngân sách riêng cho credential
18    MW->>API: chuyển tiếp
19    API->>S: verify PBKDF2-HMAC-SHA256
20    alt Sai
21        S->>DB: record_failed_attempt, khóa tạm khi vượt ngưỡng
22        API-->>W: 401 (bị tính vào ngân sách credential)
23    else Đúng
24        S->>DB: SINGLE_ACTIVE_SESSION: thu hồi mọi phiên cũ
25        S->>DB: INSERT UserSession (chỉ lưu SHA-256 của token)
26        API-->>W: token ngẫu nhiên 32 byte, không mang claim
27    end
28    end
29
30    rect rgb(245, 240, 235)
31    note over U,DB: Đường mã một lần
32    U->>W: username
33    W->>API: POST /auth/otp/request
34    API->>O: Kiểm tra cooldown, trần theo tài khoản/giờ,<br/>trần theo địa chỉ/giờ (đếm trên login_
35    API-->>W: 200 GIỐNG HẾT nhau cho mọi trường hợp
36    API->>M: BackgroundTask gửi mã SAU khi trả lời
37    M-->>U: Email chứa mã 6 chữ số
38    U->>W: Nhập mã
39    W->>API: POST /auth/otp/verify
40    API->>O: So khớp PBKDF2 - sai thì trả vòng giả<br/>và tính vào failed_attempts của tài khoản
41    O->>DB: Challenge dùng một lần, chết ở OTP_MAX_ATTEMPTS
42    API-->>W: Phiên nhu đăng nhập thường
43    end
44
45    note over API,DB: Mọi request sau đó đọc lại permissions từ app_users.<br/>Thu hồi quyền có hiệu

```

C.0.19 19-outbox.mmd — Bàn giao bền vững qua outbox

```

1 %% Bàn giao bền vững - outbox_events theo từng profile
2 flowchart TB
3     TX["Transaction upload<br/>INSERT Case + Document + OutboxEvent PENDING<br/>COMMIT"] --> DISP[""]
4
5     DISP --> PROF["effective_queue_backend"]
6
7     PROF -->|"celery"| C1["apply_async lên broker Redis"]
8     C1 --> C2["Publish thành công = bàn giao bền vững"]
9     C2 --> C3["Đánh dấu row SENT ngay"]
10    C3 --> C4["Worker tiêu thụ hàng đợi documents"]
11
12    PROF -->|"inline"| I1["COMMIT lần thử trước<br/>để không giữ khóa suốt thời gian OCR"]
13    I1 --> I2["run_task_now(process_document)<br/>chạy ĐỒNG BỘ"]
14    I2 --> I3["Chỉ đánh dấu SENT SAU KHI chạy xong"]
15    I3 --> I4["Sập giữa chừng -> row vẫn PENDING<br/>-> sweeper trong lifespan nhật lại"]
16
17    FAIL["Task tiêu hết ngân sách retry của chính nó"] --> TERM["Đã chạy nhánh terminal:<br/>Case = "]
18    TERM --> SENT2["Vẫn tính là đã giao -> SENT<br/>nếu không sweeper sẽ lập vô tận"]
19

```

```

20 BEAT["Celery Beat<br/>dispatch_pending_outbox mỗi 30s"] --> RETRY["Thử lại các row còn PENDING"]
21 SWEEP["Sweeper in-process<br/>chỉ có ở profile local"] --> RETRY

```

C.0.20 20-deployment.mmd — Sơ đồ triển khai

```

1 %% Sơ đồ triển khai - profile docker
2 flowchart TB
3     subgraph HOST["Máy chủ / cụm container"]
4         subgraph NET["Mạng nội bộ compose"]
5             WEBC["web<br/>nginx + build SPA<br/>:3000"]
6             APIC["backend<br/>uvicorn cabqp.main:app<br/>:8000, chạy non-root"]
7             WKC["worker<br/>celery -Q celery,outbox,documents,batch"]
8             BTC["beat<br/>celery beat"]
9             MIG["migrate<br/>alembic upgrade head"]
10            SEEDC["seed<br/>registry snapshot + tài khoản"]
11            PGC["postgres:18"]
12            RDC["redis"]
13            MNC["quay.io/minio/minio"]
14            CAV["clamav"]
15            PRMC["prometheus :9090"]
16        end
17    end
18
19    USER["Người dùng"] -->|"HTTPS"| WAFN["WAF / Ingress<br/>TLS termination<br/>PHẢI nằm trong TRUSTED"]
20    WAFN --> WEBC
21    WEBC -->|"/api/v1 -> backend:8000"| APIC
22    APIC --> PGC
23    APIC --> RDC
24    APIC --> MNC
25    APIC --> CAV
26    WKC --> PGC
27    WKC --> RDC
28    WKC --> MNC
29    BTC --> RDC
30    MIG --> PGC
31    SEEDC --> PGC
32    PRMC -->|"scrape /metrics"| APIC
33
34    VOLS["Volume: postgres_data, minio_data,<br/>clamav_data, model_cache"]
35    PGC -.- VOLS
36    MNC -.- VOLS

```

C.0.21 21-benchmark-base-quality.mmd — Benchmark chất lượng tiền xử lý cấu hình Base

```

1 xychart-beta
2     title "Cấu hình Base: fixture tiền xử lý tiếng Việt"
3     x-axis ["XLSX", "DOCX", "PDF chữ", "Ảnh", "PDF quét"]
4     y-axis "Đúng ký tự (%)" 0 --> 100
5     bar [100.00, 100.00, 99.72, 98.86, 97.87]

```